Vídeo "SQL Full Course for Beginners (30 Hours) – From Zero to Hero" (Data with Baraa, 2025)

Beginner Level

1. Introduction

Database: like a container for storing data. But instead of just dumping files into folders, the database organizes the data, so it's easy to access, to manage and to search.

Why use database, can't we just use files like I do it personally? – The advantage is that you don't have to search and combine data manually: the database responds to you, using SQL. Also: spreadsheets and files break with big data; databases are more secure

a. What is SQL

SQL (Structured Query Language) is the language to talk to the database

b. Why learn SQL

1. You have to learn it to talk to the data (and most companies stores data in database, this is the standard way;

2. High demand (any data related jobs require SQL skills);

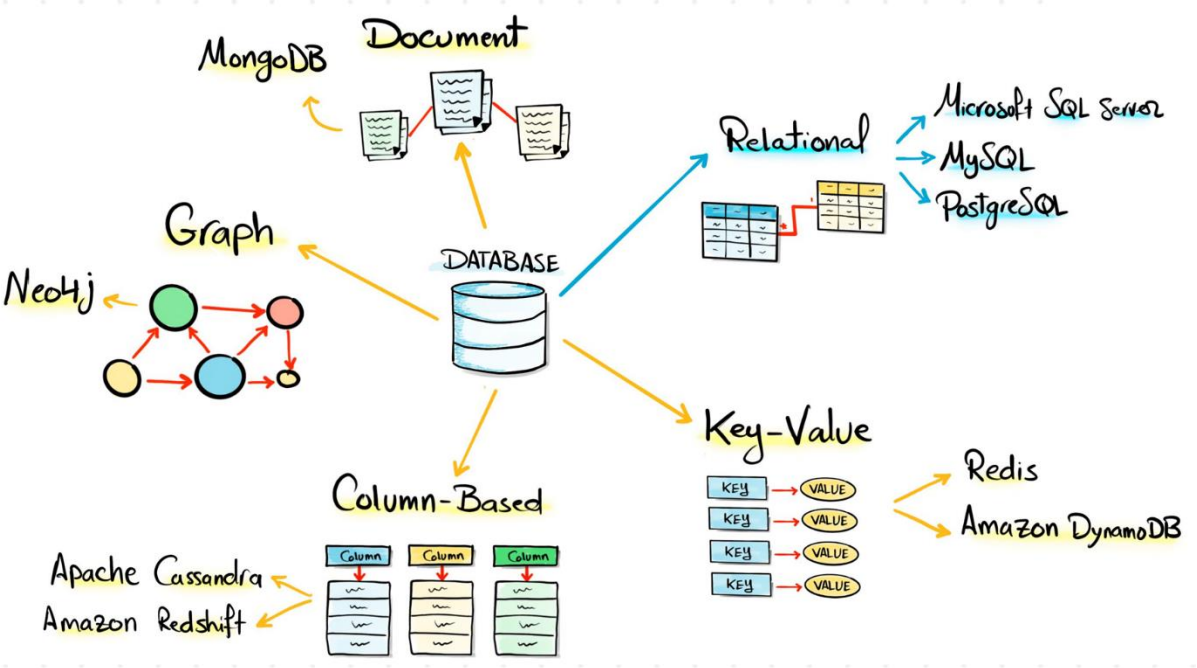
3. Industry standard (multiple modern data platforms and tools, like Power BI, tableau, Spark, KAFKA, Synapse… there will always be a section where you have to enter SQL code, so most of those vendors adopt SQL because it's the standard

c. What is database & types

Interactions with the database generate SQLs, so a software that manages all those requests is needed – DBMS (Database Management System): make the order of priority of SQL execution; manage the security (whether the SQL is allowed to be executed in the first place)

Hardware (SQL Server): where databases live. In real companies, we can't run that on our PC, because our PC is weak and goes offline – that's why we need a server (server inside the company or cloud services in order to run our database)

- Database types:



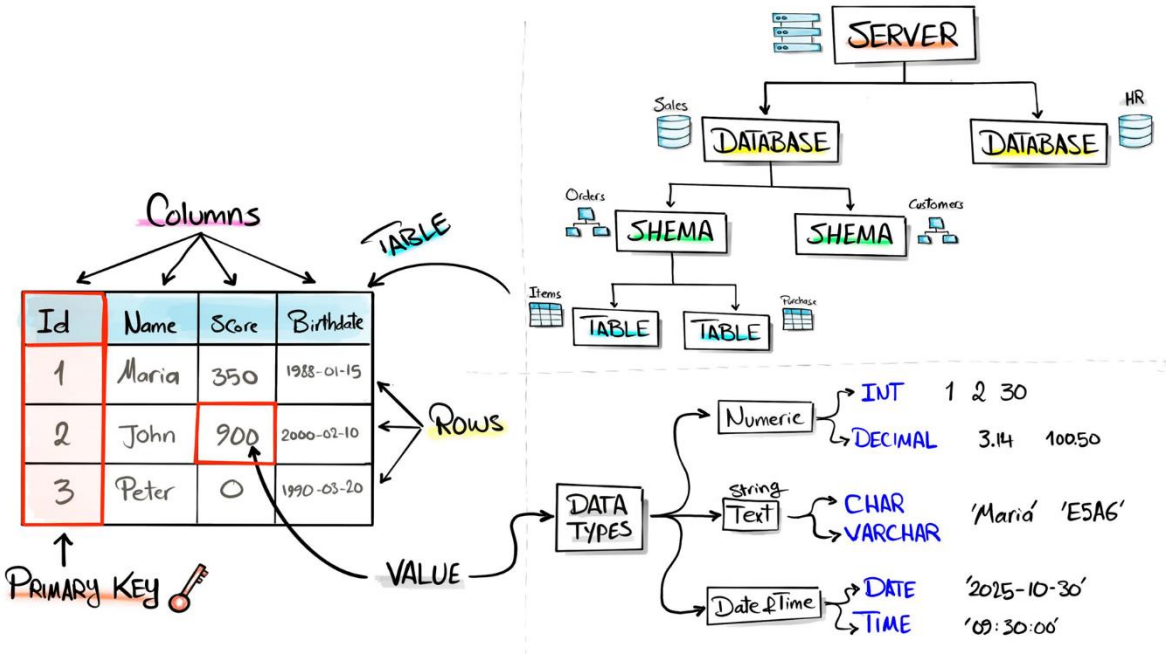
SQL database (the most famous one, the focus of this course):

- Relational database: it's like spreadsheets (table) that have a relationship between those tables to describe how they relate to each other. Examples: Microsoft SQL Server (focus of this course), MySQL, PostgreSQL (all those are SQL relational databases)

NoSQL databases:

- Key-Value: pairs of keys and values. Examples: Redis, Amazon DynamoDB
- Column-Based: instead of grouping the data by the rows, this type of database group the data into columns (it's a very advanced type of database in order to handle huge amount of data, with the main purpose to search for data). Examples: Apache Cassandra, Amazon Redshift
- Graph: the main focus is the relationship between objects. Example: Neo4j
- Document: the date is stored as entire documents, in which what's important is to fit everything in one page, rather than the structure of the data. Example: MongoDB

- Database hierarchy:



The starting point is the server (a powerful PC, where the database lives); inside, we can have multiple databases (example: one for the sales, other for the HR); in each database, we can have multiple schemas – categories or logical containers used to group up related objects, helping to organize the tables and objects in the database (example: one schema with the tables of the orders and another group of tables with the schema customers); inside of the schema, you can have multiple objects, like tables (example: tables “items” and “purchase”) – tables are like spreadsheets, organizes data into columns/fields (define the data stored inside by type) and rows/records (where actually the data is stored). In the example of table, each column is a type of data of the customer and each row represent one customer.

In each table there is one very important – always very important to have – column called “primary key”, one unique identifier for each row (and we use it for different purposes in order to combine it with another table, in order to identify quickly one customer). The overlapping between the columns and the rows are the single value cells (each value stores a specific data type, as int/Decimal/char/varchar/date/time)

d. 3 types of SQL commands

With an empty database, the first thing you have to do is to write an SQL with the command CREATE in order to create a brand-new empty table in the database. What you've done is you defined something new. We call this type of commands the Data Definition Language (DDL) – CREATE to create something new; ALTER to edit something that already exists; DROP to delete something (first family of commands)

With an empty table, we need data. Let's say we have a website or an application that is generating a lot of data. Now in order for this application to move the data inside our new table, we must use the SQL command INSERT to add new data inside the table. This type of commands we call Data Manipulation Language (DML) – INSERT to add new data; UPDATE to update an existing data; DELETE to delete data from your table

With data inside the table, we can start asking questions. Let's say you have an analytical question about your data: you have to write something called SQL query and inside it you use the command SELECT (the whole thing we call it a query) – you send a query to the database, you have a question, and the database can return the result, the data answering your query. We call this Data Query Language (DQL, {pode ser tratado como uma subdivisão de DML})

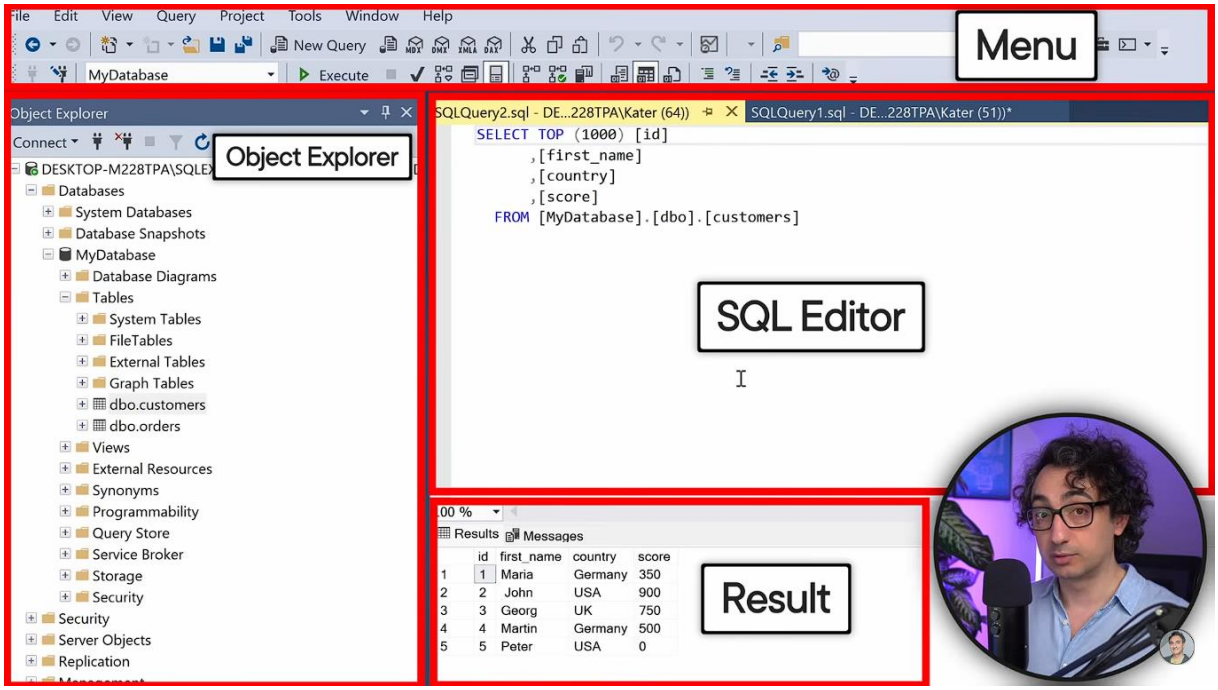
– We will spend most of our time learning how to write the correct query for the correct answer

e. Setup your environment

i. Download & install SQL Server

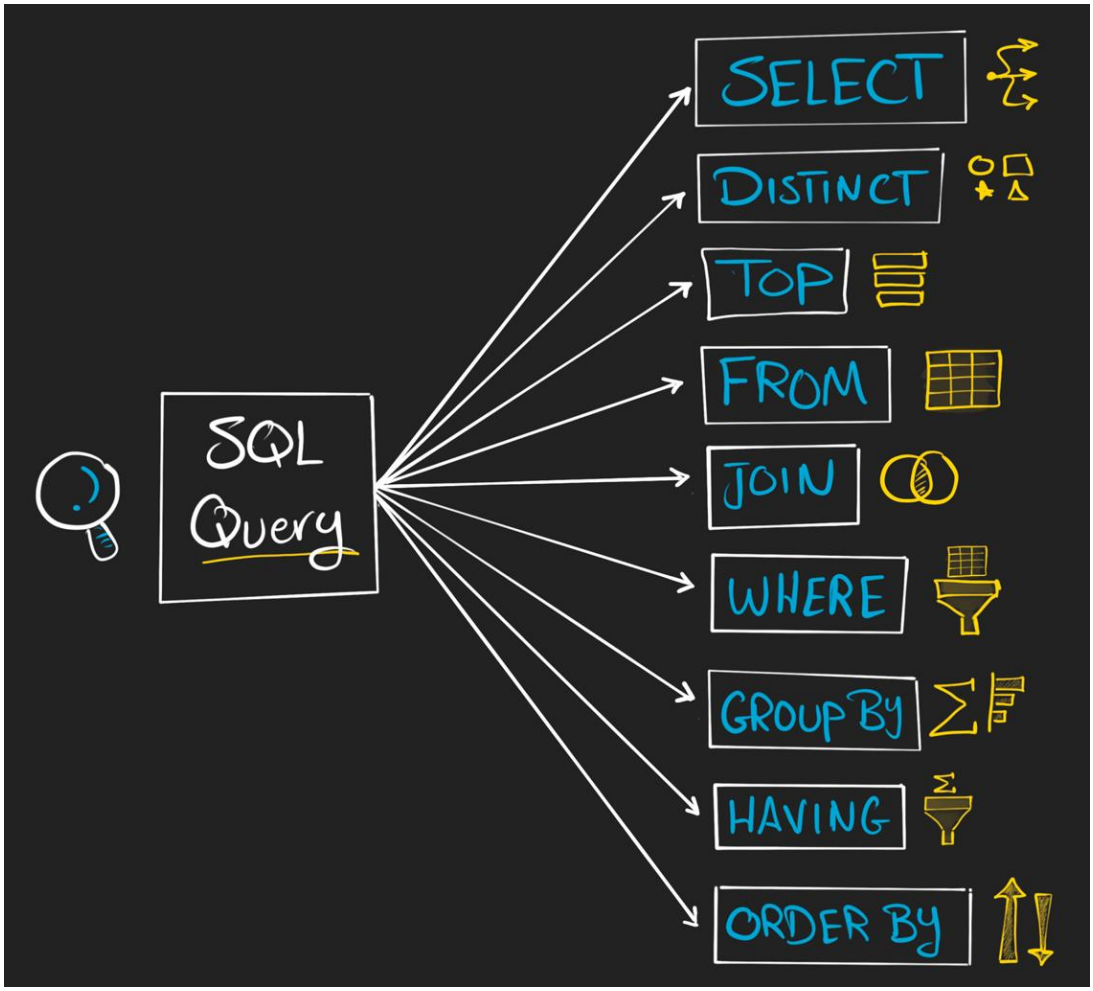
ii. Datasets and create databases

iii. GIT repo



2. Query data (SELECT)

Clauses (cláusulas para consulta):



- 1º connect to the correct database, otherwise, your queries won't work (clique em "Bancos de Dados Disponíveis" ou comando USE)

- Comments: `-- Comentário inline`

```
/* Comentário
em bloco */
```

- Values that contains characters must be inside ''

- AS = Alias → Temporary name to a table (a convenção é não usar AS) or to a column (a convenção é usar AS)

- Highlight & execute: if there's highlight, SSMS executes only what's highlighted

a. SELECT e FROM

`SELECT` → Choose columns

`FROM` → Specify the table of the data

In each query we start always with a `SELECT`

- To retrieve all customer data:

```
SELECT *
FROM customers
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0

- To retrieve selected data:

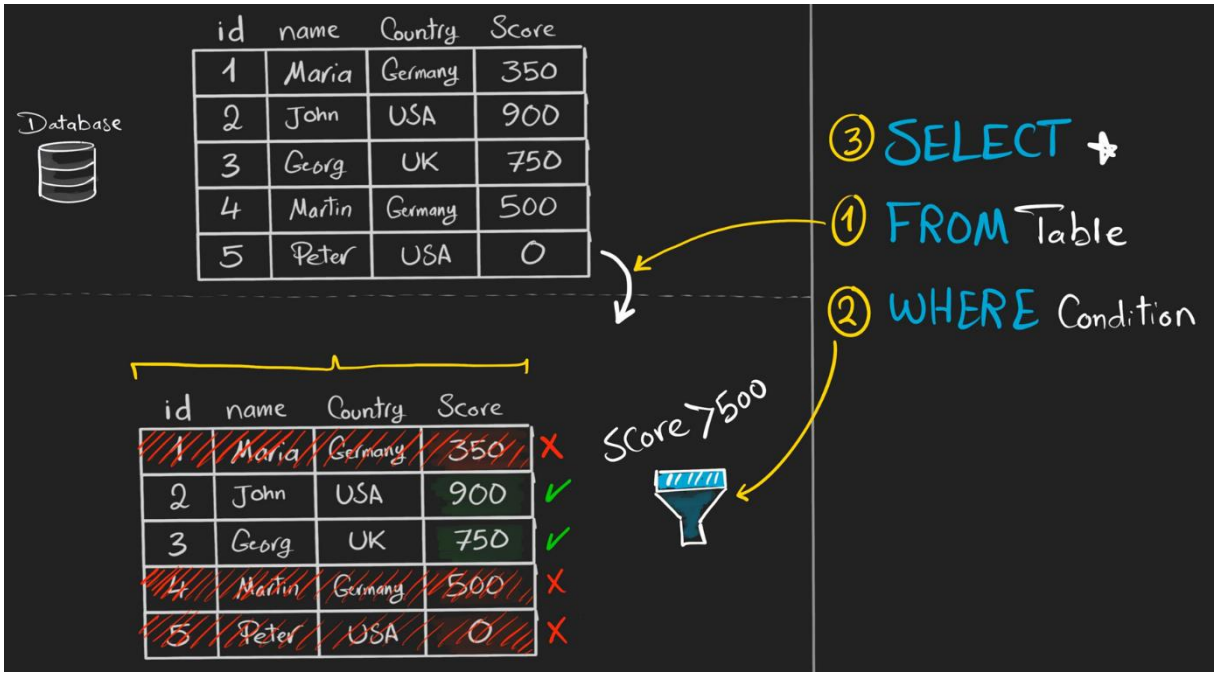
```
SELECT
      score,
      first_name,
      country
FROM customers
```

	score	first_name	country
1	350	Maria	Germany
2	900	John	USA
3	750	Georg	UK
4	500	Martin	Germany
5	0	Peter	USA

b. WHERE

Filter the data (before aggregation), based on a condition

```
SELECT
	first_name,
	country
FROM customers
WHERE score > 500
```




```
SELECT *
FROM customers
WHERE score != 0
```

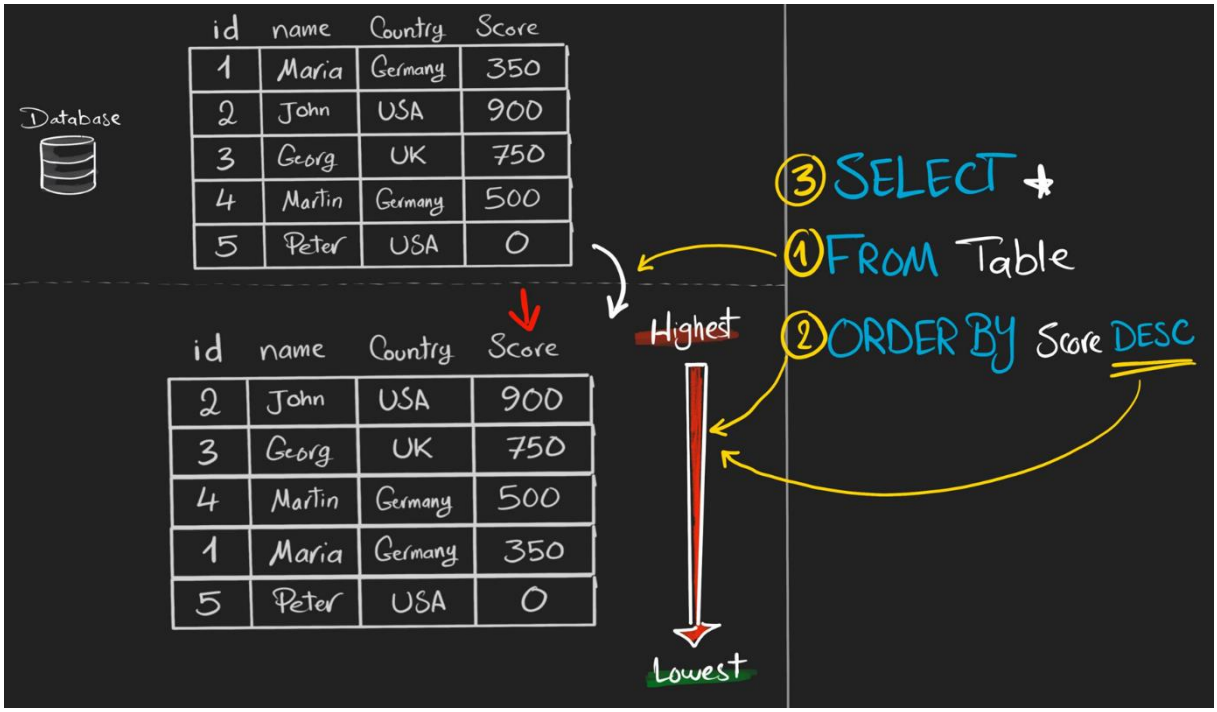
	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500

```
SELECT
	score,
	first_name
FROM customers
WHERE country = 'Germany'
```

	score	first_name
1	350	Maria
2	500	Martin

c. ORDER BY

Sort the data

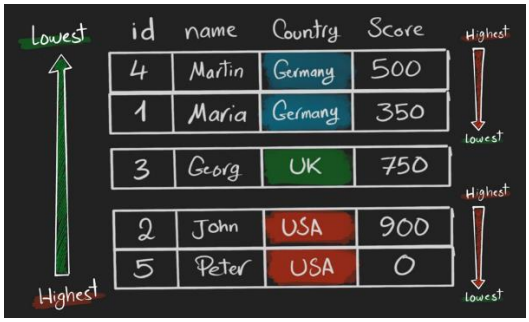



```sql
SELECT *
FROM customers
ORDER BY score ASC -- ASC is default, but is better to specify for clarity
```

	id	first_name	country	score
1	5	Peter	USA	0
2	1	Maria	Germany	350
3	4	Martin	Germany	500
4	3	Georg	UK	750
5	2	John	USA	900

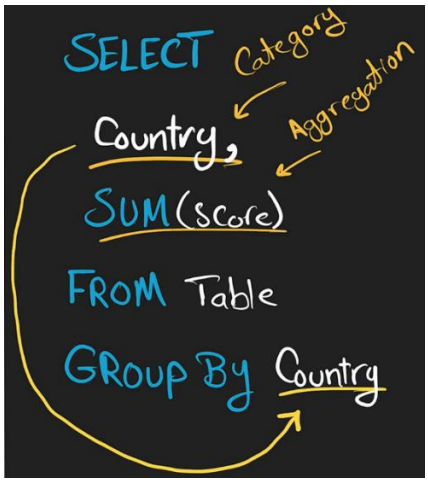
- Nested sorting:

```sql
SELECT *
FROM customers
ORDER BY
      country ASC,
      score DESC
```

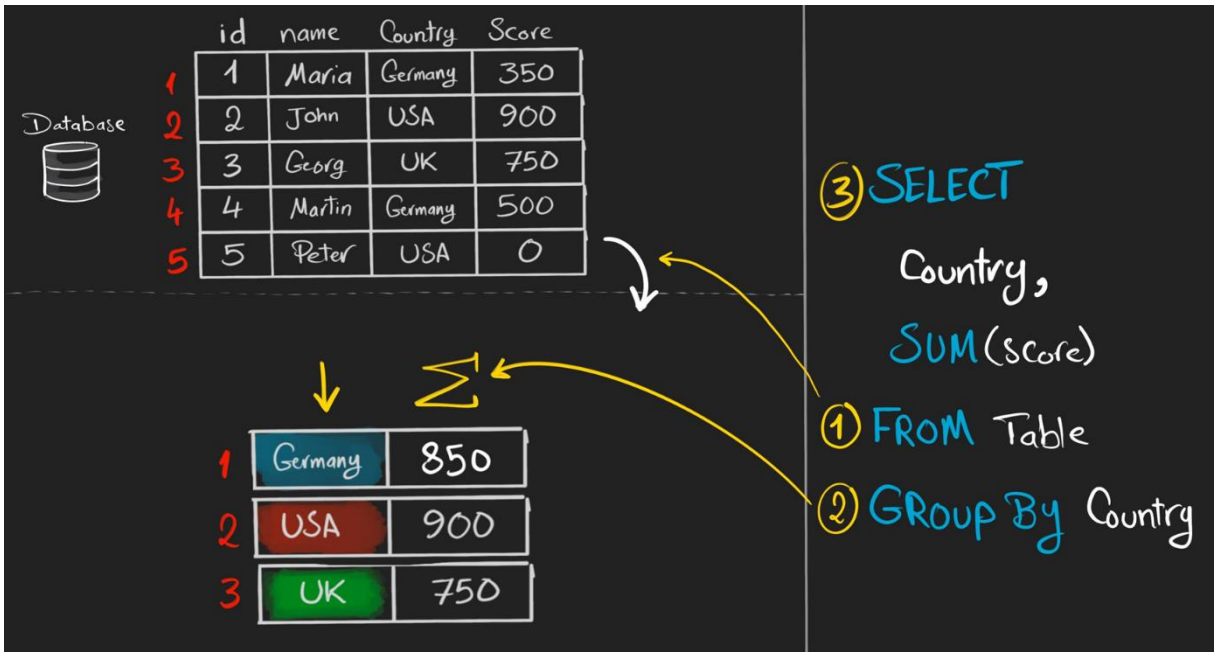


d. GROUP BY

Combina linhas com o mesmo valor em uma ou mais colunas, agregando os valores das outras colunas incluídas (com funções como SUM(), COUNT(), AVG(), MAX()...)




```sql
SELECT
	country, -- value to group the data by
	SUM(score) /* value to be aggregated */ AS total_score
FROM customers
GROUP BY country -- group the data by the selected value
```



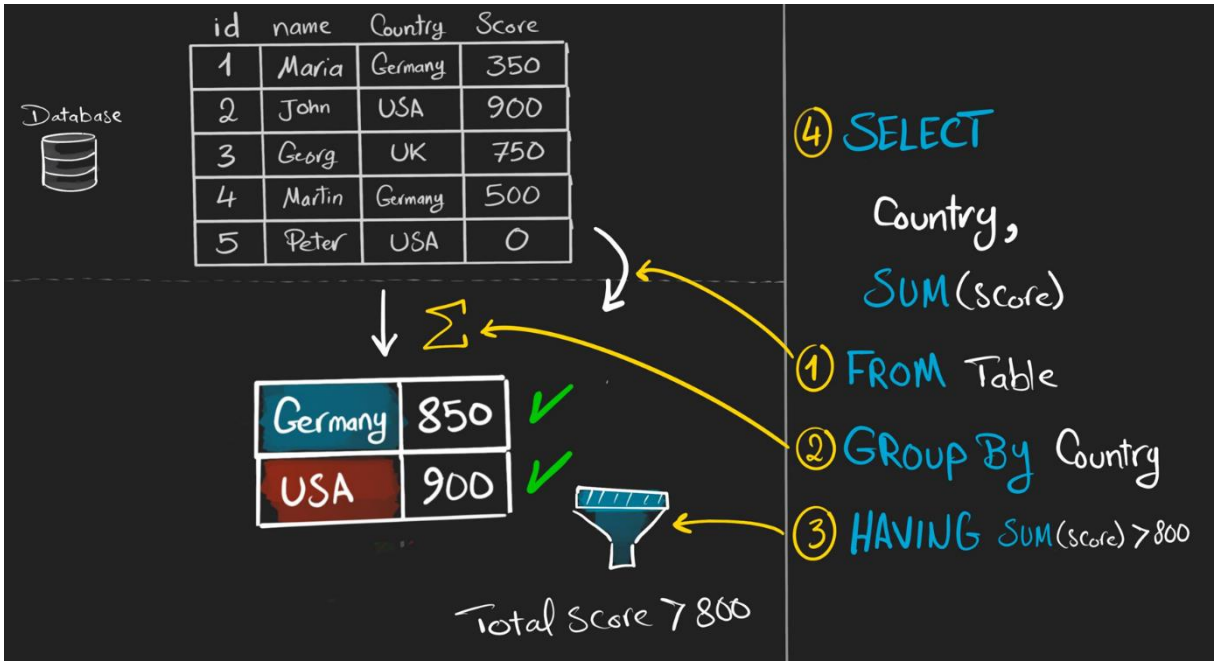
All columns in the SELECT must be either aggregated or included in the GROUP BY

e. HAVING

Filter aggregated data (after aggregation), based on a condition

Can only be used with GROUP BY

```
SELECT
	country,
	COUNT(*) AS total_customers,
	SUM(score) AS total_score
FROM customers
GROUP BY country
HAVING SUM(score) > 800
```



f. DISTINCT

Remove duplicates

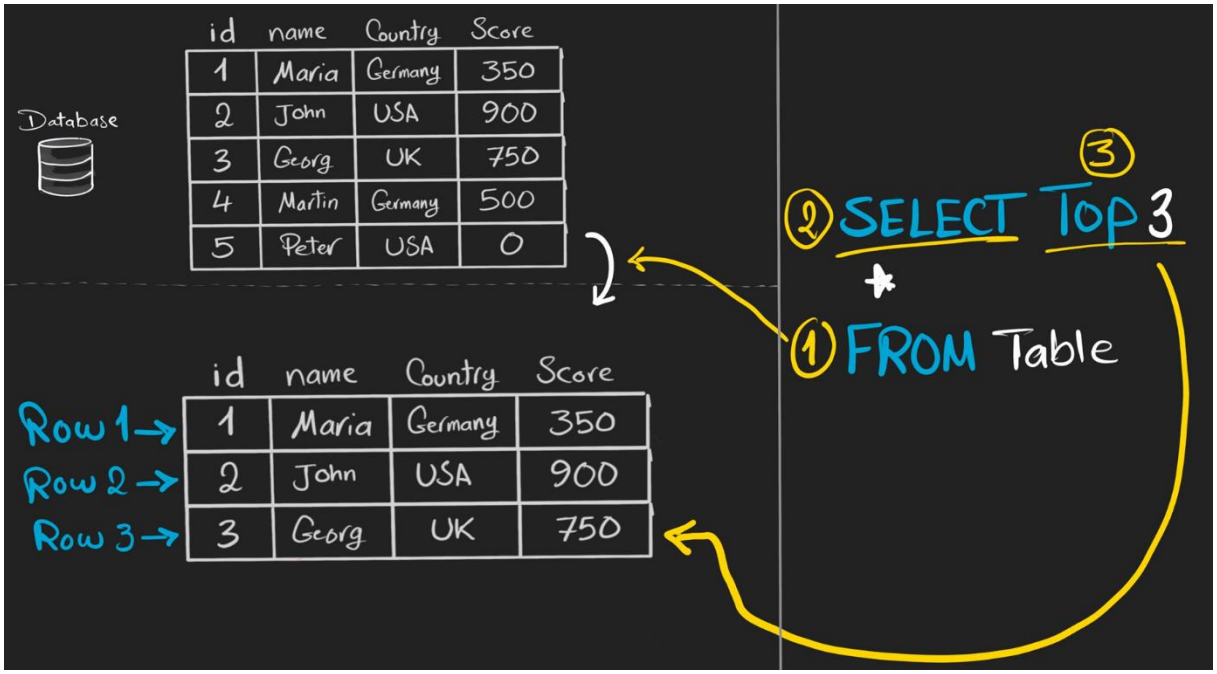
```
SELECT DISTINCT
	country
FROM customers
```

	country
1	Germany
2	UK
3	USA

Don’t use DISTINCT unless it’s necessary, cause it can slow down your query

g. TOP (LIMIT)

Restrict the number of rows returned in the result



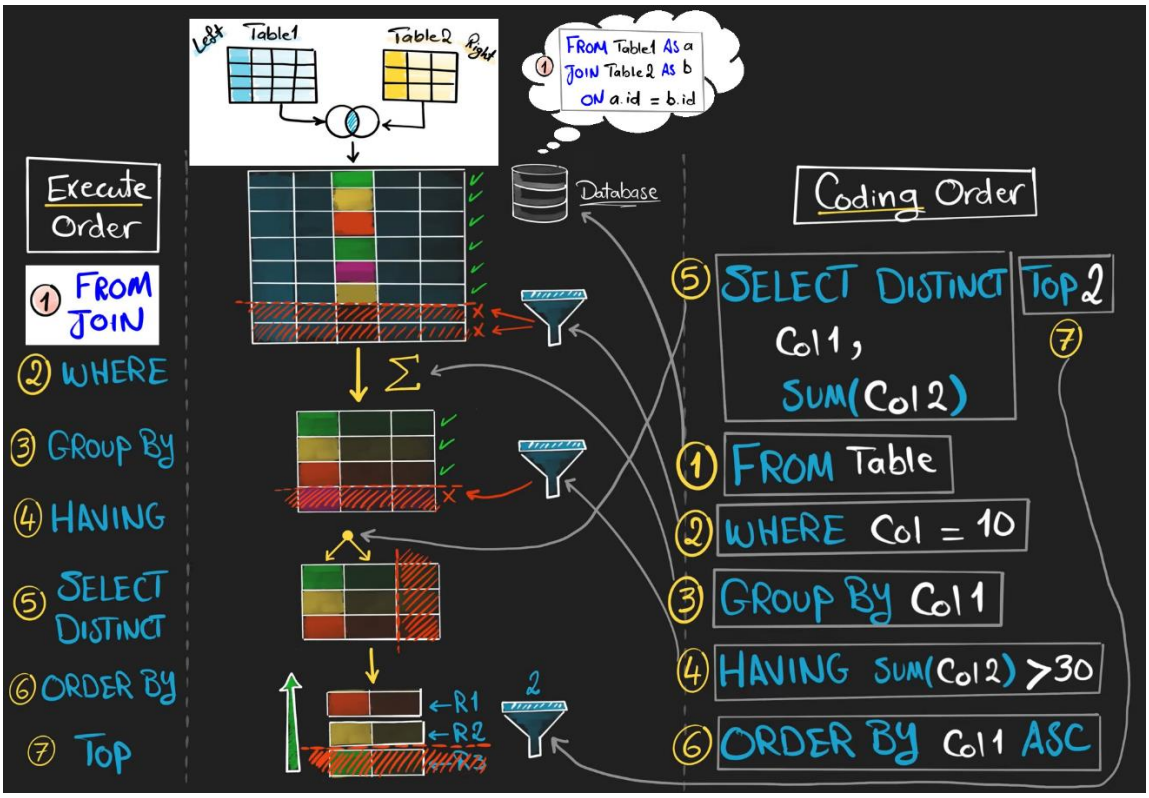

```
SELECT TOP 3 *
FROM customers
ORDER BY score DESC
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750

```
SELECT TOP 2 *
FROM orders
ORDER BY order_date DESC
```

	order_id	customer_id	order_date	sales
1	1004	6	2021-08-31	10
2	1003	3	2021-06-18	20

✓ Execution order x coding order:



✓ Multi-queries:

```
SELECT *
FROM customers;

SELECT *
FROM orders;
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0

	order_id	customer_id	order_date	sales
1	1001	1	2021-01-11	35
2	1002	2	2021-04-05	15
3	1003	3	2021-06-18	20
4	1004	6	2021-08-31	10

✓ Static (fixed) values:

```
SELECT 123 AS static_number
```

	static_number
1	123

3. Data Definition Language (DDL)

With an empty database, you have to define the structure of the data

a. CREATE

Create a new object inside the database (for example, a table)

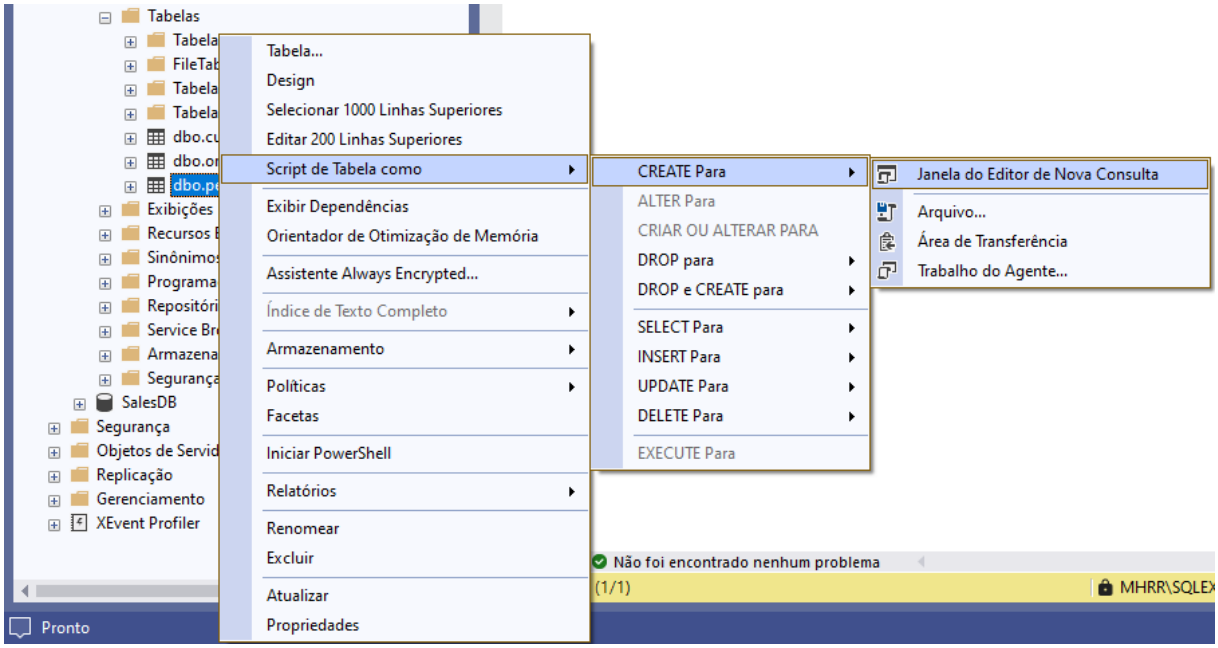
```
CREATE TABLE persons (
	id INT, -- por ser primary key, não pode ter valores NULL nem repetidos
	person_name VARCHAR(50) NOT NULL,
	birth_date DATE,
	phone VARCHAR(15) NOT NULL,
	CONSTRAINT primary_key_persons PRIMARY KEY (id)
)
```

Each database table must have a primary key, for the table have an integrity and be connectable to other tables

```
SELECT * FROM persons
```

id	person_name	birth_date	phone

It’s important to save the DDL in a SQL script. But, if you lost it, to recover:



b. ALTER

With a table, you can edit the definition of the table

```
ALTER TABLE persons
ADD email VARCHAR(50) NOT NULL -- new columns are appended at the end by default
```

```
SELECT * FROM persons
```

	id	person_name	birth_date	phone	email

To position the new column differently, the only way is to DROP the table and CREATE another one

```
ALTER TABLE persons
DROP COLUMN phone -- by deleting a column, the data inside is deleted as well
```

```
SELECT * FROM persons
```

	id	person_name	birth_date	email

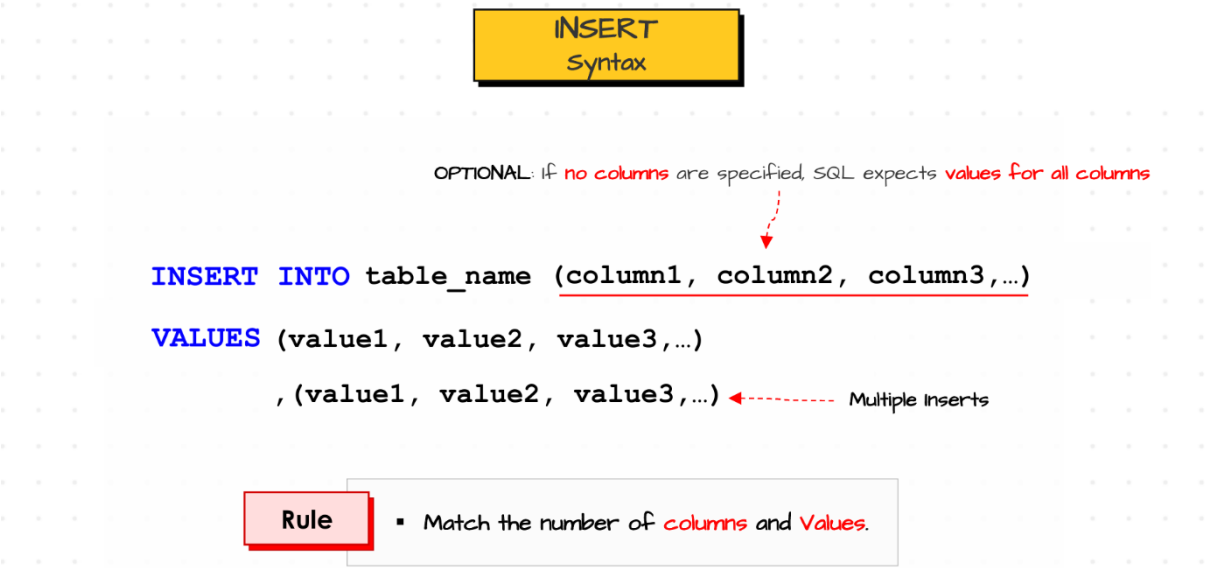
c. DROP

Remove completely an object from the database, with the data inside it

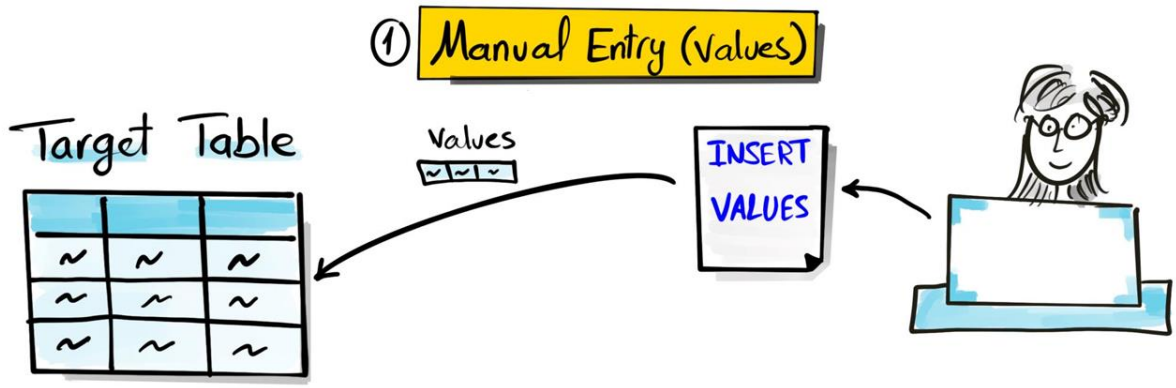
```
DROP TABLE persons
```

4. Data Manipulation Language (DML)

a. INSERT



i. Manual entry




```sql
INSERT INTO customers (id, first_name, country, score) -- always list columns explicitly for clarity and maintainability
VALUES
	(6, 'Anna', 'USA', NULL),
	(7, 'Sam', NULL, 100)
```

```sql
SELECT * FROM customers
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0
6	6	Anna	USA	NULL
7	7	Sam	NULL	100

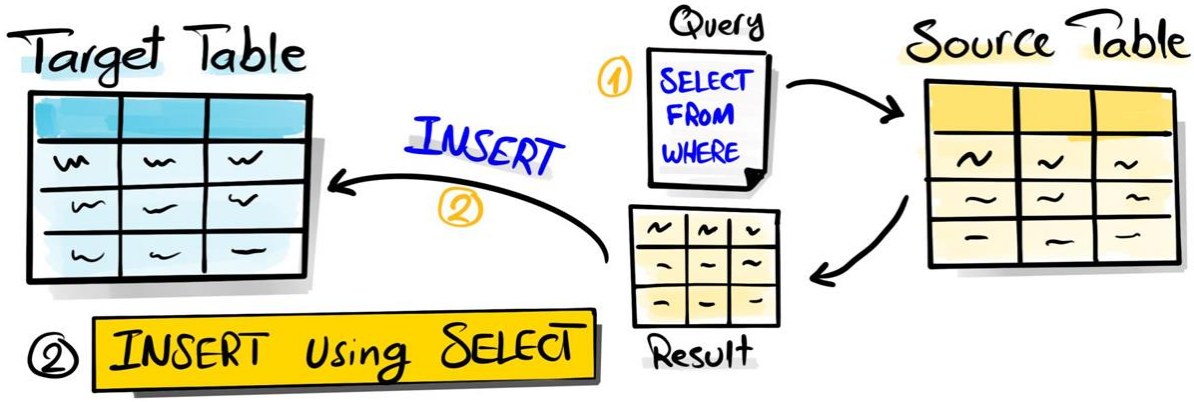
- Columns not included in INSERT become NULL (unless a default or constraint exists):

```
INSERT INTO customers (id, first_name)
VALUES (8, 'Max')
```

```
SELECT * FROM customers
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0
6	6	Anna	USA	NULL
7	7	Sam	NULL	100
8	8	Max	NULL	NULL

ii. Using SELECT




```
INSERT INTO persons (id, person_name, birth_date, phone)
SELECT
	id,
	first_name,
	NULL,
	'Unknown'
FROM customers
```

```
SELECT * FROM persons
```

	id	person_name	birth_date	phone

→

	id	person_name	birth_date	phone
1	1	Maria	NULL	Unknown
2	2	John	NULL	Unknown
3	3	Georg	NULL	Unknown
4	4	Martin	NULL	Unknown
5	5	Peter	NULL	Unknown
6	6	Anna	NULL	Unknown
7	7	Sam	NULL	Unknown
8	8	Max	NULL	Unknown

b. UPDATE

UPDATE Syntax

```
UPDATE table_name
   SET  column1 = value1,
        column2 = value2
 WHERE <condition>
```

NOTE ▪ Always use WHERE to avoid UPDATING all rows unintentionally

```
UPDATE customers
SET score = 0
WHERE id = 6 -- without WHERE, all rows would be updated
```

```
SELECT * FROM customers -- best practice: check with SELECT before running UPDATE
WHERE id = 6
```

	id	first_name	country	score
1	6	Anna	USA	NULL

→

	id	first_name	country	score
1	6	Anna	USA	0

- UPDATE multiple specific values:

```
UPDATE customers
SET
      score = 0,
      country = 'UK'
WHERE id = 8
```

```
SELECT * FROM customers
WHERE id = 8
```

	id	first_name	country	score
1	8	Max	NULL	NULL

→

	id	first_name	country	score
1	8	Max	UK	0

- UPDATE multiple values based on a condition:

```
UPDATE customers
SET country = 'Brasil'
WHERE country IS NULL
```

```
SELECT * FROM customers
WHERE country IS NULL
```

	id	first_name	country	score
1	7	Sam	NULL	100

→

```
SELECT * FROM customers
WHERE country = 'Brasil'
```

	id	first_name	country	score
1	7	Sam	Brasil	100

c. DELETE

DELETE Syntax

```
DELETE FROM table_name
WHERE <condition>
```

NOTE ▪ Always use WHERE to avoid DELETING all rows unintentionally

```
DELETE FROM customers
WHERE id > 5
```

```
SELECT * FROM customers
```

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0
6	6	Anna	USA	0
7	7	Sam	Brasil	100
8	8	Max	UK	0

→

	id	first_name	country	score
1	1	Maria	Germany	350
2	2	John	USA	900
3	3	Georg	UK	750
4	4	Martin	Germany	500
5	5	Peter	USA	0

- To DELETE all data from a table:

```
DELETE FROM persons
```

ou

```
TRUNCATE TABLE persons -- way faster than DELETE (clears the whole table at once without checking or logging)
```

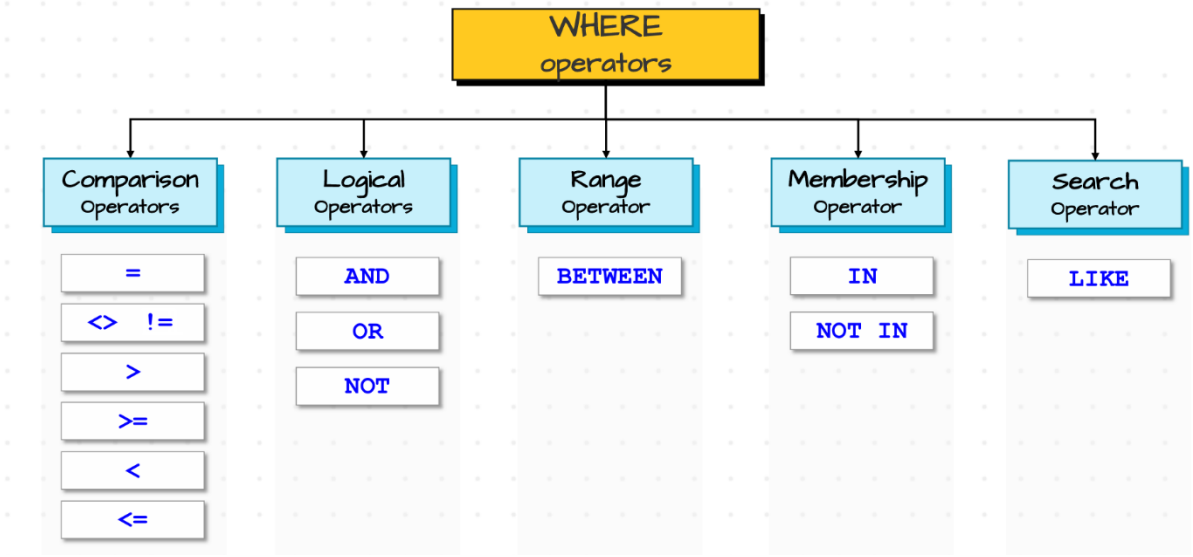
	id	person_name	birth_date	phone
1	1	Maria	NULL	Unknown
2	2	John	NULL	Unknown
3	3	Georg	NULL	Unknown
4	4	Martin	NULL	Unknown
5	5	Peter	NULL	Unknown
6	6	Anna	NULL	Unknown
7	7	Sam	NULL	Unknown
8	8	Max	NULL	Unknown

→

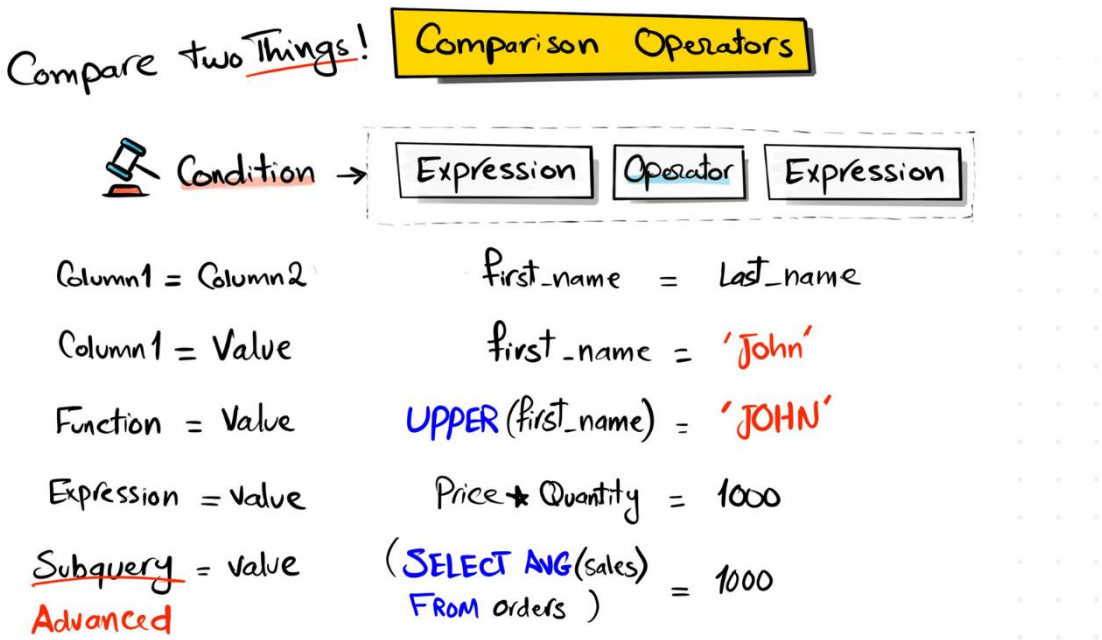
	id	person_name	birth_date	phone

Intermediate Level

5. Filtering data



a. Comparison operators



b. Logical operators

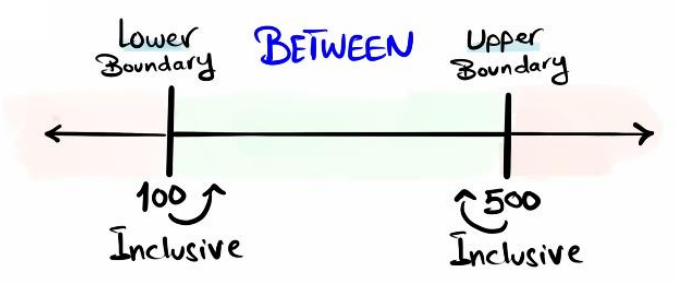
```sql
SELECT * FROM customers
WHERE country = 'USA' AND score > 500;

SELECT * FROM customers
WHERE country = 'USA' OR score > 500;

SELECT * FROM customers
WHERE NOT score <= 500;
```

IN	Checks if a value matches any value in a list
NOT IN	Checks if a value does not matches any value in a list
EXISTS	Checks if subquery returns any rows
NOT EXISTS	Checks if subquery returns no rows
ANY	Returns true if a value matches any value in a list.
ALL	Returns true if a value matches all values in a list.

```sql
SELECT * FROM customers
WHERE country IN ('Germany', 'USA')
```

c. Range operator

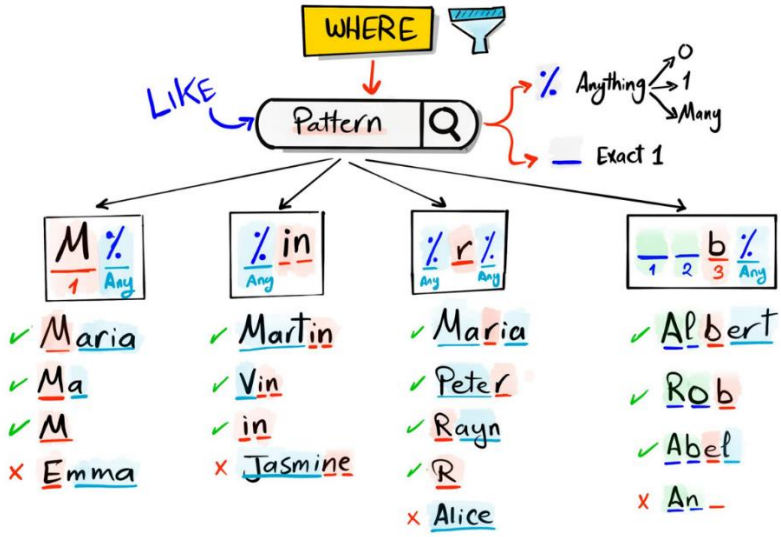
Check if a value is within a range




```sql
SELECT * FROM customers
WHERE score BETWEEN 100 AND 500 -- equivale a WHERE score >= 100 AND score <= 500
```

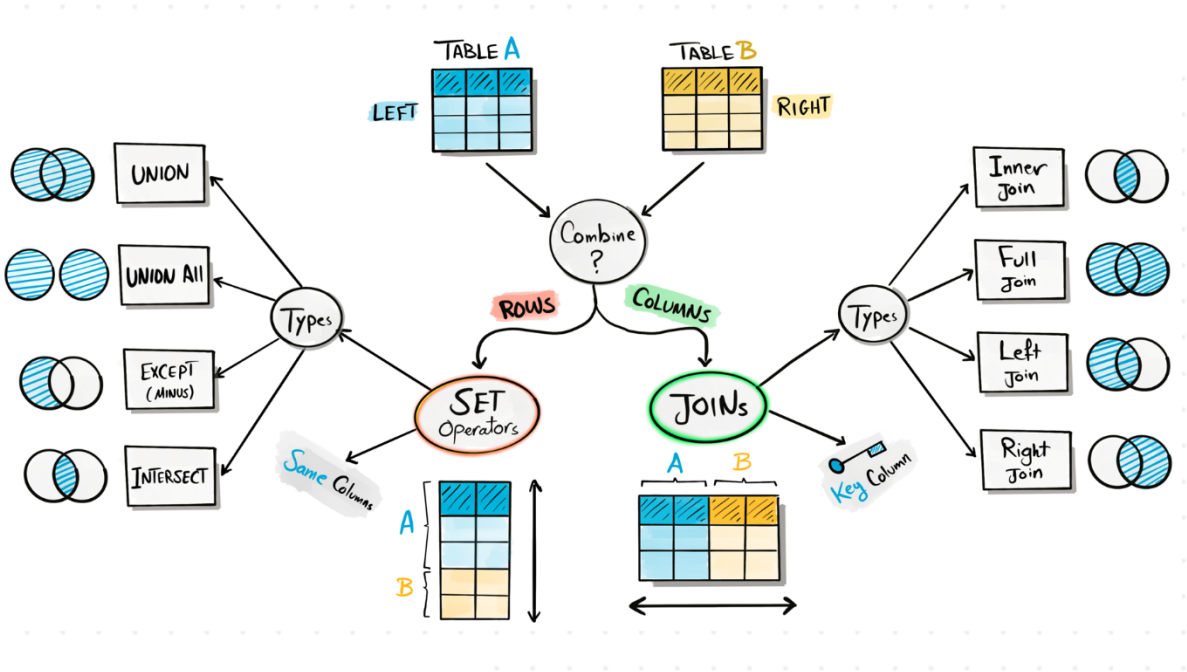
d. Search operator

Search for a pattern in text



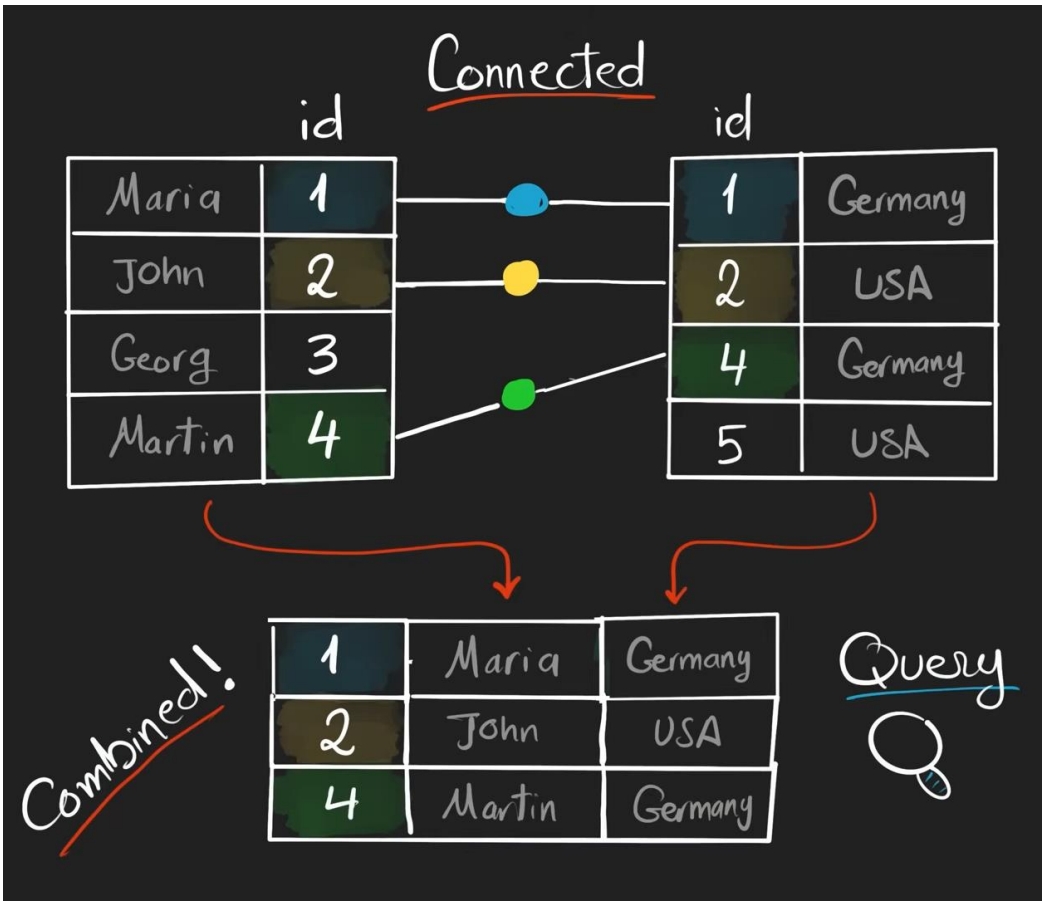

```sql
SELECT * FROM customers
WHERE first_name LIKE '__r%'
```

6. Combining data



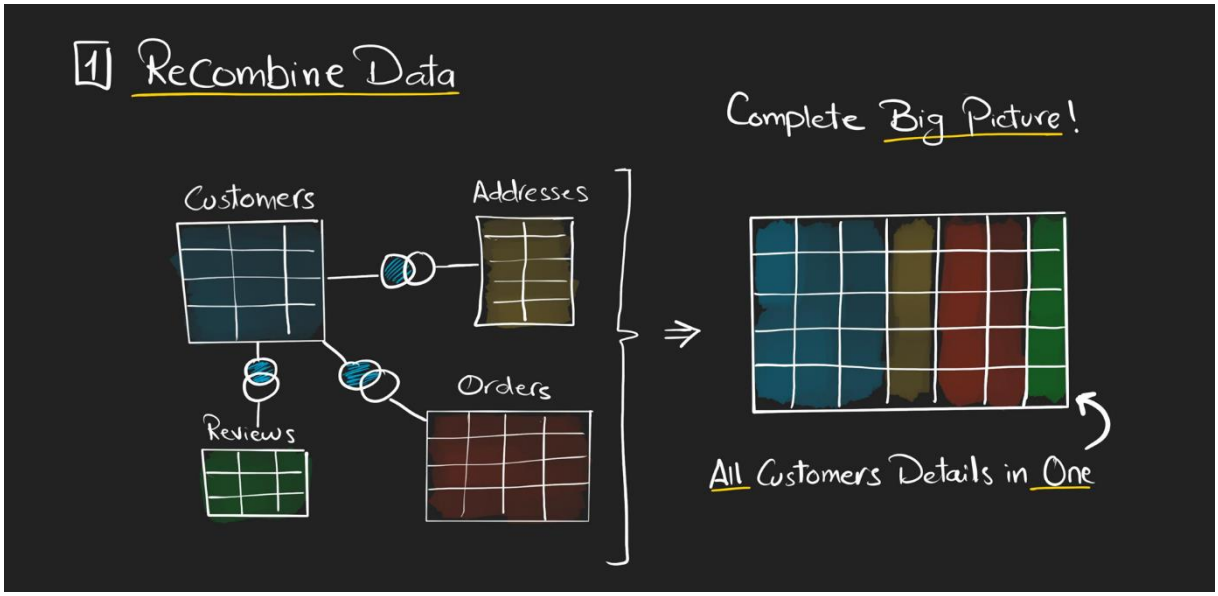
a. Joining data

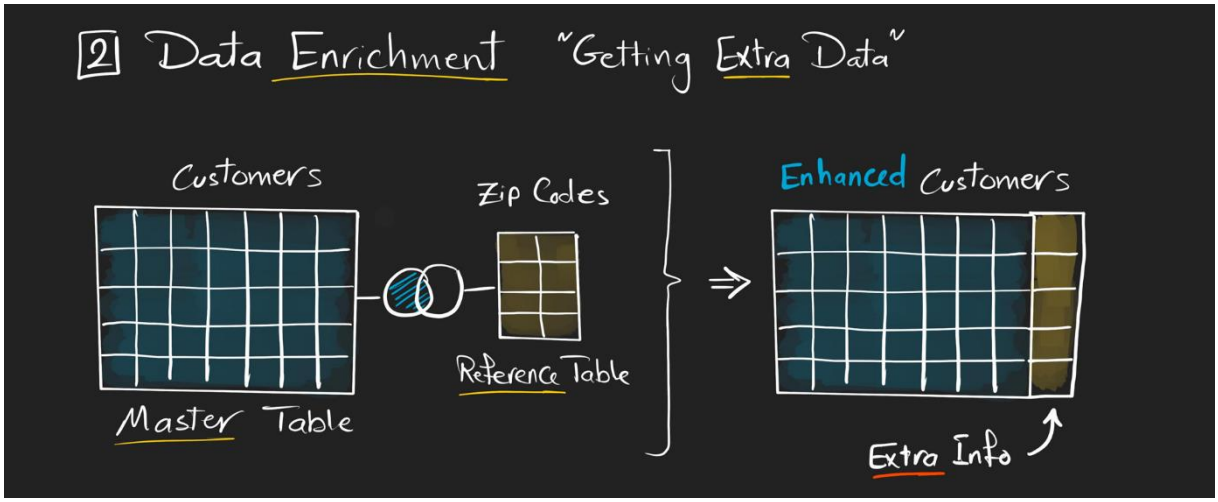
To get all information in one query, first we have to connect the tables – and for that, we need a key (a column that exists on the left and on the right)



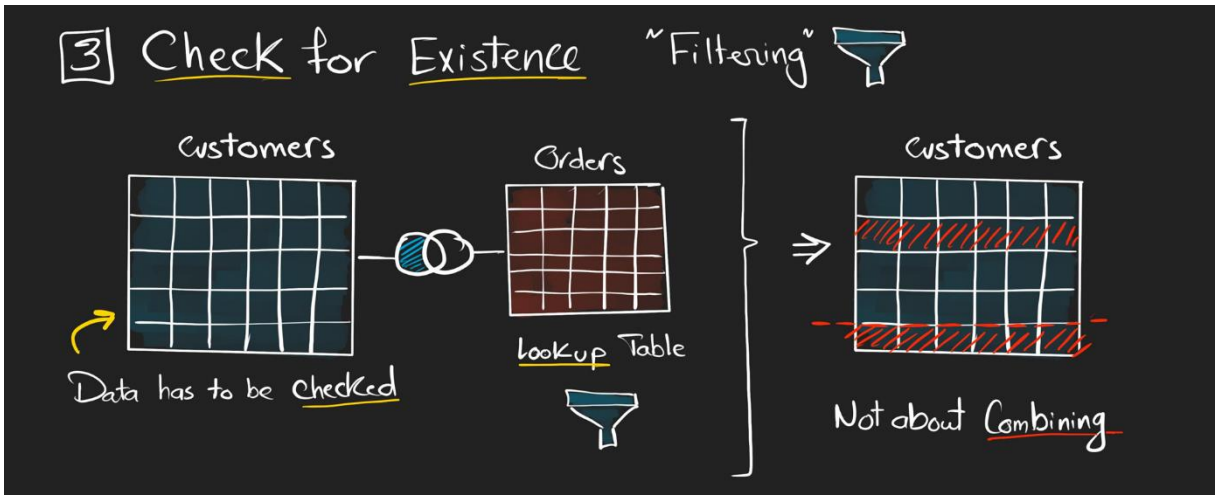
- When to use:

1. The most important reason:

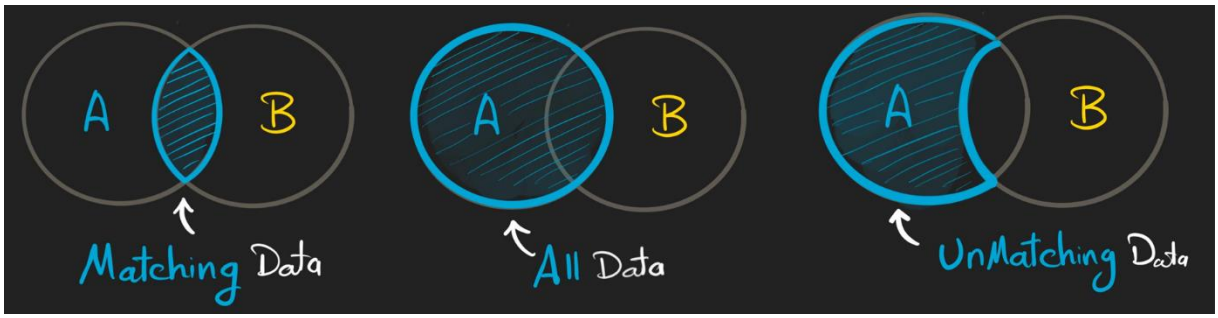


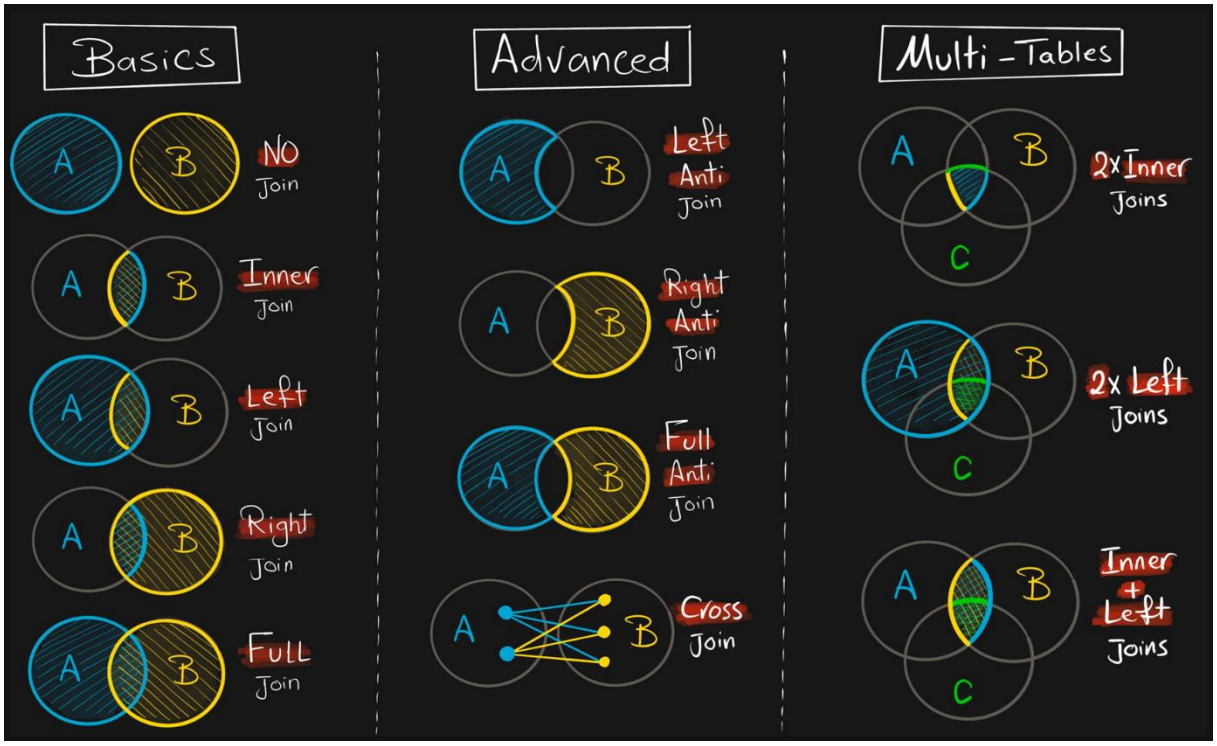


3. Check for existence of data in another table (in the example: to eliminate in the main table customers that didn't order anything, comparing with another table)

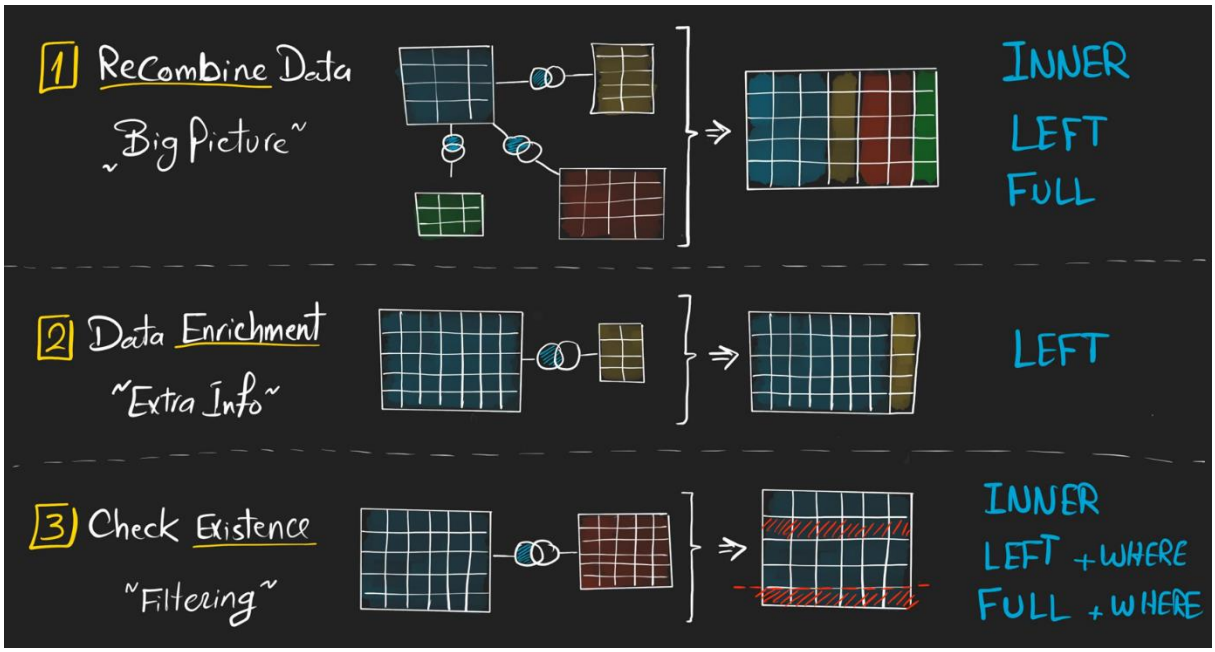


- JOIN types:

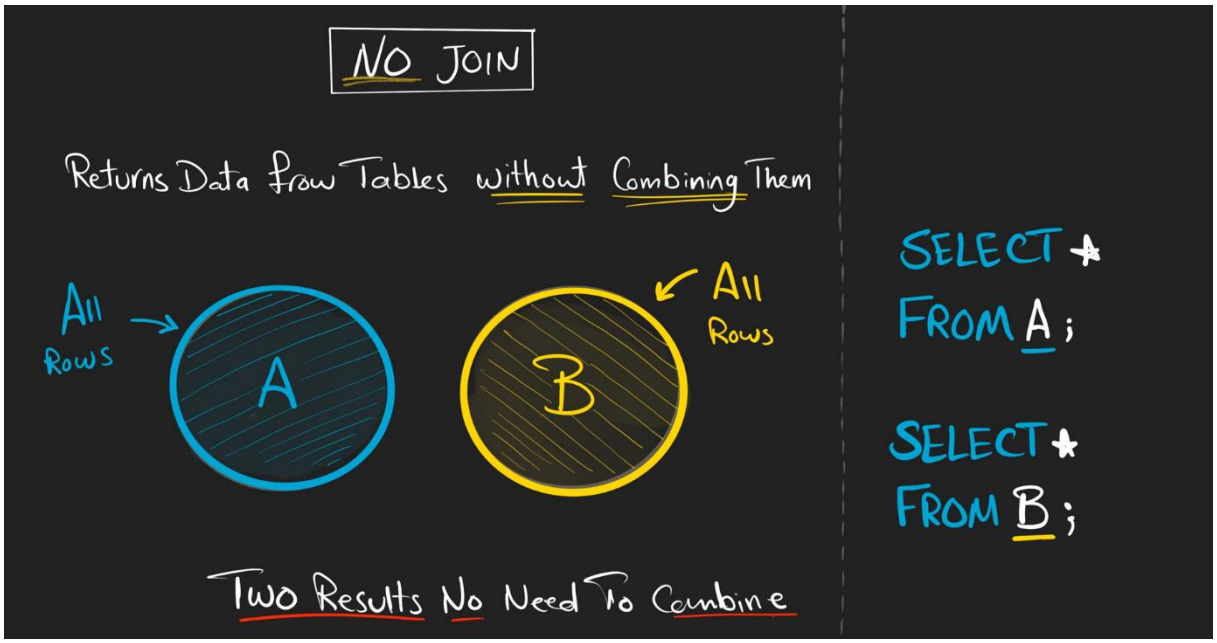

A
B
Matching Data
A
B
All Data
A
B
UnMatching Data


Basics
NO Join
Inner Join
Left Join
Right Join
FULL Join
Advanced
Left Anti Join
Right Anti Join
Full Anti Join
Cross Join
Multi-Tables
2x Inner Joins
2x Left Joins
Inner + Left Joins
A
B
C

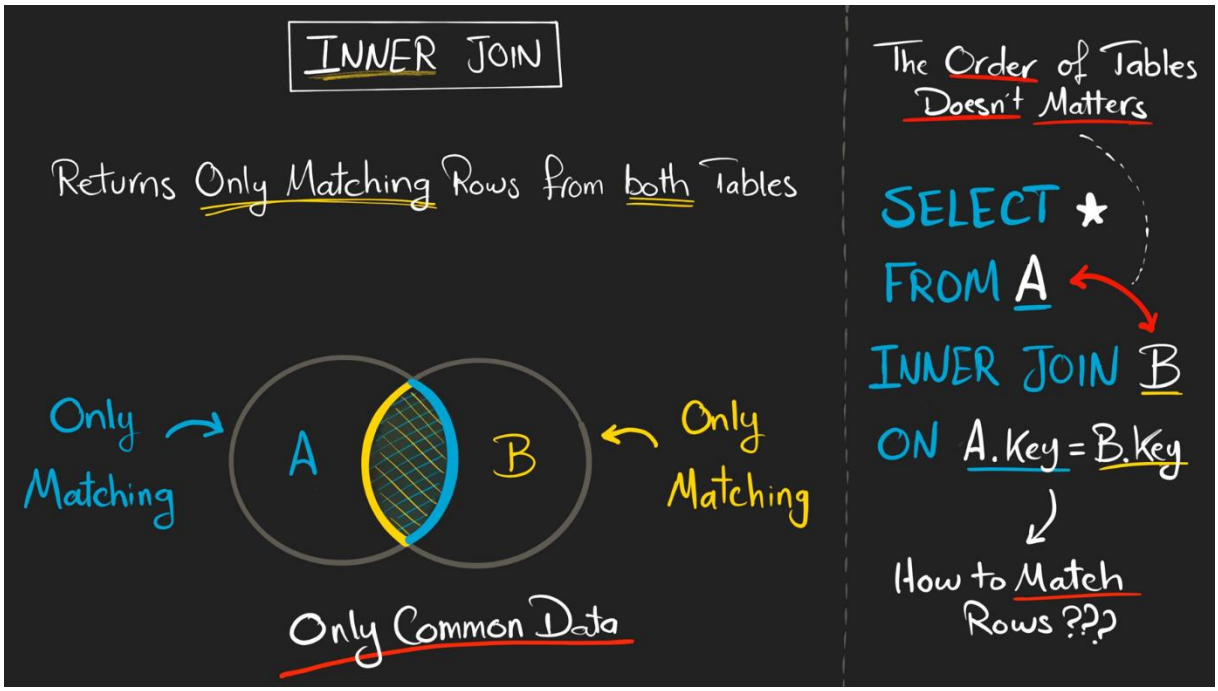
i. Basic Joins



- No JOIN:



1. INNER JOIN




```sql
SELECT
	c.id, -- to avoid ambiguity, reference the table name before the column
	c.first_name,
	o.order_id,
	o.sales
FROM customers c -- alias to simplify the references (AS implícito)
INNER JOIN orders o -- INNER is the default JOIN, but it's better to specify
ON c.id = o.customer_id -- condition of the JOIN
```

customers

id	first_name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	USA	750
4	Martin	Germany	500
5	Peter	USA	0

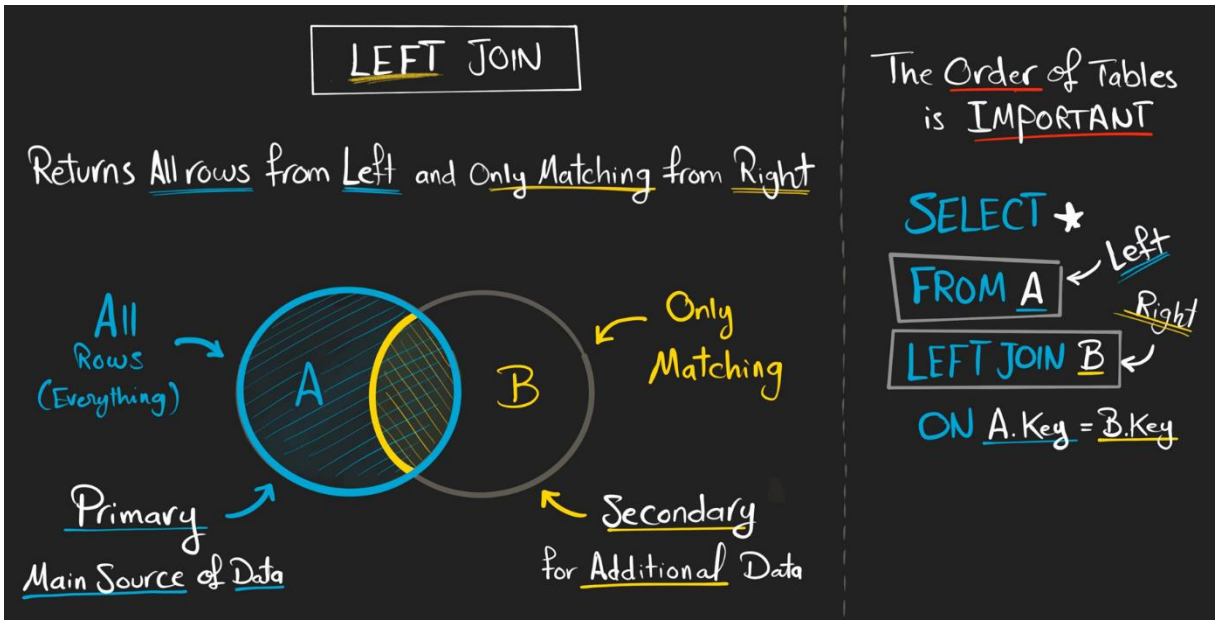
orders

order_id	order_date	sales	customer_id
1001	2021-01-11	35	1
1002	2021-04-05	15	2
1003	2021-06-18	20	3
1004	2021-08-31	10	6

RESULT

id	first_name	order_id	sales
1	Maria	1001	35
2	John	1002	15
3	Georg	1003	20

2. LEFT JOIN




```sql
SELECT
	c.id,
	c.first_name,
	o.order_id,
	o.sales
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id
```

customers

id	first_name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	USA	750
4	Martin	Germany	500
5	Peter	USA	0

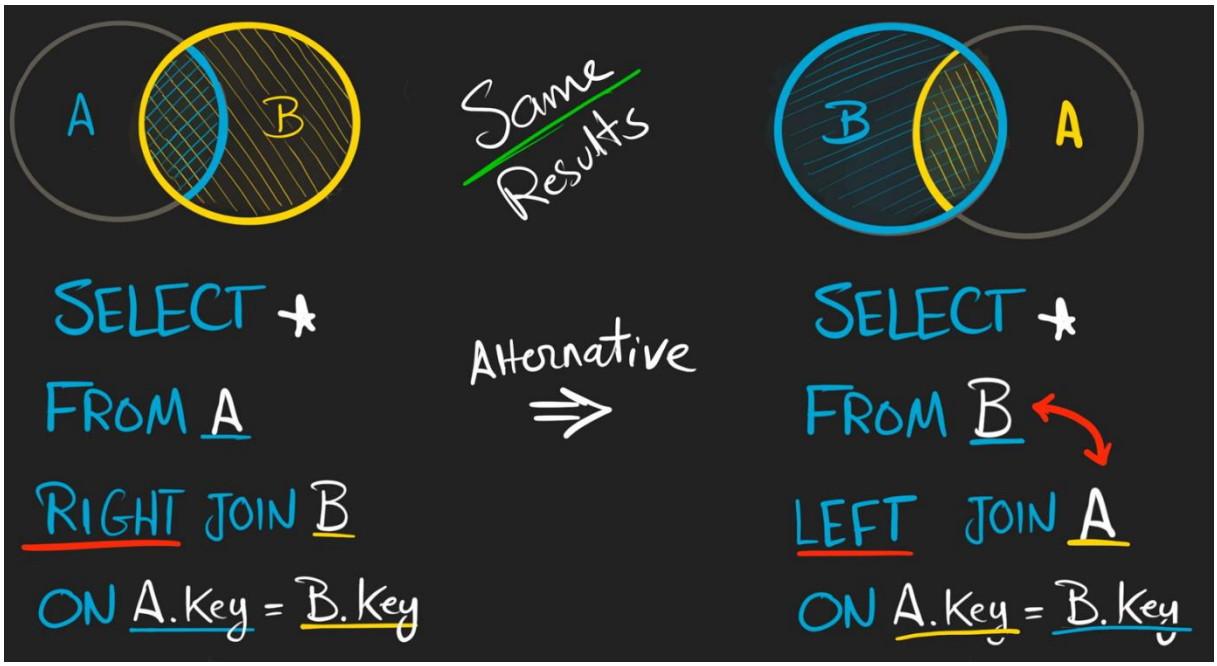
orders

order_id	order_date	sales	customer_id
1001	2021-01-11	35	1
1002	2021-04-05	15	2
1003	2021-06-18	20	3
1004	2021-08-31	10	6

RESULT

id	first_name	order_id	sales
1	Maria	1001	35
2	John	1002	15
3	Georg	1003	20
4	Martin	NULL	NULL
5	Peter	NULL	NULL

3. RIGHT JOIN



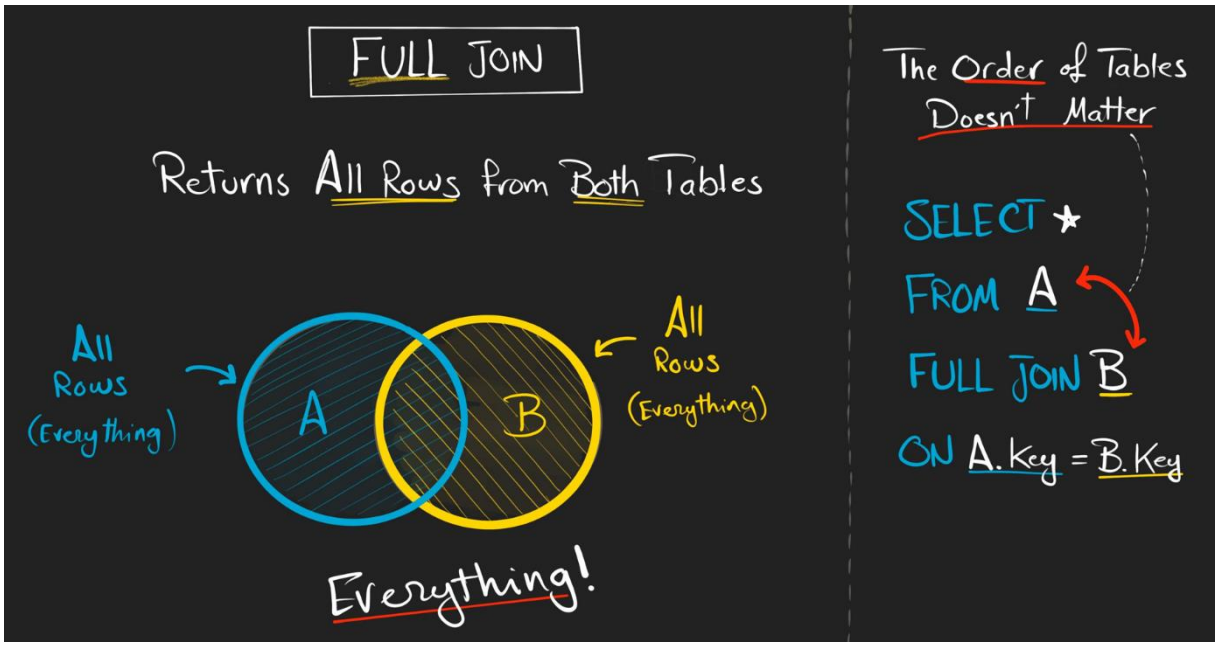

```sql
SELECT
	c.id,
	c.first_name,
	o.order_id,
	o.sales
FROM customers c
RIGHT JOIN orders o
ON c.id = o.customer_id
```

Alternative:

```sql
SELECT
	c.id,
	c.first_name,
	o.order_id,
	o.sales
FROM orders o
LEFT JOIN customers c
ON c.id = o.customer_id
```

Advice: always try to skip the RIGHT JOIN and stick with the LEFT JOIN

4. FULL JOIN




```sql
SELECT
	c.id,
	c.first_name,
	o.order_id,
	o.sales
FROM customers c
FULL JOIN orders o
ON c.id = o.customer_id
```

customers

id	first_name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	USA	750
4	Martin	Germany	500
5	Peter	USA	0

orders

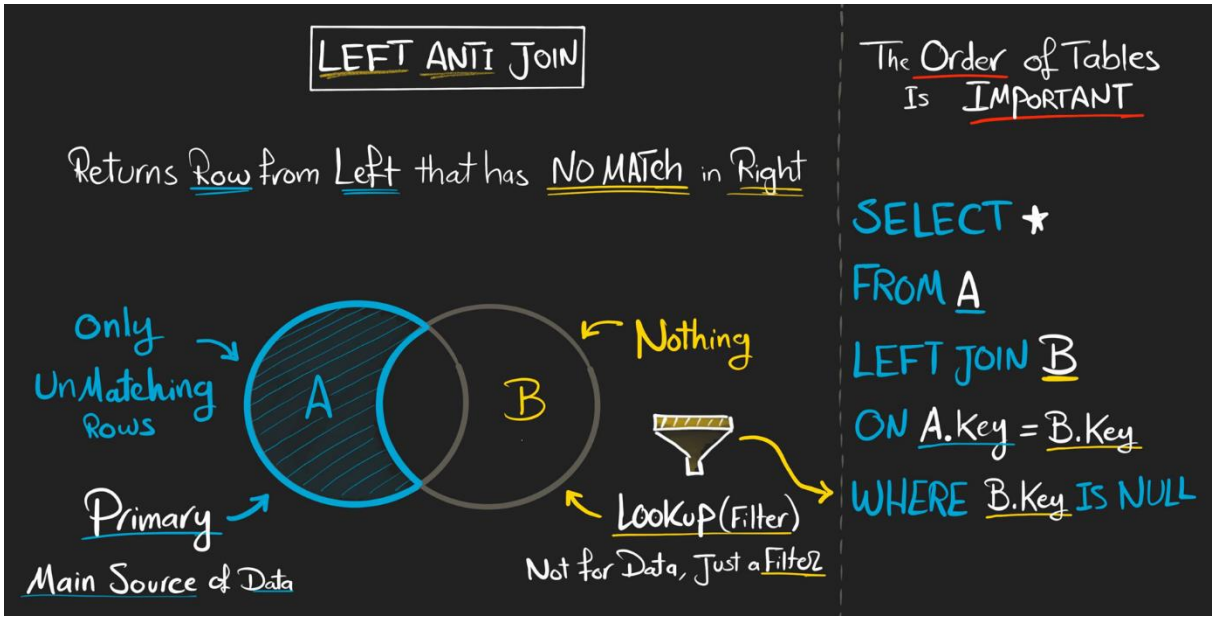
order_id	order_date	sales	customer_id
1001	2021-01-11	35	1
1002	2021-04-05	15	2
1003	2021-06-18	20	3
1004	2021-08-31	10	6

RESULT

id	first_name	order_id	sales
1	Maria	1001	35
2	John	1002	15
3	Georg	1003	20
4	Martin	NULL	NULL
5	Peter	NULL	NULL
NULL	NULL	1004	10

ii. Advanced Joins

1. LEFT Anti Join



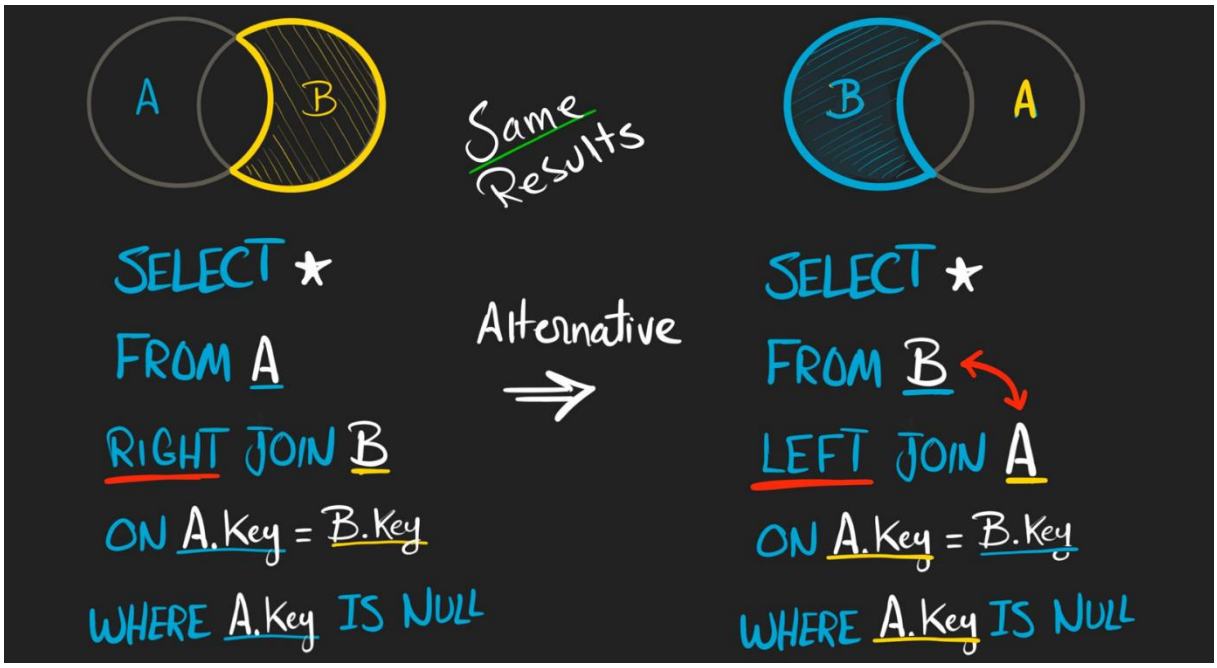

```sql
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id
WHERE o.customer_id IS NULL -- se o id em B é NULL, o dado só existe em A
```

- O resultado do INNER JOIN pode ser obtido com essa lógica:

```sql
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id
WHERE o.customer_id IS NOT NULL
```

```sql
SELECT *
FROM customers c
FULL JOIN orders o
ON c.id = o.customer_id
WHERE c.id IS NOT NULL AND o.customer_id IS NOT NULL
```

2. RIGHT Anti Join



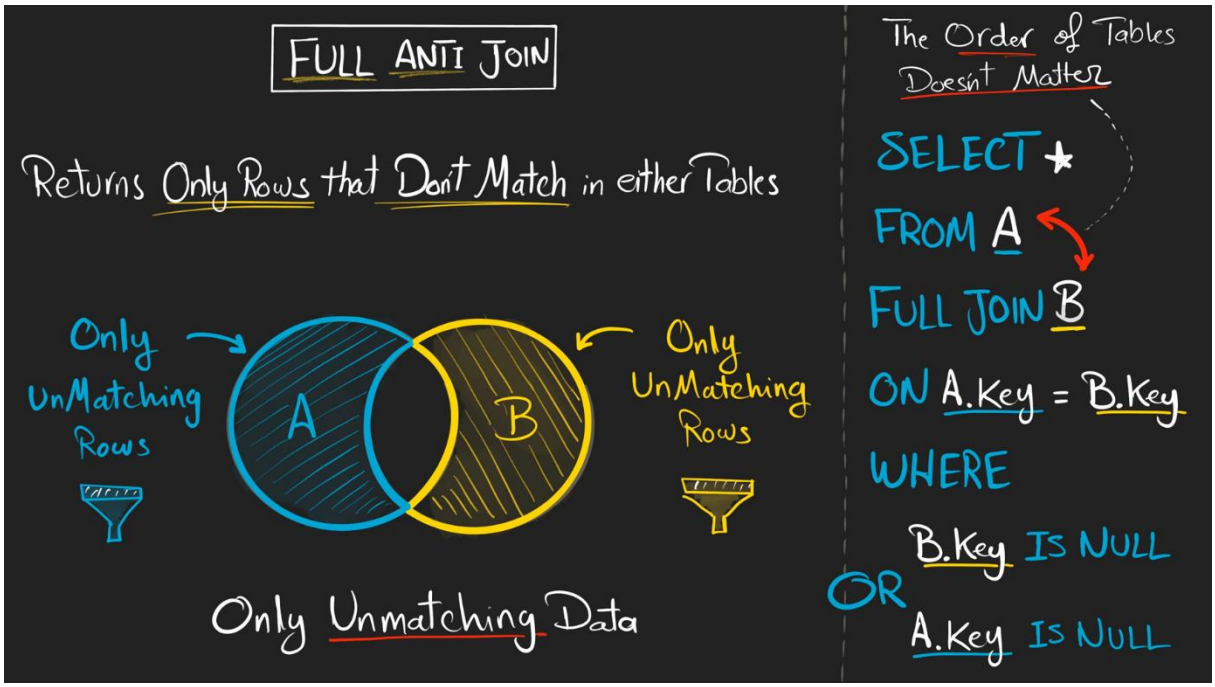

```sql
SELECT *
FROM customers c
RIGHT JOIN orders o
ON c.id = o.customer_id
WHERE c.id IS NULL
```

Alternative:

```sql
SELECT *
FROM orders o
LEFT JOIN customers c
ON c.id = o.customer_id
WHERE c.id IS NULL
```

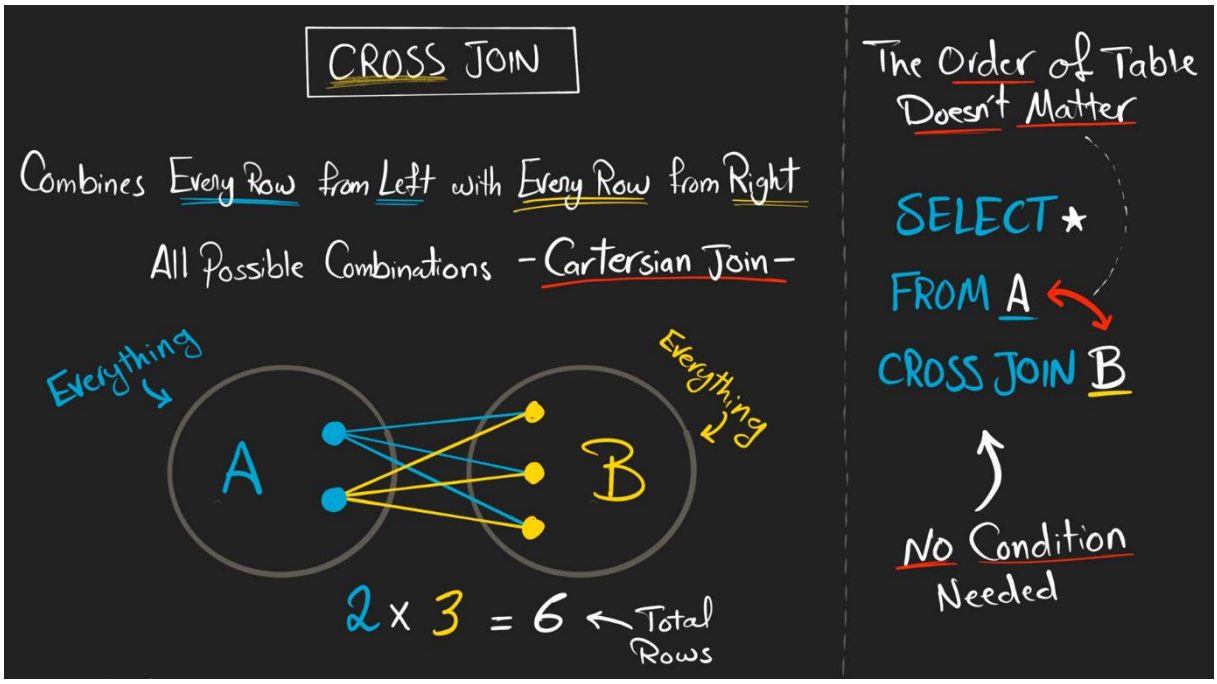
Advice: always try to skip the RIGHT JOIN and stick with the LEFT JOIN

3. FULL Anti Join




```sql
SELECT *
FROM customers c
FULL JOIN orders o
ON c.id = o.customer_id
WHERE c.id IS NULL OR o.customer_id IS NULL
```

4. CROSS JOIN

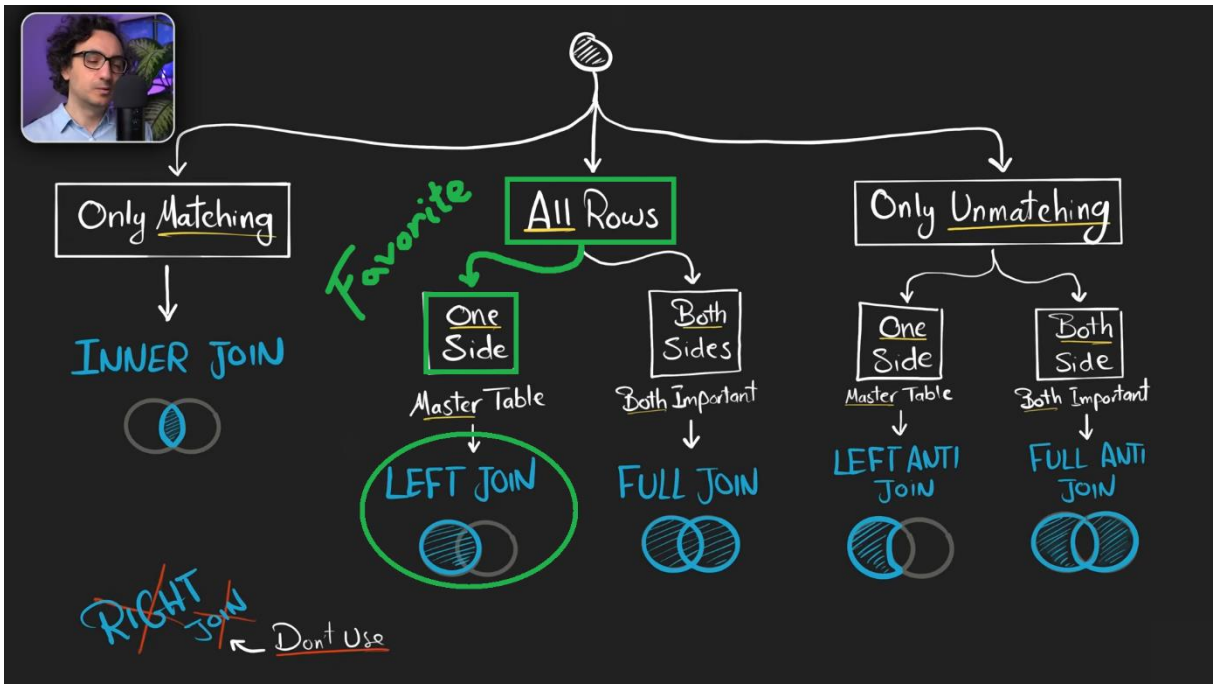



```sql
SELECT *
FROM customers
CROSS JOIN orders
```

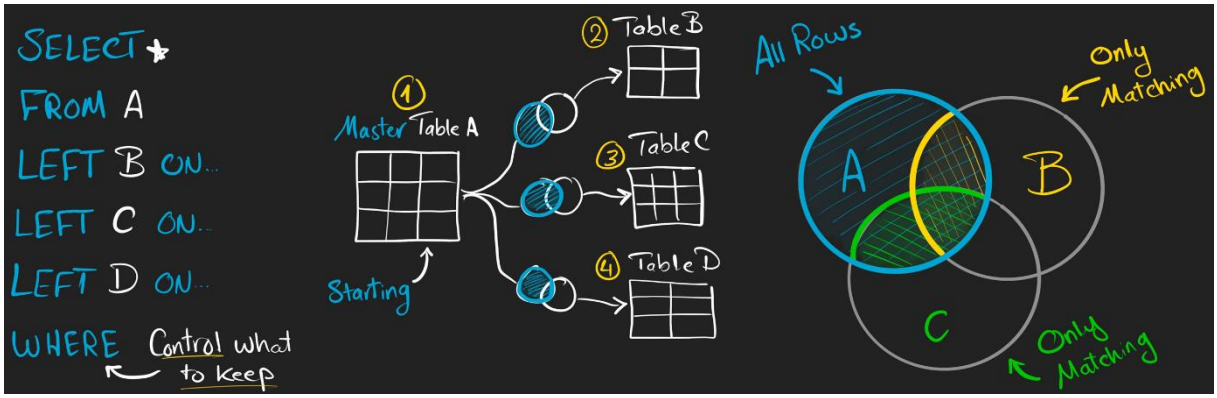
Be careful using the CROSS JOIN, because it generates crazy numbers of rows and this makes the database really busy

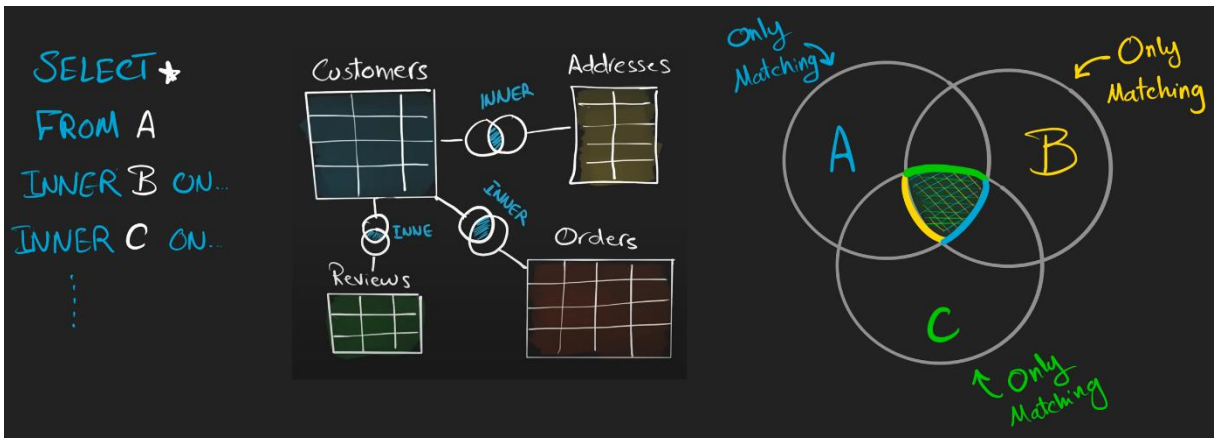
Rare to use it, but is useful in a few scenarios for simulation or testing

iii. How to choose the right Join



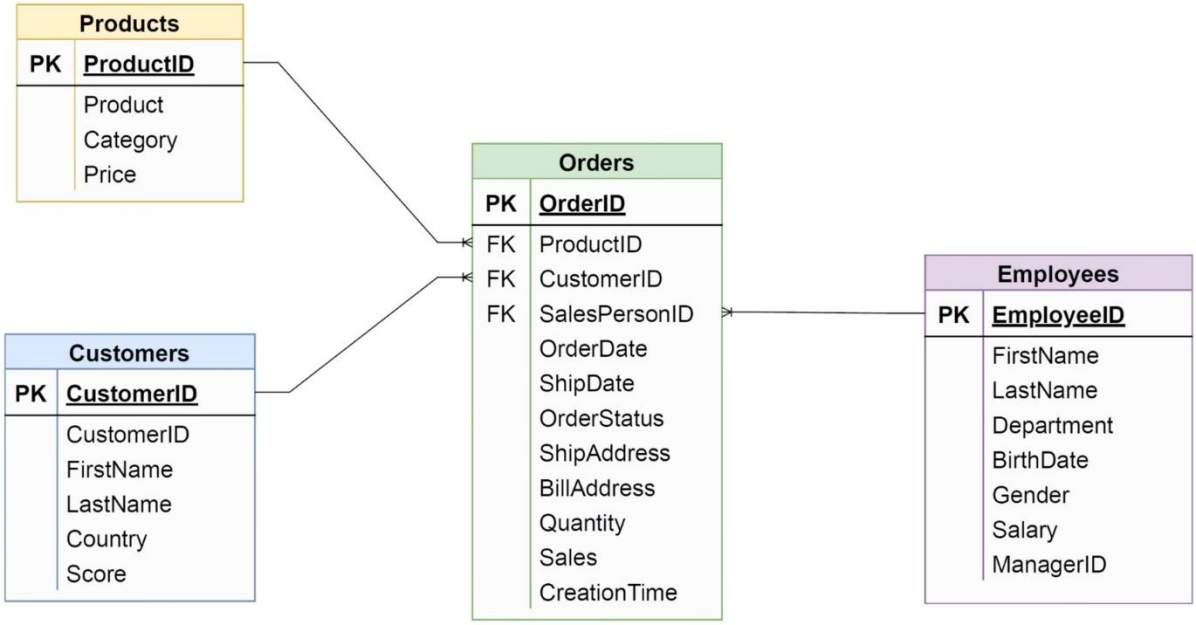
iv. How to Join multiple tables





- ✓ Task: Using SalesDB, retrieve a list of all orders, along with the related customer, product and employee details. For each order, display Order ID, Customer's name, Product name, Sales amount, Product price, Salesperson's name.

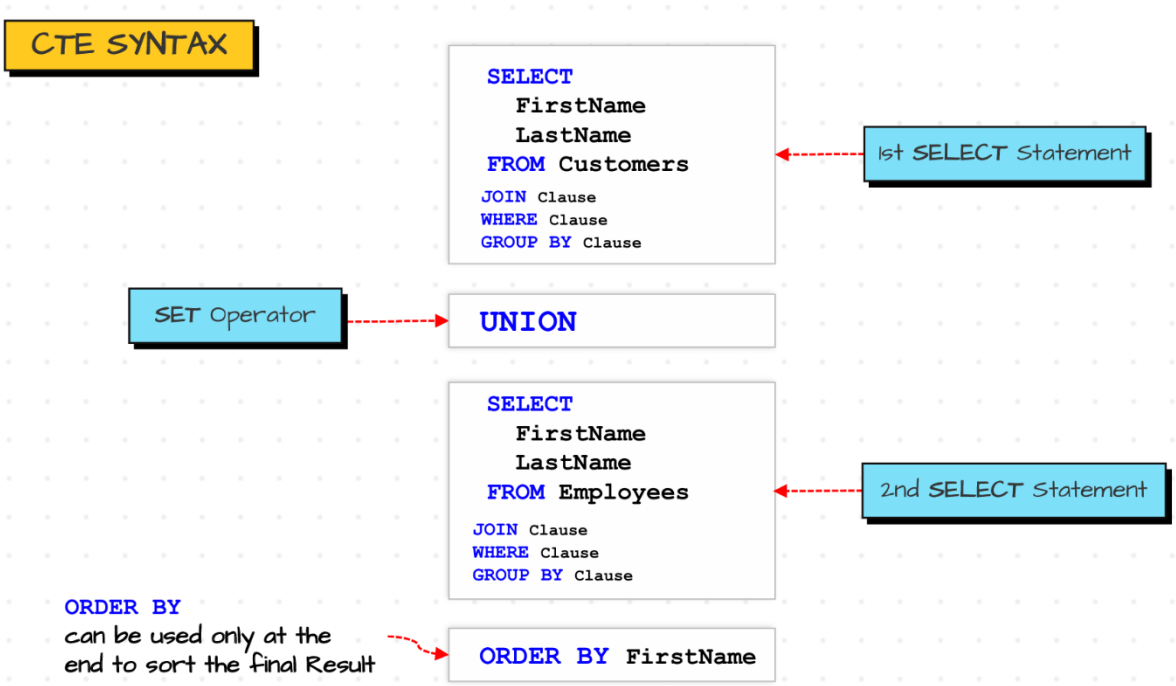
- In big projects, with a lot of tables, is hard to explore each one of them. A good project usually has an Entity-Relationship (ER) model, where you can find easily the tables that you have inside the database and the relationship between them:




```sql
SELECT
	o.OrderID,
	c.FirstName AS CustomerFirstName,
	c.LastName AS CustomerLastName,
	p.Product AS ProductName,
	o.Sales AS SalesAmount,
	p.Price,
	e.FirstName AS EmployeeFirstName,
	e.LastName AS EmployeeLastName
FROM Sales.Orders o -- the schema Sales must be inserted
LEFT JOIN Sales.Customers c
ON o.CustomerID = c.CustomerID
LEFT JOIN Sales.Products p
ON o.ProductID = p.ProductID
LEFT JOIN Sales.Employees e
ON o.SalesPersonID = e.EmployeeID
```

	OrderID	Sales	CustomerFirstName	CustomerLastName	ProductName	Price	EmployeeFirstName	EmployeeLastName
2	2	15	Mary	NULL	Tire	15	Mary	NULL
3	3	20	Jossef	Goldberg	Bottle	10	Carol	Baker
4	4	60	Jossef	Goldberg	Gloves	30	Mary	NULL
5	5	25	Kevin	Brown	Caps	25	Carol	Baker
6	6	50	Mary	NULL	Caps	25	Carol	Baker
7	7	30	Jossef	Goldberg	Tire	15	Frank	Lee
8	8	90	Mark	Schwarz	Bottle	10	Mary	NULL
9	9	20	Kevin	Brown	Bottle	10	Mary	NULL
10	10	60	Mary	NULL	Tire	15	Carol	Baker

b. SET operators

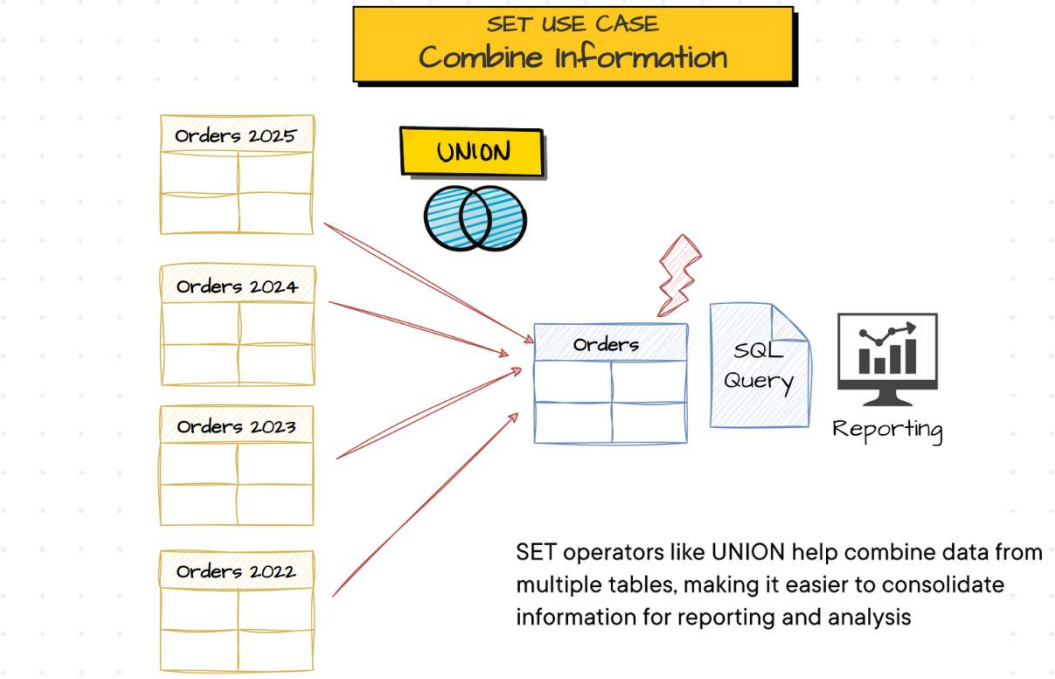


* CTE = Common Table Expression (uma forma de organizar a consulta, nomeando blocos de dados com WITH nome AS (...))

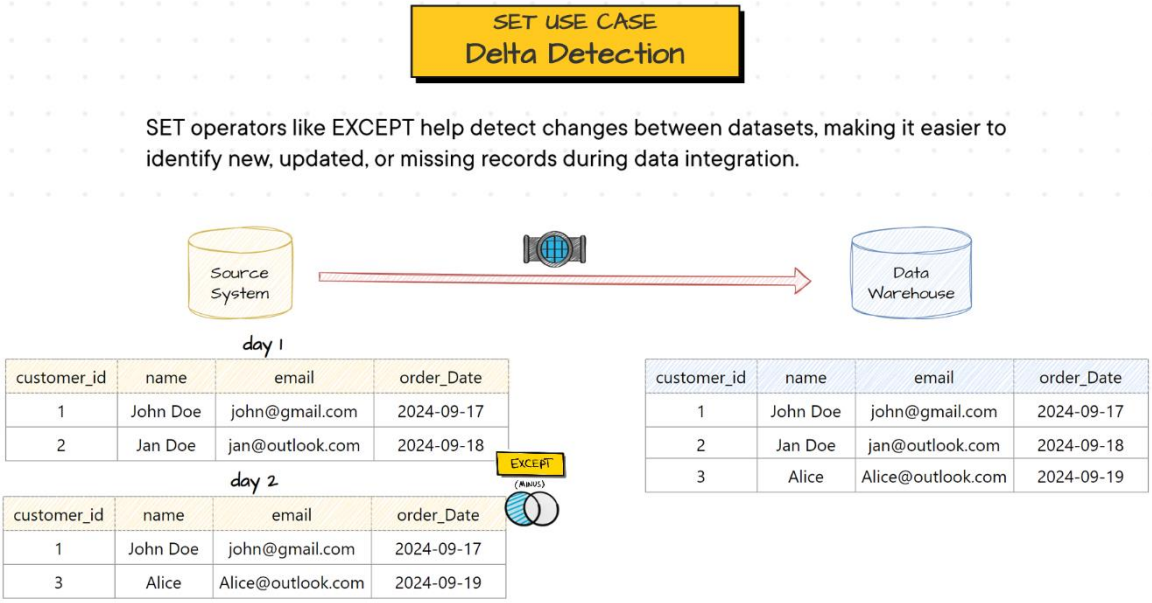
- Características:
  - SET operators can be used between SELECT statements and in almost all clauses
  - The column names in the result are determined by the column names/Aliases specified in the first query

- Rules:
  - The number of columns in each query must be the same
  - The order of columns in each query must combine
  - The column data types must be compatible across queries
  - ORDER BY is allowed only once at the end of the query

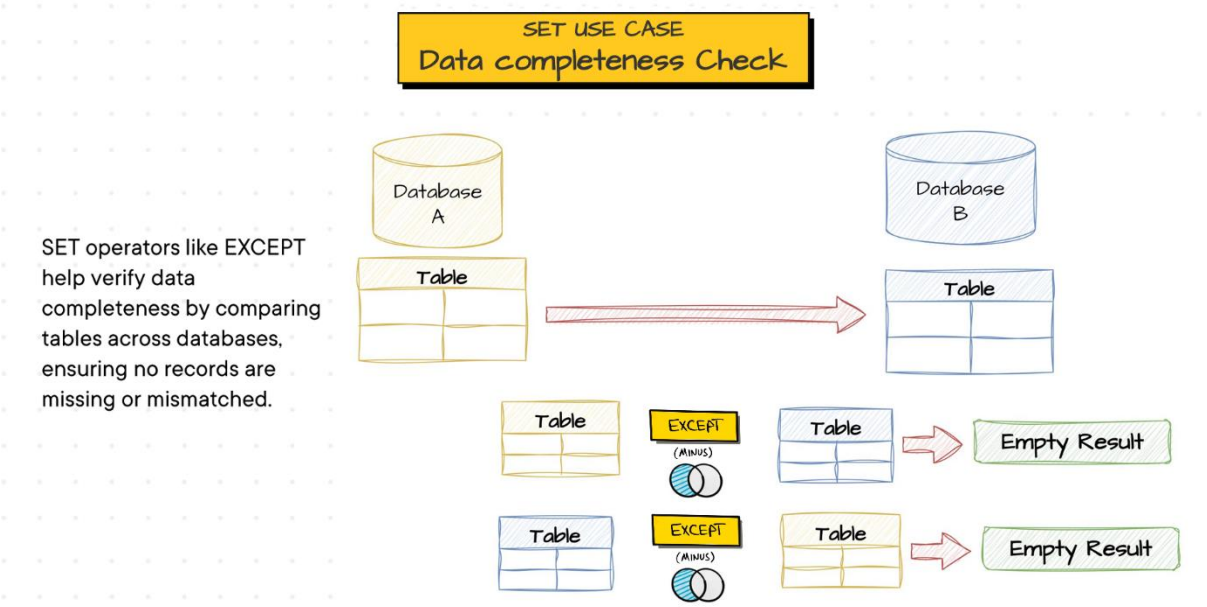
- Most important use cases:
  - To combine similar information before analyzing the data



  - Identifying the differences (delta) between two batches of data



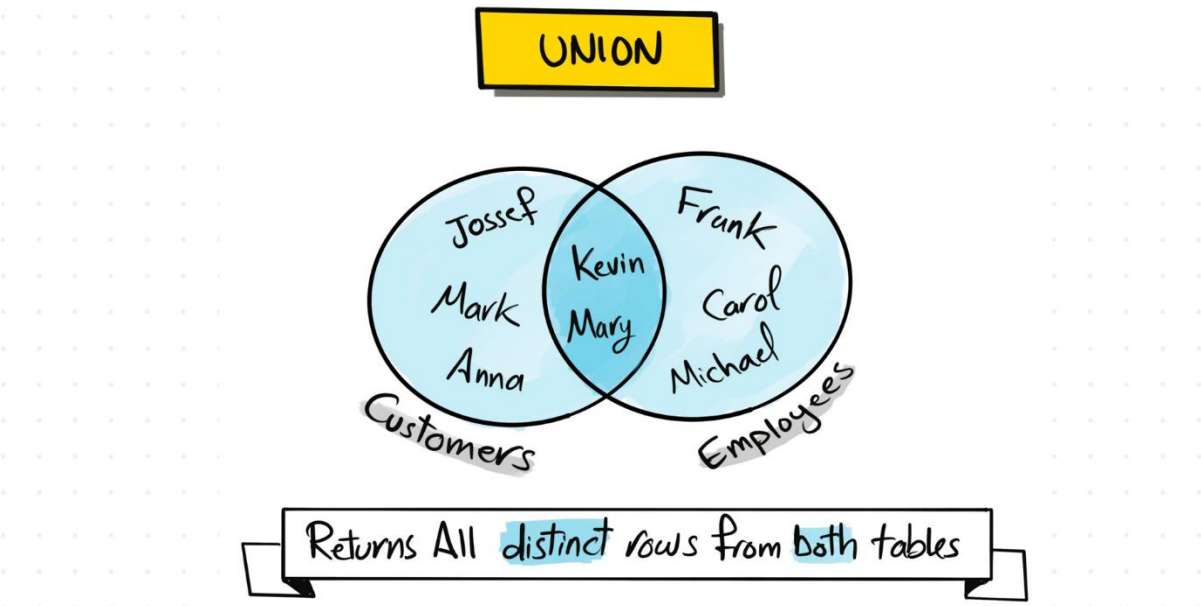
- Comparing tables to detect discrepancies between databases



- Best practices:
  - Never use an asterisk (*) to combine tables; list needed columns instead. The columns may match in one point, but with development the schema of the tables might change (renaming, adding/removing or switching)
  - Include additional static column to indicate the source of each row:

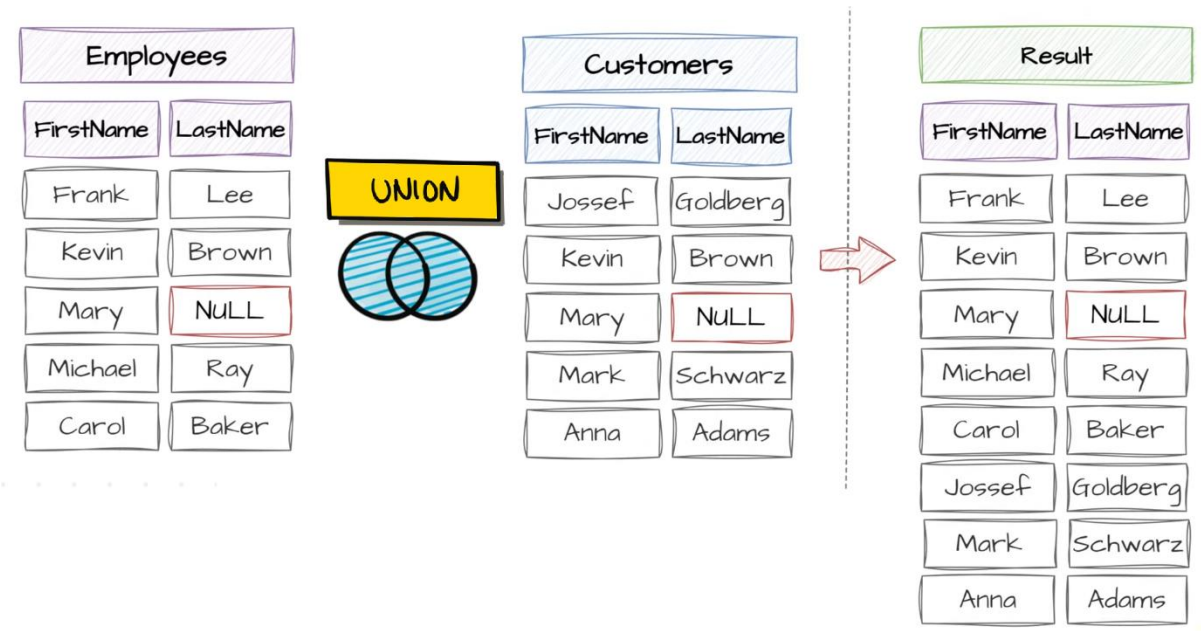
```
SELECT
	'Customers' AS SourceTable,
	c.FirstName, -- não é necessário referenciar a tabela (como em JOIN), pois em SET cada SELECT/FROM é isolado - mas ainda é uma boa prática
	c.LastName
FROM Sales.Customers c;

UNION

SELECT
	'Employees' AS SourceTable,
	e.FirstName,
	e.LastName
FROM Sales.Employees e;
```

i. UNION

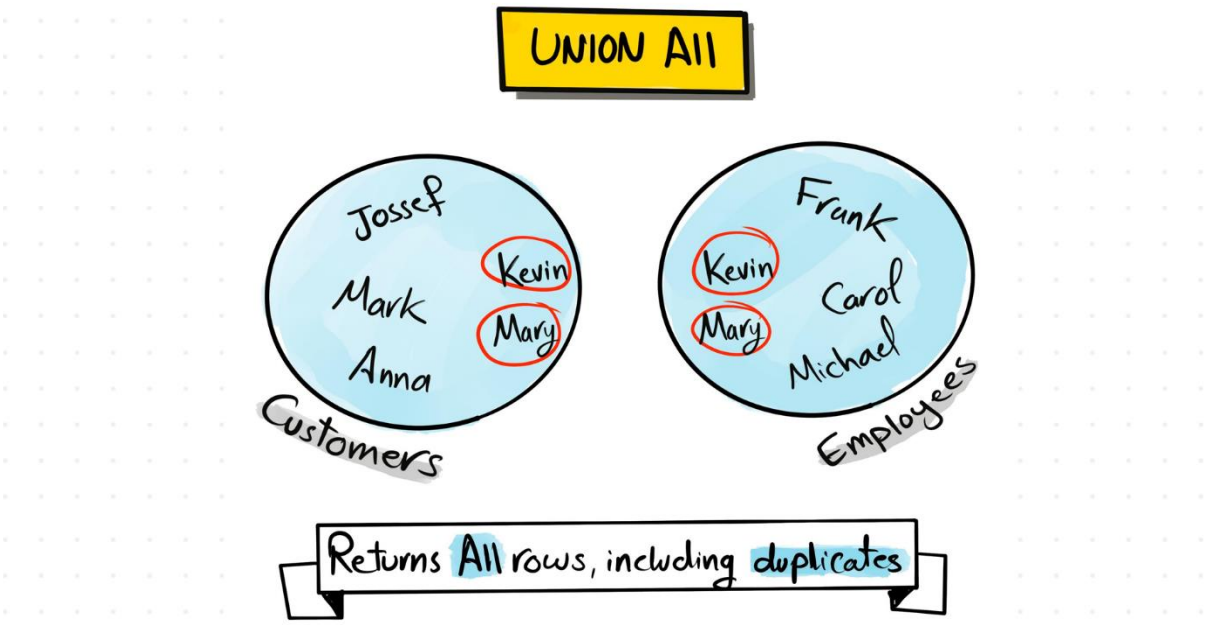


* Removes duplicate rows from the result

```
SELECT
	FirstName,
	LastName
FROM Sales.Customers;

UNION

SELECT
	FirstName,
	LastName
FROM Sales.Employees;
```



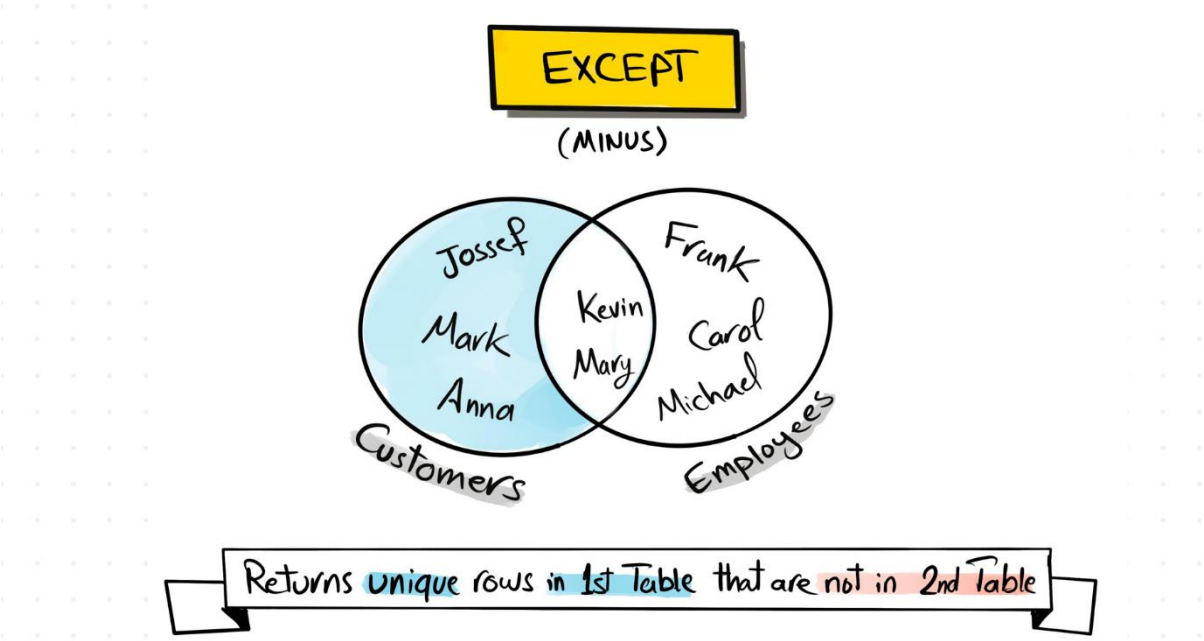
ii. UNION ALL



- Utility:
    1. Generally faster than UNION, so if you're confident that there's no duplicates, use UNION ALL
    2. Another use of UNION ALL is to find duplicates and quality issues

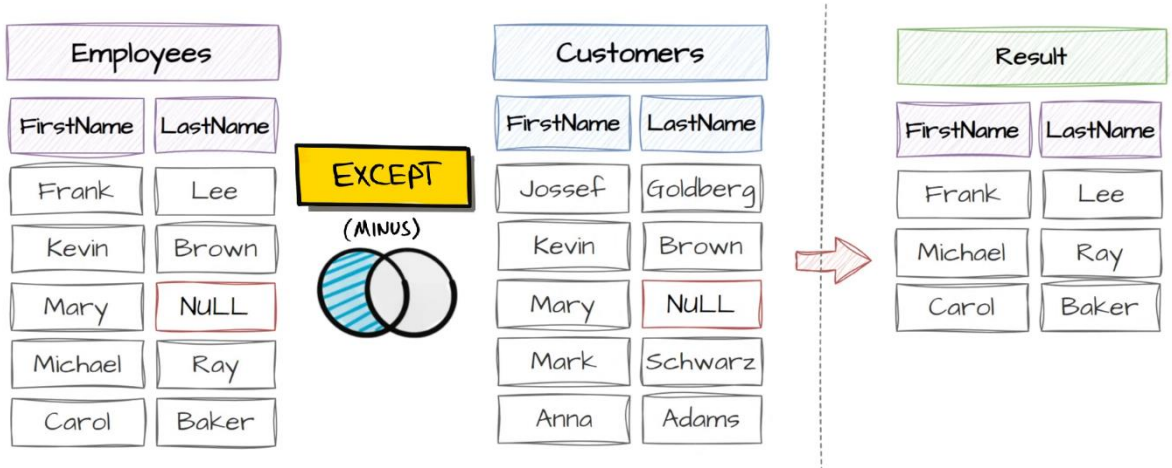
```
SELECT
      FirstName,
      LastName
FROM Sales.Customers;

UNION ALL

SELECT
      FirstName,
      LastName
FROM Sales.Employees;
```

iii. EXCEPT

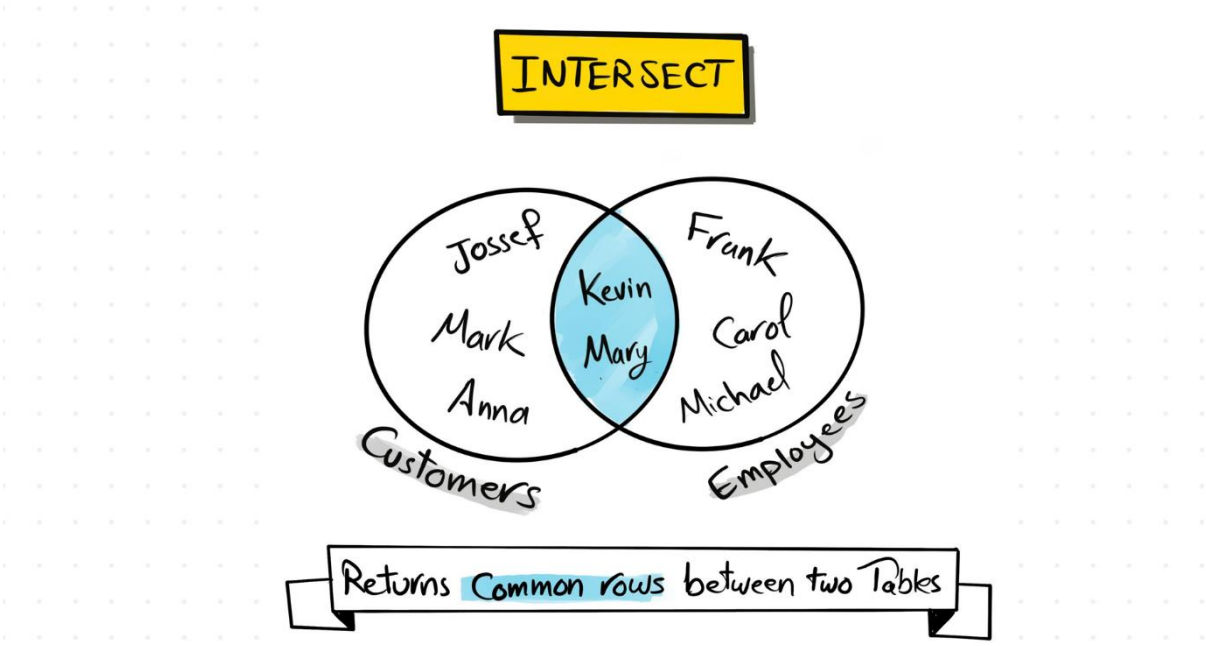


* The only SET operator where the order of queries affects the final result

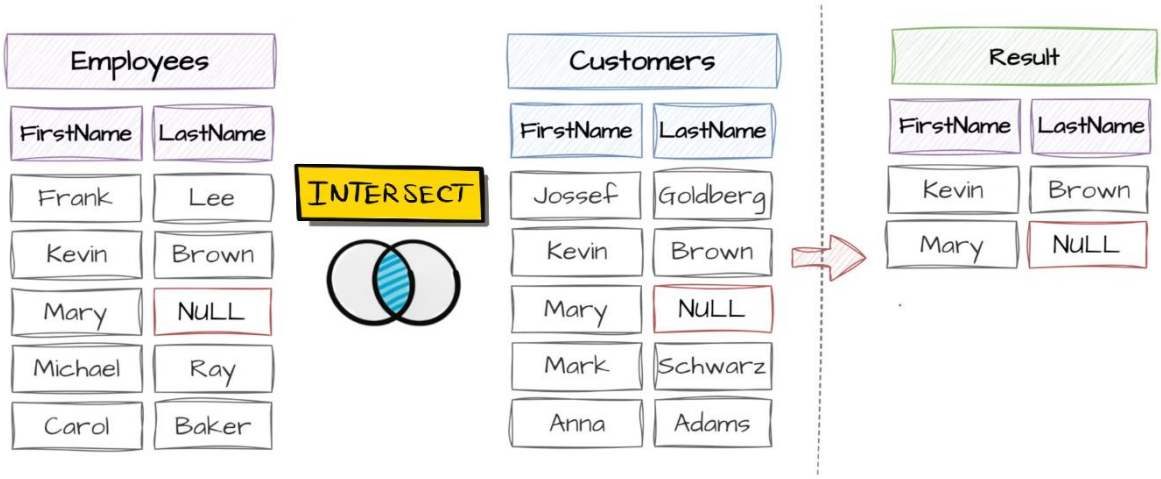
```
SELECT
	FirstName,
	LastName
FROM Sales.Employees;

EXCEPT

SELECT
	FirstName,
	LastName
FROM Sales.Customers;
```



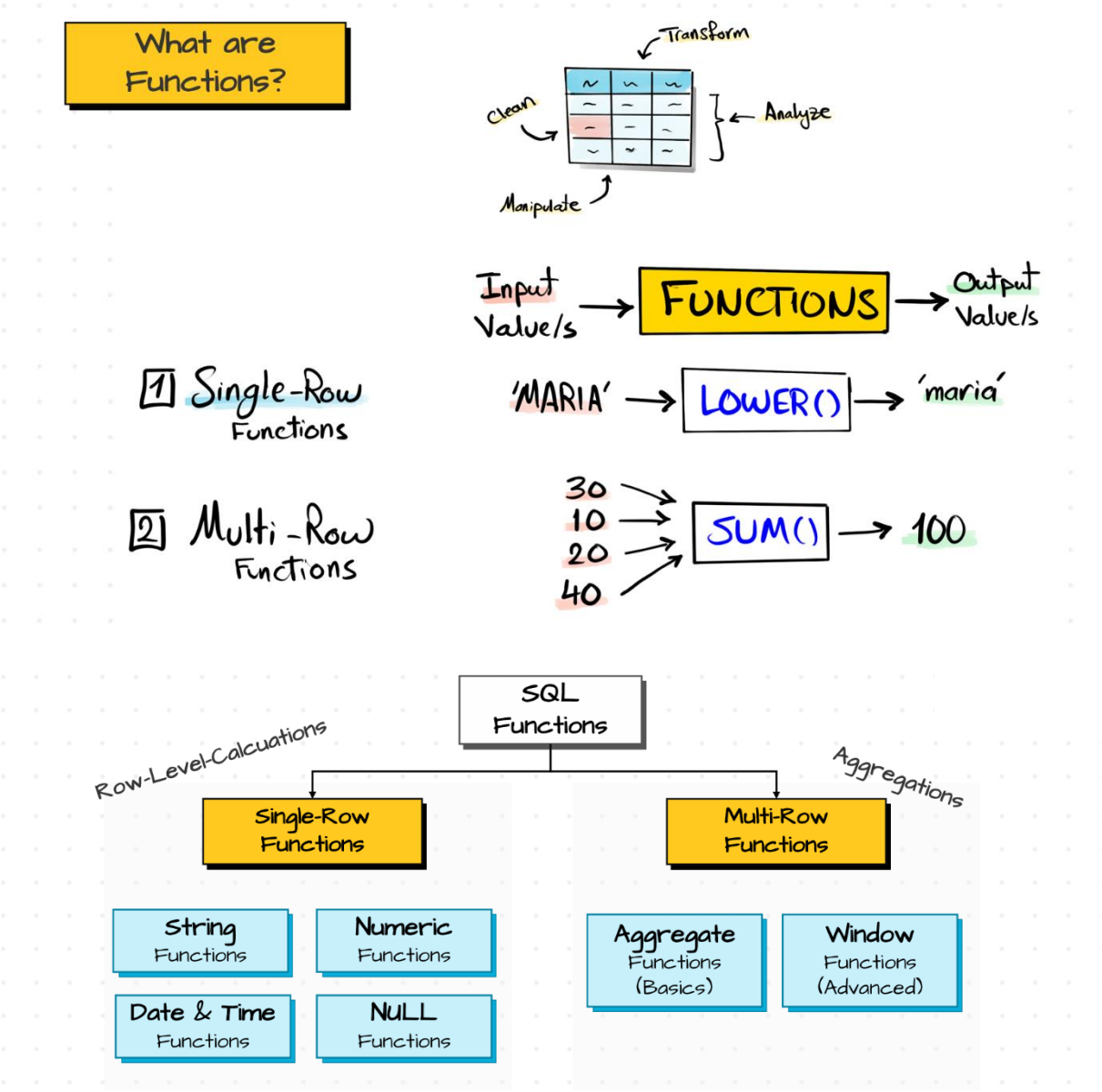
iv. INTERSECT




```
SELECT
	FirstName,
	LastName
FROM Sales.Employees;

INTERSECT

SELECT
	FirstName,
	LastName
FROM Sales.Customers;
```



7. Functions

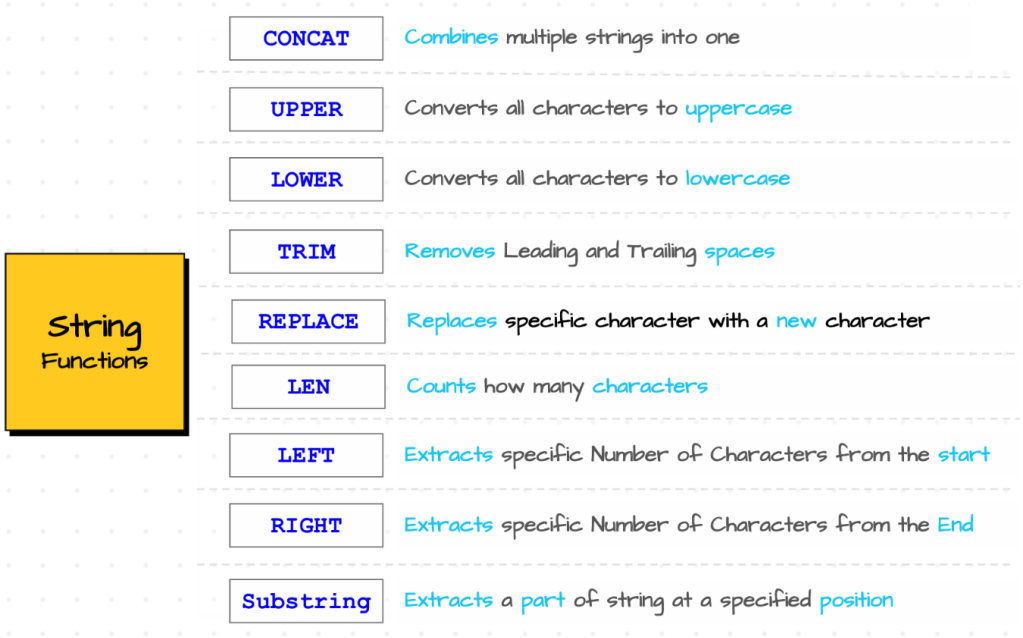


Single-Row: functions to manipulate and prepare the data (by a data engineer)

Multi-Row: functions to analyze the data (by a data analyst)

a. Row-level functions

i. String functions




```sql
SELECT
	first_name,
	country,
	CONCAT('Name: ', first_name, '. Country: ', country, '.') AS name_country,
-- (values to be concatenated)
	UPPER(first_name) AS up_name,
	LOWER(first_name) AS low_name,
	TRIM(first_name) AS trimmed,
	REPLACE(country, 'USA', 'United States') AS replaced_country, --
(expression, pattern, replacement)
	LEN(first_name) AS length,
	LEFT(country, 3) AS first3, -- (expression, number of characters)
	RIGHT(country, 1) AS last1, -- (expression, number of characters)
	SUBSTRING(first_name, 1, 3) AS substr -- (expression, start, length)
FROM customers
```

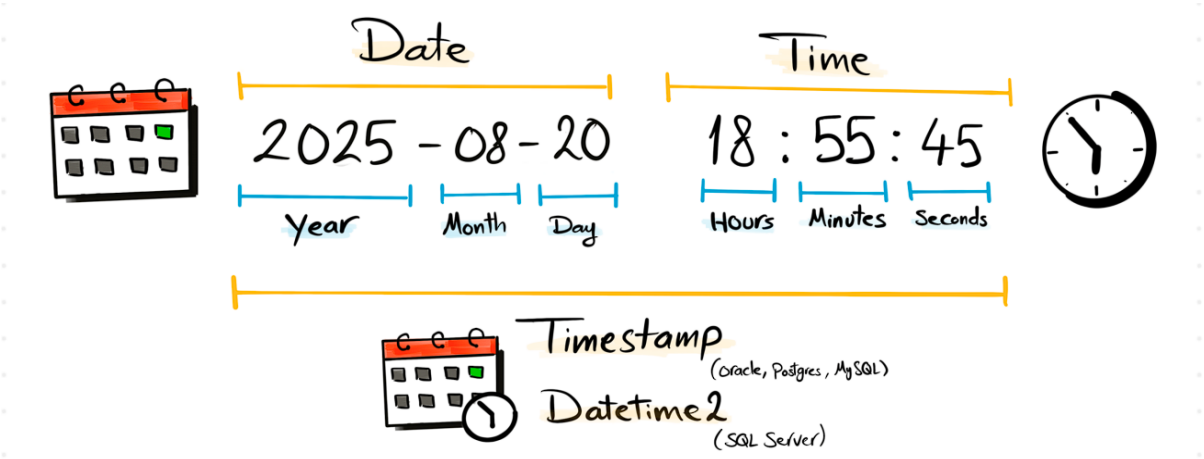
	first_name	country	name_country	up_name	low_name	trimmed	replaced_country	length	first3	last1	substr
1	Maria	Germany	Name: Maria. Country: Germany.	MARIA	maria	Maria	Germany	5	Ger	y	Mar
2	John	USA	Name: John. Country: USA.	JOHN	john	John	United States	5	USA	A	Jo
3	Georg	UK	Name: Georg. Country: UK.	GEORG	georg	Georg	UK	5	UK	K	Geo
4	Martin	Germany	Name: Martin. Country: Germany.	MARTIN	martin	Martin	Germany	6	Ger	y	Mar
5	Peter	USA	Name: Peter. Country: USA.	PETER	peter	Peter	United States	5	USA	A	Pet

ii. Number functions

```sql
SELECT
	ROUND(123.45, 1) AS R1, -- (expression, decimal length, truncate if ≠ 0)
	ROUND(123.45, 1, 1) AS R2,
	ROUND(123.45, -1) AS R3,
	ABS(-123) AS A1 -- returns the absolute (positive) value of a number
```

	R1	R2	R3	A1
1	123.50	123.40	120.00	123

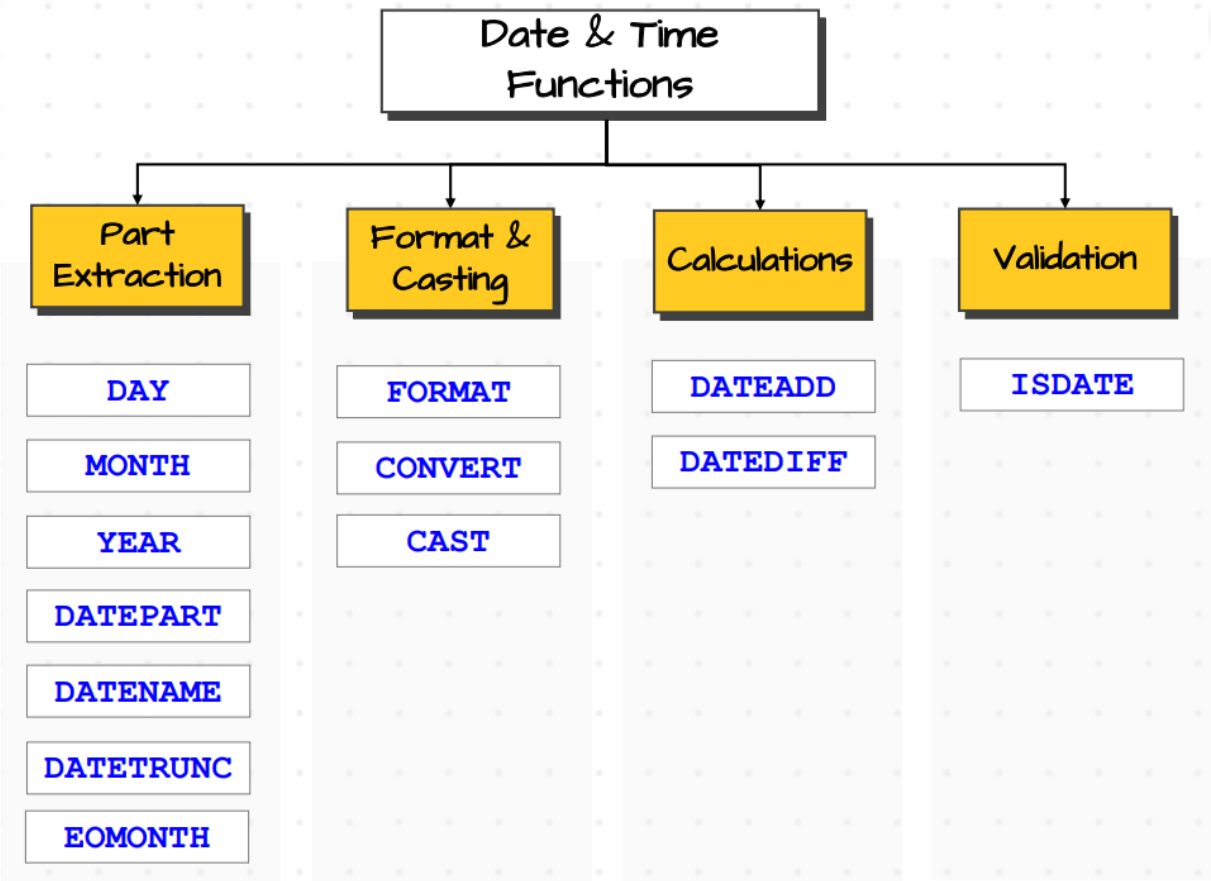
iii. Date & time functions



- 3 different sources to query the dates:

```
SELECT
	CreationTime, -- value stored inside the database
	'2025-08-18' AS Hardcoded, -- hardcoded string (static value)
	GETDATE() AS Agora -- GETDATE() function
FROM Sales.Orders
```

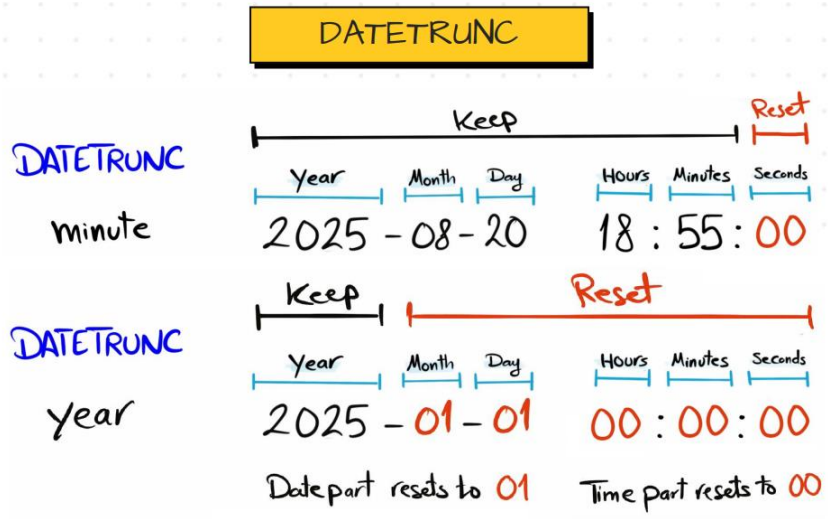
	CreationTime	Hardcoded	Agora
1	2025-01-01 12:34:56.0000000	2025-08-18	2025-08-18 23:03:17.170
2	2025-01-05 23:22:04.0000000	2025-08-18	2025-08-18 23:03:17.170
3	2025-01-10 18:24:08.0000000	2025-08-18	2025-08-18 23:03:17.170
4	2025-01-20 05:50:33.0000000	2025-08-18	2025-08-18 23:03:17.170
5	2025-02-01 14:02:41.0000000	2025-08-18	2025-08-18 23:03:17.170
6	2025-02-06 15:34:57.0000000	2025-08-18	2025-08-18 23:03:17.170
7	2025-02-16 06:22:01.0000000	2025-08-18	2025-08-18 23:03:17.170
8	2025-02-18 10:45:22.0000000	2025-08-18	2025-08-18 23:03:17.170
9	2025-03-10 12:59:04.0000000	2025-08-18	2025-08-18 23:03:17.170
10	2025-03-16 23:25:15.0000000	2025-08-18	2025-08-18 23:03:17.170



1. Part extraction

```sql
SELECT
	CreationTime,
	DAY(CreationTime) AS Dia,
	MONTH('2025-08-18') AS Hardcoded,
	YEAR(GETDATE()) AS Agora,
	DATEPART(quarter, CreationTime) AS Trimestre, -- (part, date), returns int
	DATEPART(week, CreationTime) AS Semana,
	DATEPART(weekday, CreationTime) AS DiaSemana,
	DATEPART(hour, CreationTime) AS Hora,
	DATEPART(weekday, CreationTime) AS DiaSemana,
	DATENAME(month, CreationTime) AS NomeMês, -- (part, date), returns nvarchar
	DATENAME(weekday, CreationTime) AS NomeDia,
	EOMONTH(CreationTime) AS EndOfMonth
FROM Sales.Orders
```

	CreationTime	Dia	Hardcoded	Agora	Trimestre	Semana	DiaSemana	Hora	DiaSemana	NomeMês	NomeDia	EndOfMonth
1	2025-01-01 12:34:56.0000000	1	8	2025	1	1	4	12	4	Janeiro	Quarta-Feira	2025-01-31
2	2025-01-05 23:22:04.0000000	5	8	2025	1	2	1	23	1	Janeiro	Domingo	2025-01-31
3	2025-01-10 18:24:08.0000000	10	8	2025	1	2	6	18	6	Janeiro	Sexta-Feira	2025-01-31
4	2025-01-20 05:50:33.0000000	20	8	2025	1	4	2	5	2	Janeiro	Segunda-Feira	2025-01-31
5	2025-02-01 14:02:41.0000000	1	8	2025	1	5	7	14	7	Fevereiro	Sábado	2025-02-28
6	2025-02-06 15:34:57.0000000	6	8	2025	1	6	5	15	5	Fevereiro	Quinta-Feira	2025-02-28
7	2025-02-16 06:22:01.0000000	16	8	2025	1	8	1	6	1	Fevereiro	Domingo	2025-02-28
8	2025-02-18 10:45:22.0000000	18	8	2025	1	8	3	10	3	Fevereiro	Terça-Feira	2025-02-28
9	2025-03-10 12:59:04.0000000	10	8	2025	1	11	2	12	2	Março	Segunda-Feira	2025-03-31
10	2025-03-16 23:25:15.0000000	16	8	2025	1	12	1	23	1	Março	Domingo	2025-03-31




```sql
SELECT
	CreationTime,
	DATETRUNC(month, CreationTime) AS TruncadoAtéMês -- (part, date)
FROM Sales.Orders
```

	CreationTime	TruncadoAtéMês
1	2025-01-01 12:34:56.0000000	2025-01-01 00:00:00.0000000
2	2025-01-05 23:22:04.0000000	2025-01-01 00:00:00.0000000
3	2025-01-10 18:24:08.0000000	2025-01-01 00:00:00.0000000
4	2025-01-20 05:50:33.0000000	2025-01-01 00:00:00.0000000
5	2025-02-01 14:02:41.0000000	2025-02-01 00:00:00.0000000
6	2025-02-06 15:34:57.0000000	2025-02-01 00:00:00.0000000
7	2025-02-16 06:22:01.0000000	2025-02-01 00:00:00.0000000
8	2025-02-18 10:45:22.0000000	2025-02-01 00:00:00.0000000
9	2025-03-10 12:59:04.0000000	2025-03-01 00:00:00.0000000
10	2025-03-16 23:25:15.0000000	2025-03-01 00:00:00.0000000

- Aggregation with DATETRUNC():

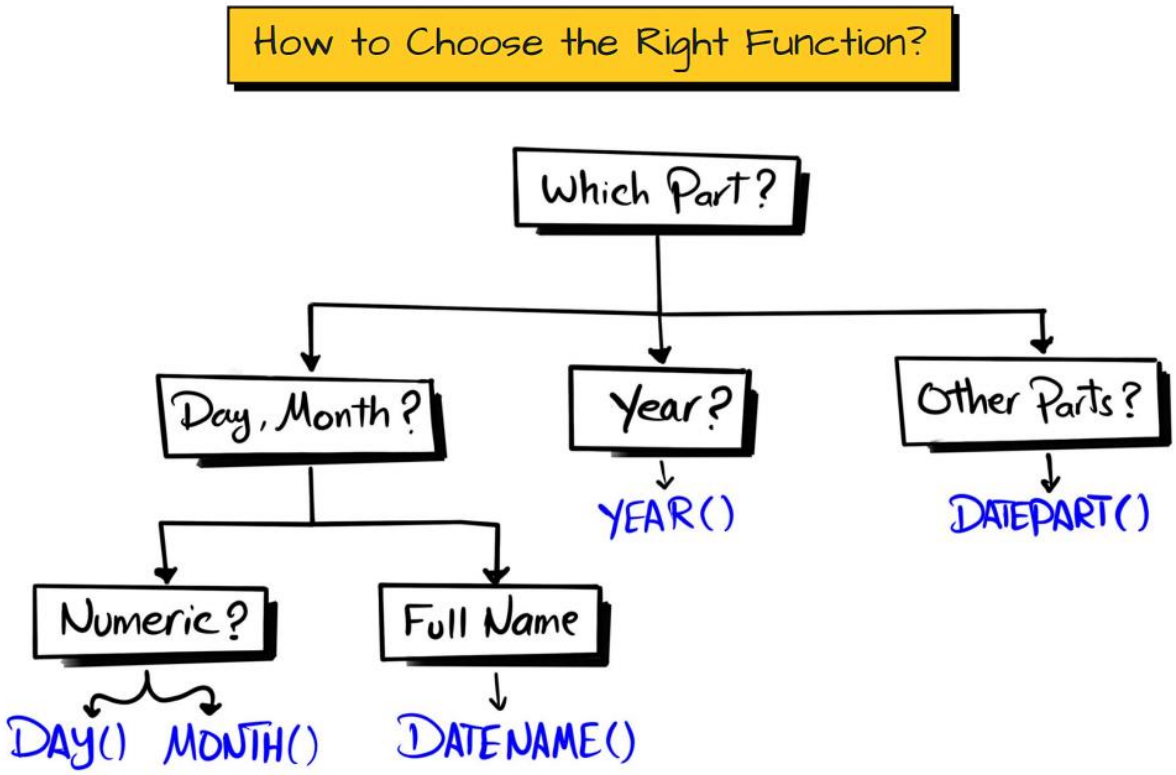
```sql
SELECT
	DATENAME(month, DATETRUNC(month, CreationTime)) AS Mês,
	COUNT(*) AS Pedidos
FROM Sales.Orders
GROUP BY DATETRUNC(month, CreationTime)
```

	Mês	Pedidos
1	Janeiro	4
2	Fevereiro	4
3	Março	2

- Data filtering:

```sql
SELECT * FROM Sales.Orders
WHERE MONTH(OrderDate) = 2
```

* Best practice: filtering data using an Integer is faster than using a String (for filtering, use DATEPART() instead of DATENAME())



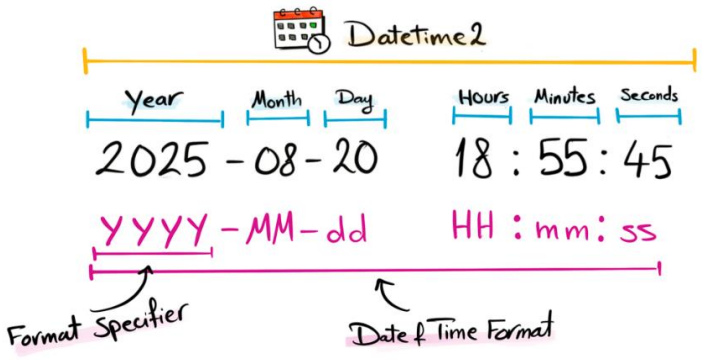
- All date parts:

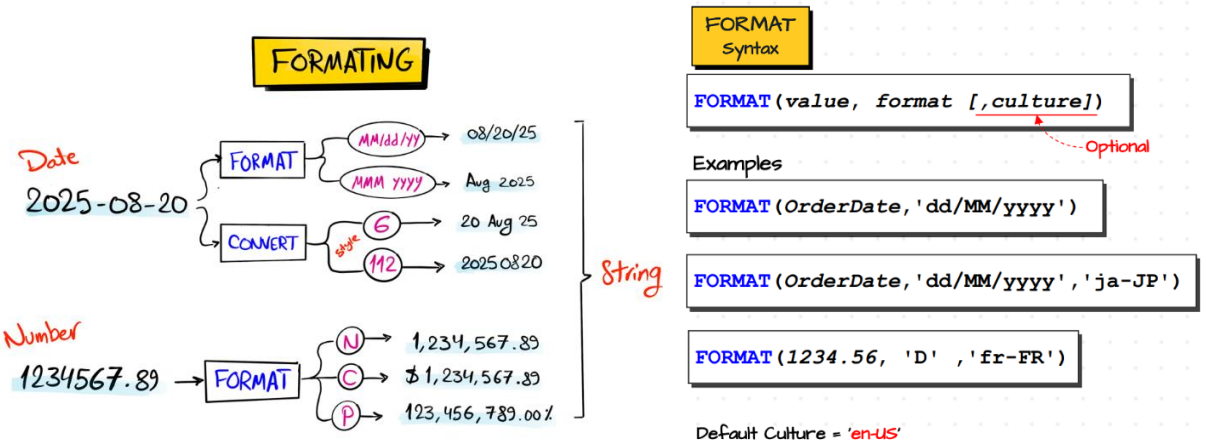
```
SELECT
    'Year (YY, YYYY)' AS DatePart,
    DATEPART(year, GETDATE()) AS DatePart_Output,
    DATENAME(year, GETDATE()) AS DateName_Output,
    DATETRUNC(year, GETDATE()) AS DateTrunc_Output
UNION ALL
SELECT
[...]
```

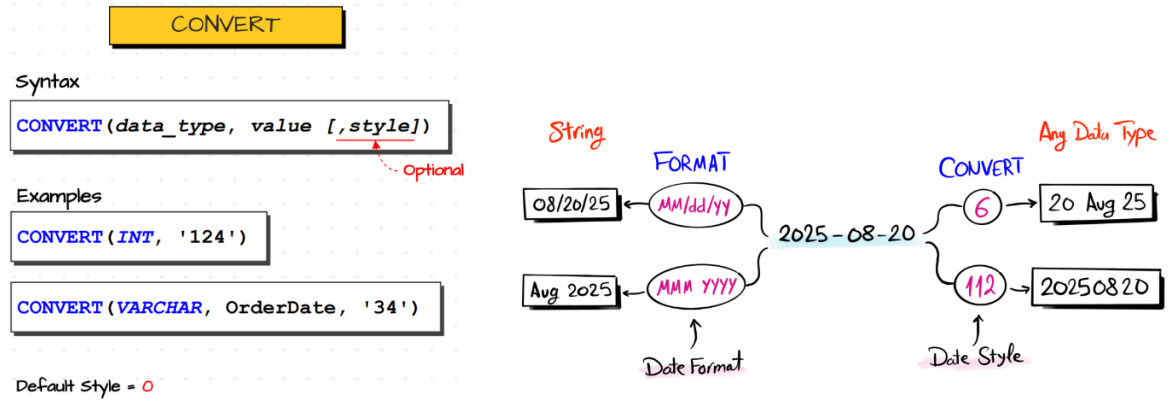
	DatePart	DatePart_Output	DateName_Output	DateTrunc_Output
1	Year (YY, YYYY)	2025	2025	2025-01-01 00:00:00.000
2	Quarter (QQ, Q)	3	3	2025-07-01 00:00:00.000
3	Month (MM, M)	8	Agosto	2025-08-01 00:00:00.000
4	DayOfYear (DY, Y)	231	231	2025-08-19 00:00:00.000
5	Day (DD, D)	19	19	2025-08-19 00:00:00.000
6	Week (WK, WW)	34	34	2025-08-17 00:00:00.000
7	Weekday (DW)	3	Terça-Feira	NULL
8	Hour (HH)	13	13	2025-08-19 13:00:00.000
9	Minute (MI, N)	37	37	2025-08-19 13:37:00.000
10	Second (SS, S)	37	37	2025-08-19 13:37:37.000
11	Millisecond (MS)	446	446	2025-08-19 13:37:37.447
12	Microsecond (MSC)	446666	446666	NULL
13	Nanosecond (NS)	446666666	446666666	NULL
14	ISOWeek (ISOWK, ISOWW)	34	34	2025-08-18 00:00:00.000

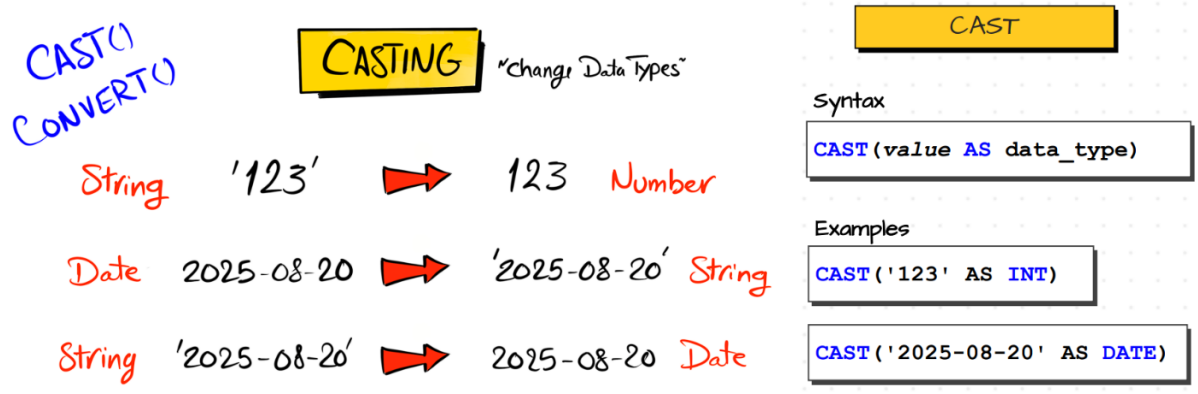
2. Format, convert & cast

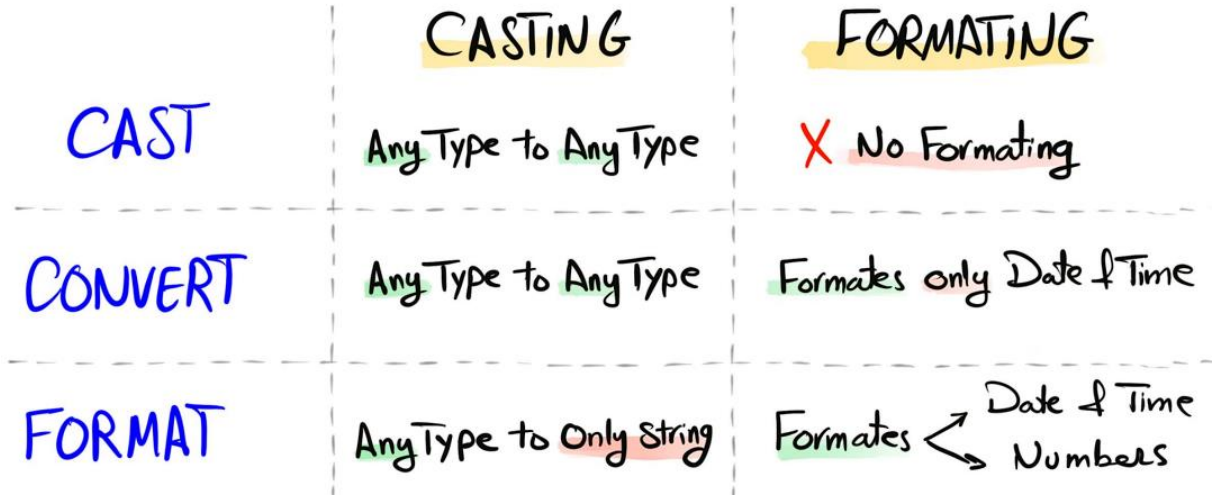
Data can be stored in different technologies (CSV files, APIs, databases). We can get different formats extracting the data from different sources into one center storage, so we need to standardize the incoming data, for analytics and reports












```sql
SELECT
	CreationTime,
	FORMAT(CreationTime, 'dd/MM/yy') AS Format1,
	FORMAT(CreationTime, 'ddd, MMM/yy') AS Format2,
	FORMAT(CreationTime, 'dddd, MMMM/yyyy') AS Format3,
	'DAY ' + FORMAT(CreationTime, 'ddd MMM') + ' Q' +
	DATENAME(quarter, CreationTime) +
	FORMAT(CreationTime, ' yyyy hh:mm:ss tt') AS FormatBizarro,
	CONVERT(DATE, CreationTime) AS Convert1,
	CAST(CreationTime AS DATE) AS Cast1
FROM Sales.Orders
```

	CreationTime	Format1	Format2	Format3	FormatBizarro	Convert1	Cast1
1	2025-01-01 12:34:56.0000000	01/01/25	qua, jan/25	quarta-feira, janeiro/2025	DAY qua jan Q1 2025 12:34:56	2025-01-01	2025-01-01
2	2025-01-05 23:22:04.0000000	05/01/25	dom, jan/25	domingo, janeiro/2025	DAY dom jan Q1 2025 11:22:04	2025-01-05	2025-01-05
3	2025-01-10 18:24:08.0000000	10/01/25	sex, jan/25	sexta-feira, janeiro/2025	DAY sex jan Q1 2025 06:24:08	2025-01-10	2025-01-10
4	2025-01-20 05:50:33.0000000	20/01/25	seg, jan/25	segunda-feira, janeiro/2025	DAY seg jan Q1 2025 05:50:33	2025-01-20	2025-01-20
5	2025-02-01 14:02:41.0000000	01/02/25	sáb, fev/25	sábado, fevereiro/2025	DAY sáb fev Q1 2025 02:02:41	2025-02-01	2025-02-01
6	2025-02-06 15:34:57.0000000	06/02/25	qui, fev/25	quinta-feira, fevereiro/2025	DAY qui fev Q1 2025 03:34:57	2025-02-06	2025-02-06
7	2025-02-16 06:22:01.0000000	16/02/25	dom, fev/25	domingo, fevereiro/2025	DAY dom fev Q1 2025 06:22:01	2025-02-16	2025-02-16
8	2025-02-18 10:45:22.0000000	18/02/25	ter, fev/25	terça-feira, fevereiro/2025	DAY ter fev Q1 2025 10:45:22	2025-02-18	2025-02-18
9	2025-03-10 12:59:04.0000000	10/03/25	seg, mar/25	segunda-feira, março/2025	DAY seg mar Q1 2025 12:59:04	2025-03-10	2025-03-10
10	2025-03-16 23:25:15.0000000	16/03/25	dom, mar/25	domingo, março/2025	DAY dom mar Q1 2025 11:25:15	2025-03-16	2025-03-16

- Aggregation with FORMAT():

```sql
SELECT
	FORMAT(CreationTime, 'MMM, yy') AS Format1,
	COUNT(*) AS Pedidos
FROM Sales.Orders
GROUP BY FORMAT(CreationTime, 'MMM, yy')
```

	Format1	Pedidos
1	fev, 25	4
2	jan, 25	4
3	mar, 25	2

- All date formats:

```
SELECT
    'D' AS FormatType,
    FORMAT(GETDATE(), 'D') AS FormattedValue,
    'Full date pattern' AS Description
UNION ALL
SELECT
```

[...]

	FormatType	FormattedValue	Description
1	D	terça-feira, 19 de agosto de 2025	Full date pattern
2	d	19/08/2025	Short date pattern
3	dd	19	Day of month with leading zero
4	ddd	ter	Abbreviated name of day
5	dddd	terça-feira	Full name of day
6	M	19 de agosto	Month without leading zero
7	MM	08	Month with leading zero
8	MMM	ago	Abbreviated name of month
9	MMMM	agosto	Full name of month
10	yy	25	Two-digit year
11	yyyy	2025	Four-digit year
12	hh	03	Hour in 12-hour clock with leading zero
13	HH	15	Hour in 24-hour clock with leading zero
14	m	19 de agosto	Minutes without leading zero
15	mm	05	Minutes with leading zero
16	s	2025-08-19T15:05:56	Seconds without leading zero
17	ss	56	Seconds with leading zero
18	f	terça-feira, 19 de agosto de 20...	Tenths of a second
19	ff	88	Hundredths of a second
20	fff	880	Milliseconds
21	T	15:05:56	Full AM/PM designator
22	t	15:05	Single character AM/PM designator
23	tt		Two character AM/PM designator

- All number formats:

```
SELECT 'N' AS FormatType, FORMAT(1234.56, 'N') AS FormattedValue
UNION ALL
SELECT [...]
```

	FormatType	FormattedValue
1	N	1.234,56
2	P	123.456,00%
3	C	R$ 1.234,56
4	E	1,234560E+003
5	F	1234,56
6	N0	1.235
7	N1	1.234,6
8	N2	1.234,56
9	N_de-DE	1.234,56
10	N_en-US	1,234.56

- All convert styles:

Date

#	Format	Example
1	mm/dd/yy	12/30/25
2	yy.mm.dd	25.12.30
3	dd/mm/yy	30/12/2025
4	dd.mm.yy	30.12.25
5	dd-mm-yy	30/12/2025
6	dd-Mon-yy	30-Dec-25
7	Mon dd, yy	Dec 30, 25
10	mm-dd-yy	12-30-25
11	yy/mm/dd	25/12/1930
12	yymmdd	251230
23	yyyy-mm-dd	30/12/2025
31	yyyy-dd-mm	2025-30-12
32	mm-dd-yyyy	12-30-2025
33	mm-yyyy-dd	12-2025-30
34	dd-mm-yyyy	30/12/2025
35	dd-yyyy-mm	30-2025-12
101	mm/dd/yyyy	12/30/2025
102	yyyy.mm.dd	2025.12.30
103	dd/mm/yyyy	30/12/2025
104	dd.mm.yyyy	30.12.2025
105	dd-mm-yyyy	30/12/2025
106	dd Mon yyyy	30-Dec-25
107	Mon dd, yyyy	Dec 30, 2025
110	mm-dd-yyyy	12-30-2025
111	yyyy/mm/dd	30/12/2025
112	yyyymmdd	20251230

Time

#	Format	Example
8	hh:mm:ss	00:38:54
14	hh:mm:ss:nnn	00:38:54:840
24	hh:mm:ss	00:38:54
108	hh:mm:ss	00:38:54
114	hh:mm:ss:nnn	00:38:54:840

Date & Time Styles

CONVERT

Datetime2

#	Format	Example
0	Mon dd yyyy hh:mm AM/PM	Dec 30 2025 12:38AM
9	Mon dd yyyy hh:mm:ss:nnn AM/PM	Dec 30 2025 12:38:54:840AM
13	dd Mon yyyy hh:mm:ss:nnn AM/PM	30 Dec 2025 00:38:54:840AM
20	yyyy-mm-dd hh:mm:ss	2025-12-30 00:38:54
21	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
22	mm/dd/yy hh:mm:ss AM/PM	12/30/25 12:38:54 AM
25	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
26	yyyy-dd-mm hh:mm:ss:nnn	2025-30-12 00:38:54.840
27	mm-dd-yyyy hh:mm:ss:nnn	12-30-2025 00:38:54.840
28	mm-yyyy-dd hh:mm:ss:nnn	12-2025-30 00:38:54.840
29	dd-mm-yyyy hh:mm:ss:nnn	30-12-2025 00:38:54.840
30	dd-yyyy-mm hh:mm:ss:nnn	30-2025-12 00:38:54.840
100	Mon dd yyyy hh:mm AM/PM	Dec 30 2025 12:38AM
109	Mon dd yyyy hh:mm:ss:nnn AM/PM	Dec 30 2025 12:38:54:840AM
113	dd Mon yyyy hh:mm:ss:nnn	30 Dec 2025 00:38:54:840
120	yyyy-mm-dd hh:mm:ss	2025-12-30 00:38:54
121	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
126	yyyy-mm-dd T hh:mm:ss:nnn	2025-12-30T00:38:54.840
127	yyyy-mm-dd T hh:mm:ss:nnn	2025-12-30T00:38:54.840

3. Calculations

DATEADD

Syntax

```
DATEADD(part, interval, date)
```

Examples

```
DATEADD(year, 2, OrderDate)
```

```
DATEADD(month, -4, OrderDate)
```

DATEDIFF

Syntax

```
DATEDIFF(part, start_date, end_date)
```

Examples

```
DATEDIFF(year, OrderDate, ShipDate)
```

```
DATEDIFF(day, OrderDate, ShipDate)
```

```sql
SELECT
	o.OrderDate,
	DATEADD(day, 1, o.OrderDate) AS DayAfter,
	DATEADD(month, -2, o.OrderDate) AS CoupleMonthsBefore,
	DATEDIFF(day, o.OrderDate, '2026-06-20') AS DaysFromMyBithday,
	DATEDIFF(year, e.BirthDate, GETDATE()) AS EmployeesAge,
	DATEDIFF(day, LAG(OrderDate) OVER(ORDER BY OrderDate), OrderDate)
	AS DaysFromEachOrder -- LAG() to access values from the previous rows
FROM Sales.Orders o
FULL JOIN Sales.Employees e
ON o.SalesPersonID = e.EmployeeID
```

	OrderDate	DayAfter	CoupleMonthsBefore	DaysFromMyBithday	EmployeesAge	DaysFromEachOrder
1	NULL	NULL	NULL	NULL	53	NULL
2	NULL	NULL	NULL	NULL	48	NULL
3	2025-01-01	2025-01-02	2024-11-01	535	39	NULL
4	2025-01-05	2025-01-06	2024-11-05	531	39	4
5	2025-01-10	2025-01-11	2024-11-10	526	43	5
6	2025-01-20	2025-01-21	2024-11-20	516	39	10
7	2025-02-01	2025-02-02	2024-12-01	504	43	12
8	2025-02-05	2025-02-06	2024-12-05	500	43	4
9	2025-02-15	2025-02-16	2024-12-15	490	37	10
10	2025-02-18	2025-02-19	2024-12-18	487	39	3
11	2025-03-10	2025-03-11	2025-01-10	467	39	20
12	2025-03-15	2025-03-16	2025-01-15	462	43	5

```sql
SELECT
	DATENAME(month, OrderDate) AS OrderMonth,
	AVG(DATEDIFF(day, OrderDate, ShipDate)) AS AverageDifference
FROM Sales.Orders
GROUP BY DATENAME(month, OrderDate), MONTH(OrderDate)
ORDER BY MONTH(OrderDate)
```

	OrderMonth	AverageDifference
1	Janeiro	7
2	Fevereiro	7
3	Março	5

4. Validation

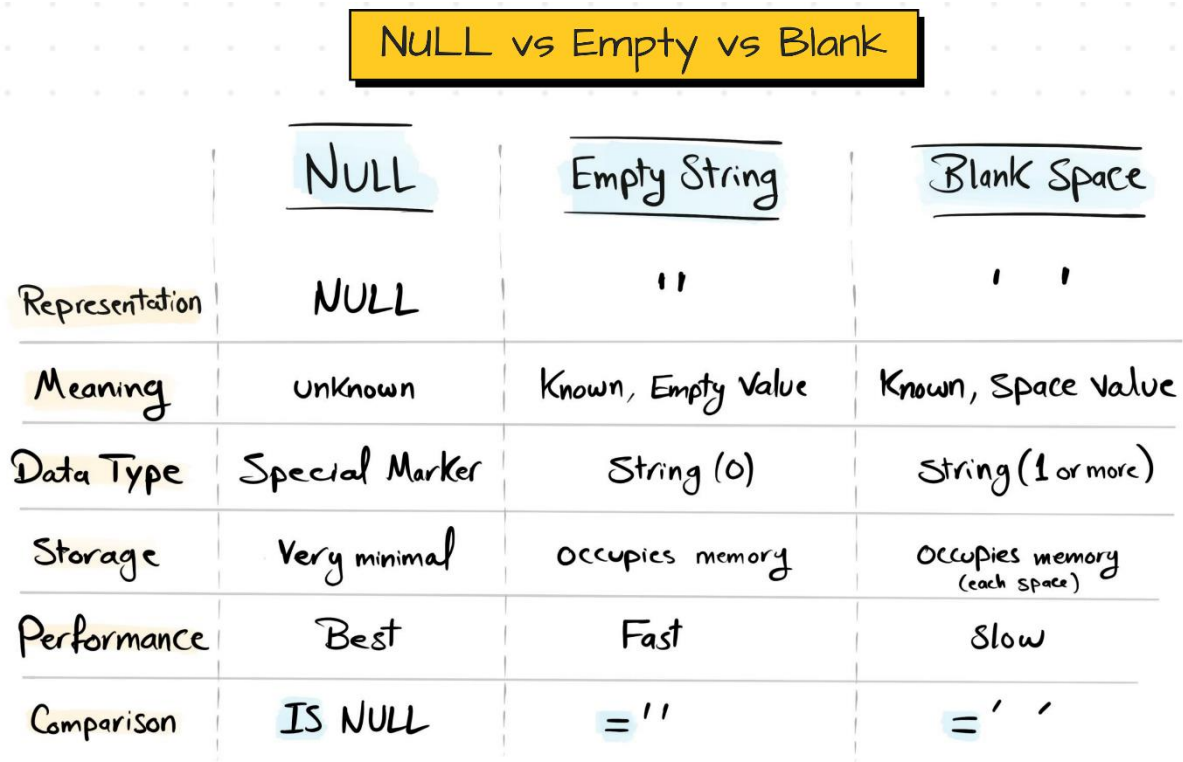
Returns 1 if the value is a valid date

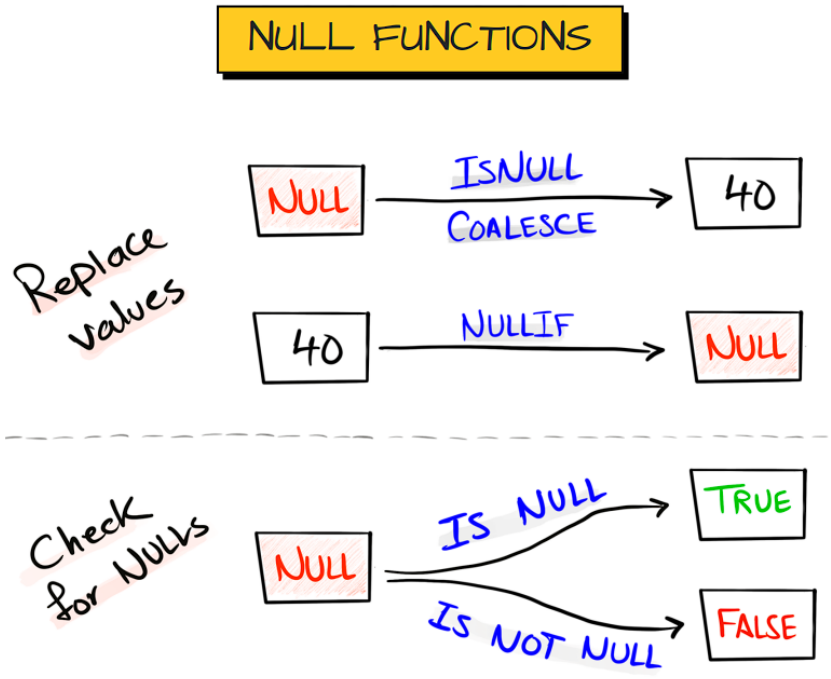
```sql
SELECT
	DateExample,
	ISDATE(DateExample) AS Verificação,
	CASE WHEN ISDATE(DateExample) = 1 THEN CAST(DateExample AS DATE)
	END NewOrderDate
FROM
	(
	SELECT '08' AS DateExample UNION
	SELECT '2000-06-20' UNION
	SELECT '20-06-2000' UNION
	SELECT '2025'
	) AS T
```

	DateExample	Verificação	NewOrderDate
1	08	0	NULL
2	2000-06-20	0	NULL
3	20-06-2000	1	2000-06-20
4	2025	1	2025-01-01

iv. Null functions

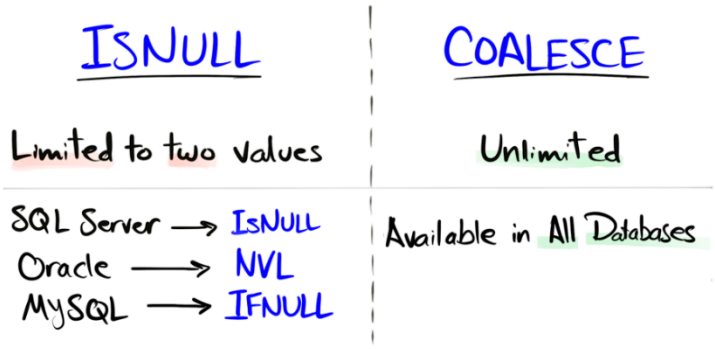
NULL = unknown information, means nothing (isn't equal to zero, an empty string or a blank space). {SQL usa Three-Valued Logic (True, False e Unknown); = NULL (ao invés de IS NULL) returns Unknown – Unknown significa que a condição não pode ser determinada porque envolve NULL}





ISNULL() -- (value, replacement), if the replacement is a column, returns NULL if its value is also NULL

COALESCE() -- (values, replacement), returns the non-null values in order from a list



* Advice: go with COALESCE() and stick with the standard

- Use cases: to handle the NULL before doing a SQL task, mathematical operations, joining tables or sorting data. Examples:
  - Aggregating with AVG(), NULL would be ignored on the division, so if you want to count it, you need to replace the NULL with 0

```
SELECT
	COALESCE(Score, 0) AS ReplacedScore,
	AVG(Score) OVER() AS Average4Scores,
	AVG(COALESCE(Score, 0)) OVER() AS Average5Scores
FROM Sales.Customers
```

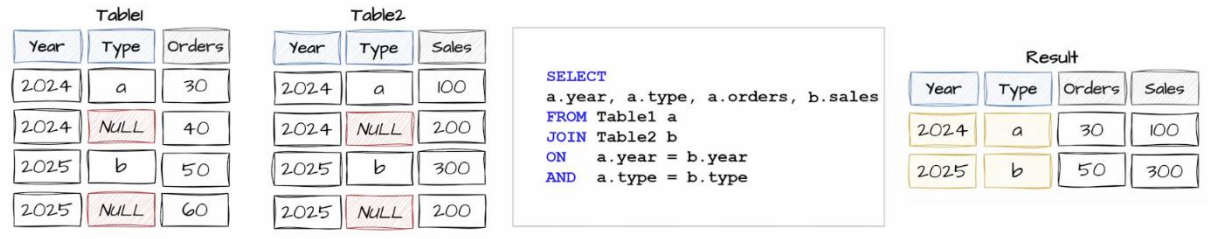
	ReplacedScore	Average4Scores	Average5Scores
1	350	625	500
2	900	625	500
3	750	625	500
4	500	625	500
5	0	625	500

  - 1+1=2 and 'A'+'B'='AB', but NULL+1=NULL and NULL+'B'=NULL

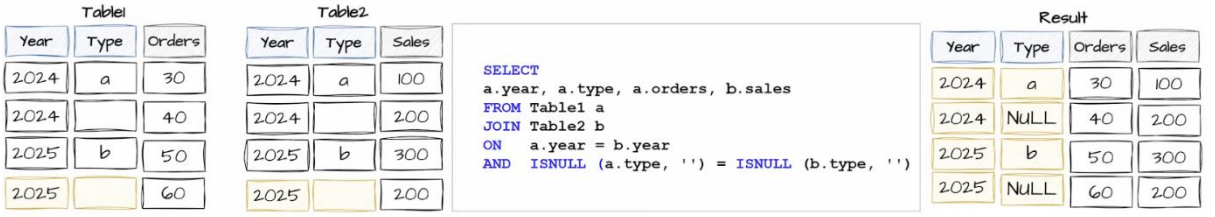
```
SELECT
	FirstName,
	LastName,
	FirstName + ' ' + LastName AS WrongFullName,
	COALESCE(FirstName, '') + ' ' + COALESCE(LastName, '') AS FullName,
	Score,
	Score + 10 AS WrongBonusScore,
	COALESCE(Score, 0) + 10 AS BonusScore
FROM Sales.Customers
```

	FirstName	LastName	WrongFullName	FullName	Score	WrongBonusScore	BonusScore
1	Jossef	Goldberg	Jossef Goldberg	Jossef Goldberg	350	360	360
2	Kevin	Brown	Kevin Brown	Kevin Brown	900	910	910
3	Mary	NULL	NULL	Mary	750	760	760
4	Mark	Schwarz	Mark Schwarz	Mark Schwarz	500	510	510
5	Anna	Adams	Anna Adams	Anna Adams	NULL	NULL	10

- If there are NULLs in the keys, these rows will not appear in the output



* SQL can't compare NULLs



* Handling the NULLs only on the JOIN, SELECT will appear the original data

- When sorting the data in ascending order, NULLs are placed before any value (and after any values in descending order), so the professional way for repositioning them, is to use a flag

```
SELECT
	Score,
	CASE WHEN Score IS NULL THEN 1 ELSE 0 END AS Flag
FROM Sales.Customers
ORDER BY Flag, Score ASC
```

	Score	Flag
1	350	0
2	500	0
3	750	0
4	900	0
5	NULL	1

NULLIF() -- (value1, value2), if the values are equal, returns NULL, else the first value

```
NULLIF(Price, -1)
```

OrderID	Price	NULLIF
1	90	90
2	-1	NULL

- Use cases:
  - To highlight/flag special cases in our data, such as when the original and discount prices are equal

```
NULLIF(Original_Price, Discount_Price)
```

OrderID	Original_Price	Discount_Price	NULLIF
1	150	50	150
2	250	250	NULL

  - To prevent zero division error

```
SELECT
	Sales,
	Quantity,
	Sales / NULLIF(Quantity, 0) AS Price
FROM Sales.Orders
```

	Sales	Quantity	Price
1	10	1	10
2	15	1	15
3	20	2	10
4	60	2	30
5	25	1	25
6	50	2	25
7	30	2	15
8	90	3	30
9	20	2	10
10	60	0	NULL

- ✓ Data policies:

Bringing standards (together with the stakeholders and the users of your reports and analysis) is a very important step before doing any analysis – define a clear data policy. 3 options:

1. Only use NULLs and empty strings, but avoid blank spaces
   ➔ Not so clear
2. Only use NULLs, instead of empty strings and blank spaces
   ➔ Best performance, useful in data preparation before inserting into a database
3. Use the default value 'unknown' and avoid NULLs, empty strings and blank spaces
   ➔ Best readability, useful in data preparation before using it in reporting

```
WITH Exemplo AS (
SELECT 1 AS Id, NULL AS Valor UNION
SELECT 2, '' UNION
SELECT 3, ' '
)
SELECT
      *,
      TRIM(Valor) AS Policy1,
      NULLIF(TRIM(Valor), '') AS Policy2,
      COALESCE(NULLIF(TRIM(Valor), ''), 'Unknown') AS Policy3
FROM Exemplo
```

	Id	Valor	Policy1	Policy2	Policy3
1	1	NULL	NULL	NULL	Unknown
2	2			NULL	Unknown
3	3			NULL	Unknown

i. Case statement

Evaluates a list of conditions and returns a value when the first condition is met

```
WITH Exemplo AS (
SELECT 1 AS Id, NULL AS Valor UNION
SELECT 2, '' UNION
SELECT 3, 'abc'
)
SELECT
	*,
	CASE -- indicates start of a conditional logic
		WHEN Valor IS NULL THEN 'Null' -- (= if) condition to be evaluated
and its result (results data type must match)
		WHEN Valor = '' THEN 'Empty string' -- (= elif)
		ELSE 'Valor' -- default value (optional) if none of the WHEN
conditions are true
	END AS Cases
FROM Exemplo
```

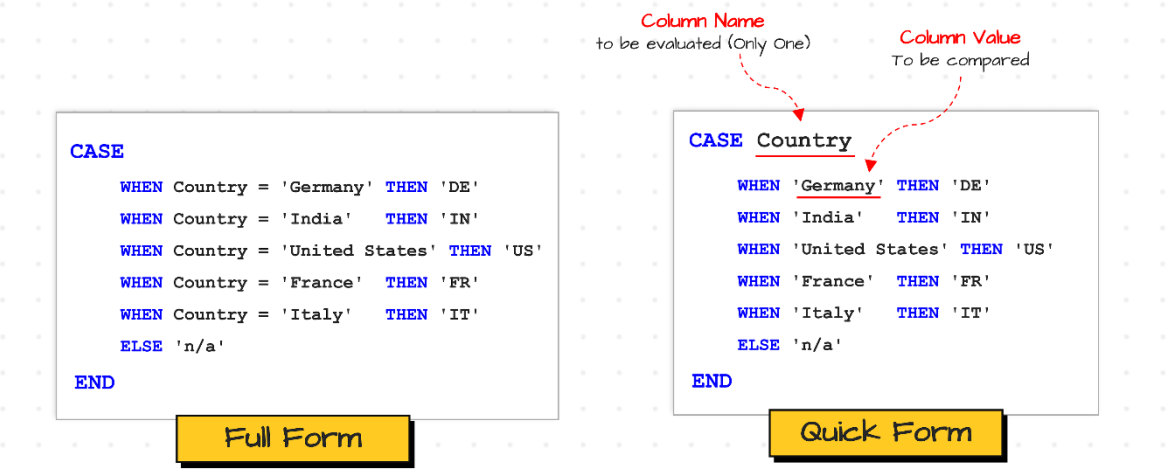
	Id	Valor	Cases
1	1	NULL	Null
2	2		Empty string
3	3	abc	Valor

- Use cases:
  - Data categorization by new columns with derived information (main purpose of case statements)

```
SELECT
	Category,
	SUM(Sales) AS TotalSales
FROM(
	SELECT
		Sales,
		CASE
			WHEN Sales > 50 THEN 'High'
			WHEN Sales > 20 THEN 'Medium'
			ELSE 'Low'
		END AS Category
	FROM Sales.Orders
) T
GROUP BY Category
ORDER BY TotalSales DESC
```

	Category	TotalSales
1	High	210
2	Medium	105
3	Low	65

- Mapping values



* The quick form only accepts conditions with equal operator (column = specified value)

Advice: if you are sure that the logic will not get complicated (as it's the case of mapping) and you can stay always with the same column, you can go with the quick form, but I would recommend to go always with the full form, because, otherwise, if you add logic, you'd have to rewrite the whole case statements back to the full form

```
SELECT
	Gender,
	CASE Gender
		WHEN 'M' THEN 'Male'
		WHEN 'F' THEN 'Female'
		ELSE 'N/A'
	END AS MoreReadableGender
FROM Sales.Employees
```

	Gender	MoreReadableGender
1	M	Male
2	M	Male
3	F	Female
4	M	Male
5	F	Female

- Handling NULLs

```
SELECT
	Score,
	AVG(Score) OVER() WrongAVG,
	CASE
		WHEN Score IS NULL THEN 0
		ELSE Score
	END AS NULLsHandled,
	AVG(
		CASE
			WHEN Score IS NULL THEN 0
			ELSE Score
		END
	) OVER() AS CorrectAVG
FROM Sales.Customers
```

	Score	WrongAVG	NULLsHandled	CorrectAVG
1	350	625	350	500
2	900	625	900	500
3	750	625	750	500
4	500	625	500	500
5	NULL	625	0	500

- Conditional aggregation, apply aggregate functions only on subsets of data that fulfill certain conditions

```
SELECT
    CustomerID,
    SUM(
        CASE
            WHEN Sales > 30 THEN 1
            ELSE 0
        END
    ) AS HighSales,
    COUNT(*) AS TotalOrders
FROM Sales.Orders
GROUP BY CustomerID
```

	CustomerID	HighSales	TotalOrders
1	1	1	3
2	2	0	3
3	3	2	3
4	4	1	1

b. Aggregation & analytical functions

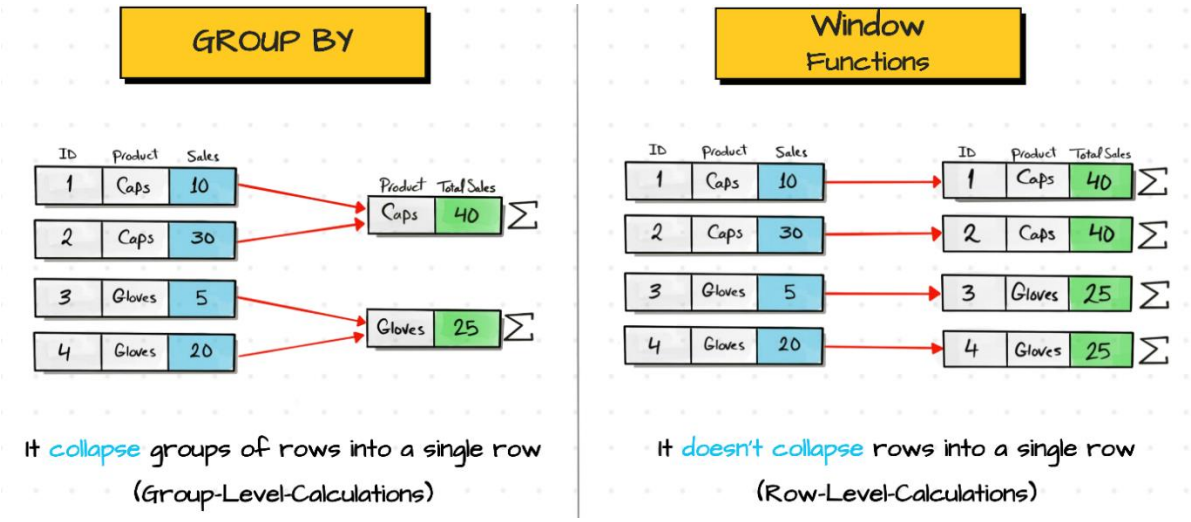
i. Aggregate functions

```
COUNT(expression) -- returns the number of non-NULL values in the column or all rows with COUNT(*)
SUM(numeric_expression) -- returns the sum of all values
AVG(numeric_expression) -- returns the average (mean) of all non-NULL values
MAX(ordenable_expression) -- returns the maximum value among all non-NULL values
MIN(ordenable_expression) -- returns the minimum value among all non-NULL values
```

ii. Window basics

Window/analytical functions: perform calculations on a specific subset of data, without losing the level of details of the rows

- Window vs GROUP BY



* GROUP BY can't do aggregations and provide details at the same time

```
SELECT
	OrderID,
	OrderDate,
	ProductID,
	SUM(Sales) OVER(PARTITION BY ProductID) AS TotalSalesByProduct
FROM Sales.Orders
```

	OrderID	OrderDate	ProductID	TotalSalesByProduct
1	1	2025-01-01	101	140
2	3	2025-01-10	101	140
3	8	2025-02-18	101	140
4	9	2025-03-10	101	140
5	10	2025-03-15	102	105
6	2	2025-01-05	102	105
7	7	2025-02-15	102	105
8	5	2025-02-01	104	75
9	6	2025-02-05	104	75
10	4	2025-01-20	105	60

- GROUP BY functions:

Aggregate functions

```
COUNT(expression)
SUM(numeric_expression)
AVG(numeric_expression)
MAX(ordenable_expression)
MIN(ordenable_expression)
```

- Window functions:

Aggregate functions

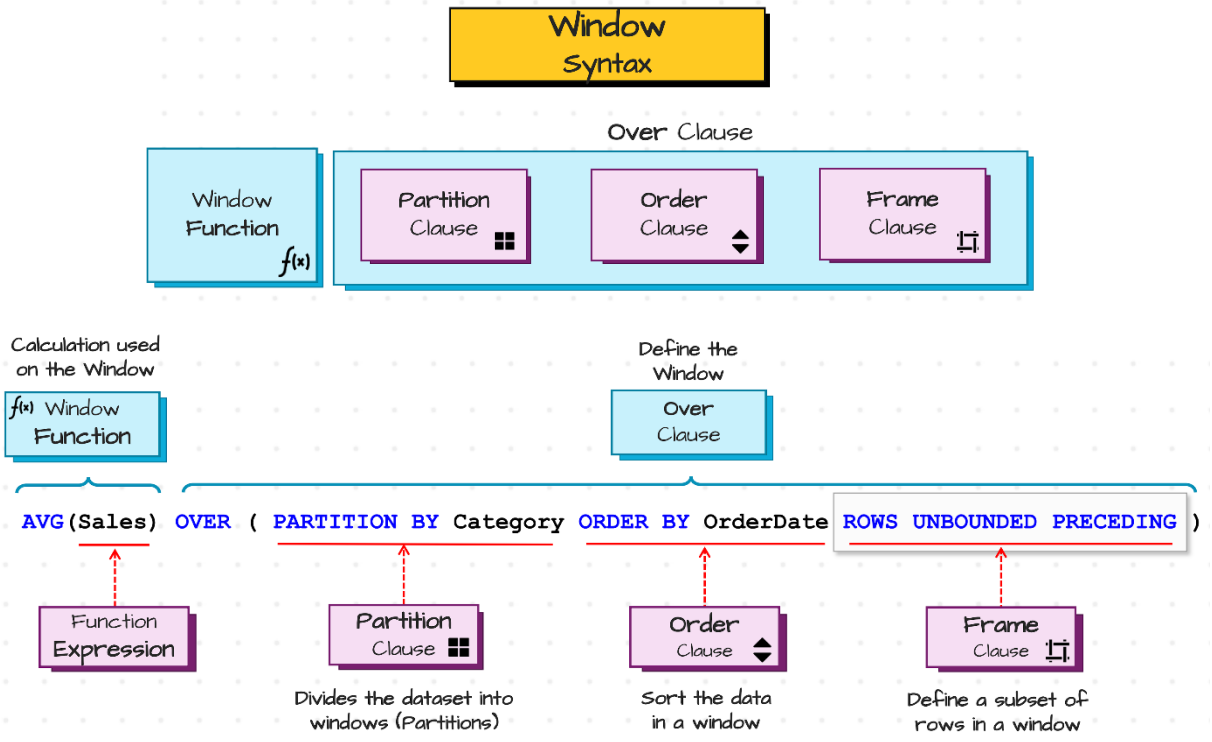
```
COUNT(expression)
SUM(numeric_expression)
AVG(numeric_expression)
MAX(ordenable_expression)
MIN(ordenable_expression)
```

Rank functions

```
ROW_NUMBER()
RANK()
DENSE_RANK()
CUME_DIST()
PERCENT_RANK()
NTILE(number)
```

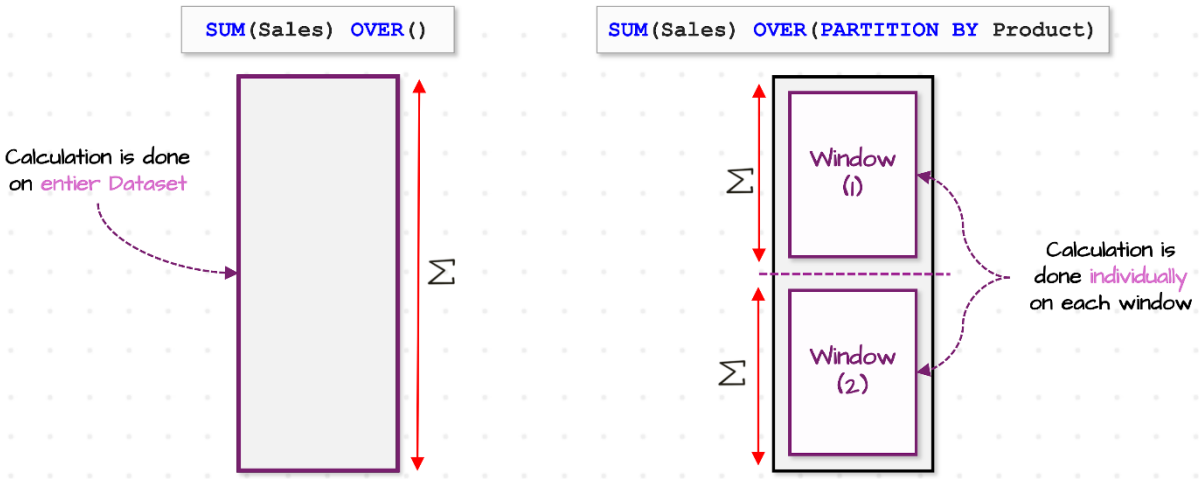
Value (analytics) functions

```
LAG(expression [, integer_offset [,
default]])
LEAD(expression [, integer_offset
[, default]])
FIRST_VALUE(expression)
LAST_VALUE(expression)
```



Partition By

PARTITION BY divides the rows into groups, based on the column/s



Without Partition By

Total sales across all rows (Entire Result Set)

```
SUM(Sales) OVER ()
```

Partition By Single Column

Total sales for each Product

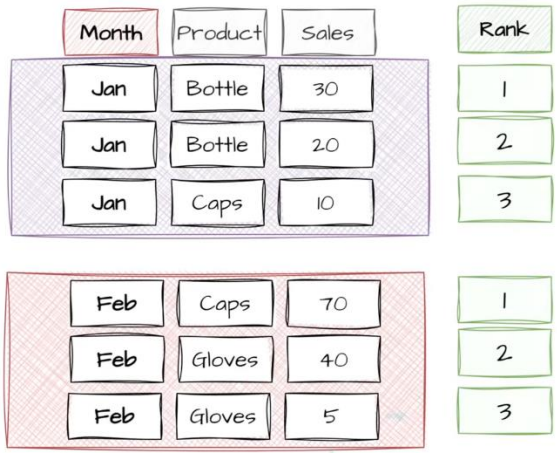
```
SUM(Sales) OVER (PARTITION BY Product)
```

Partition By Combined-Columns

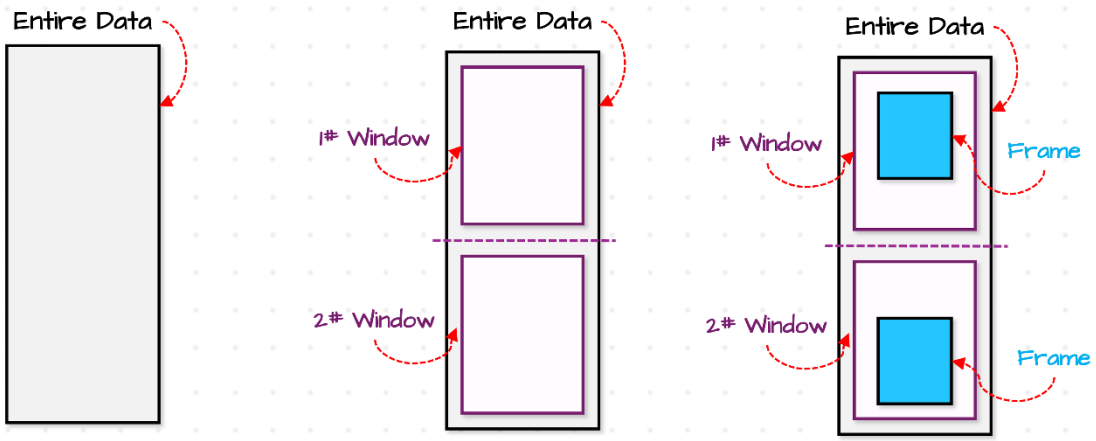
Total sales for each combination of Product and Order Status

```
SUM(Sales) OVER (PARTITION BY Product, OrderStatus)
```

```
RANK() OVER (PARTITION BY Month ORDER BY Sales DESC)
```



Frame Clause




```
AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate
    ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
```

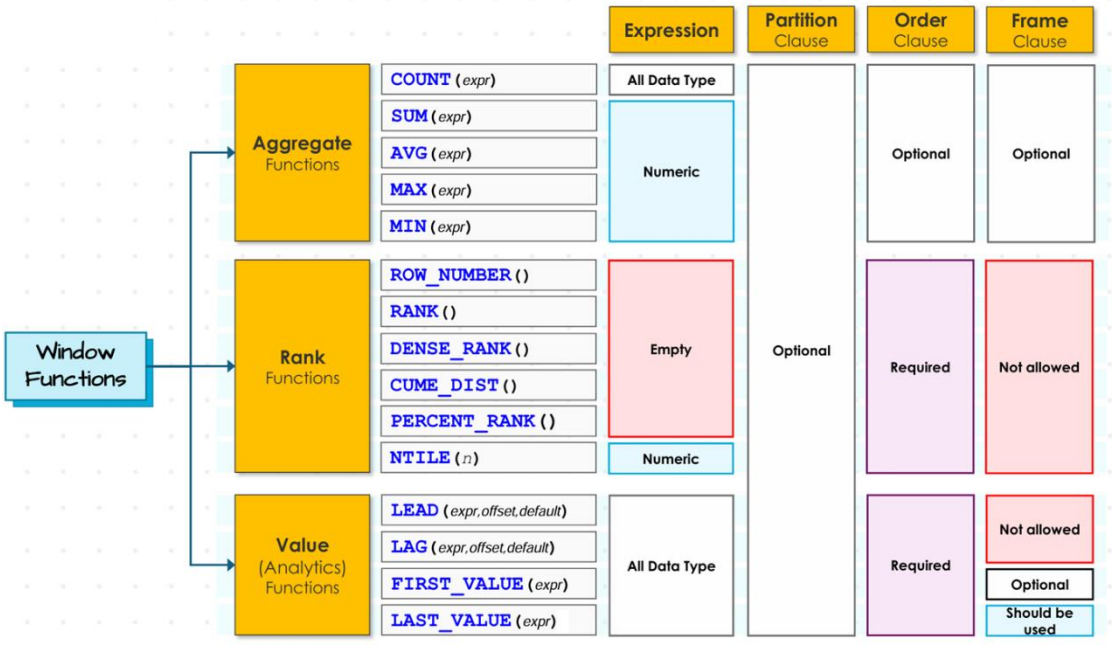
Frame Types
- ROWS
- RANGE

Frame Boundary (Lower Value)
- CURRENT ROW
- N PRECEDING
- UNBOUNDED PRECEDING

Frame Boundary (Higher Value)
- CURRENT ROW
- N FOLLOWING
- UNBOUNDED FOLLOWING

Rules

- Frame Clause can only be used together with order by clause.
- Lower Value must be BEFORE the higher Value.



- Características:
  - Como qualquer função de agregação, SQL executes window functions after WHERE clause
  - Window functions can be used together with GROUP BY in the same query, if the same columns are used. Example:

```
SELECT
	CustomerID,
	SUM(Sales) AS TotalSales,
	RANK() OVER(ORDER BY SUM(Sales) DESC) AS RankCustomers
FROM Sales.Orders
GROUP BY CustomerID
```

	CustomerID	TotalSales	RankCustomers
1	3	125	1
2	1	110	2
3	4	90	3
4	2	55	4

- Rules:
  - Window functions can only be used in SELECT and ORDER BY clauses
  - Nesting window functions is not allowed

iii. Window aggregate functions

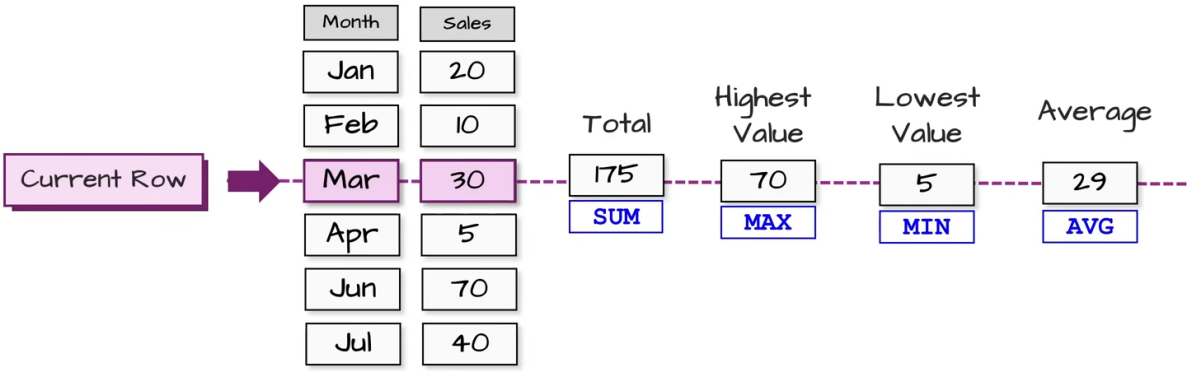
Calculate over a set of rows and return a single aggregated value for each one

- Usually in data analysis, we export a lot of data without rules like unique primary keys. So, to check the quality of the primary keys from the source, we can use COUNT():

```
SELECT *
FROM(
	SELECT
		OrderID,
		COUNT(*) OVER(PARTITION BY OrderID) AS CheckPrimaryKeys
	FROM Sales.OrdersArchive
) T
WHERE CheckPrimaryKeys > 1
```

	OrderID	CheckPrimaryKeys
1	4	2
2	4	2
3	6	3
4	6	3
5	6	3

- Comparison analysis (“part-to-whole analysis”, comparing the current value and aggregated value):




```sql
SELECT
	Sales,
	SUM(Sales) OVER() AS TotalSales,
	ROUND(CAST(Sales AS FLOAT) / SUM(Sales) OVER() * 100, 2) AS PercentageSales
FROM Sales.Orders
```

	Sales	TotalSales	PercentageSales
1	10	380	2,63
2	15	380	3,95
3	20	380	5,26
4	60	380	15,79
5	25	380	6,58
6	50	380	13,16
7	30	380	7,89
8	90	380	23,68
9	20	380	5,26
10	60	380	15,79

```sql
SELECT *
FROM(
	SELECT
		Sales,
		AVG(Sales) OVER() AS AverageSales
	FROM Sales.Orders
) T
WHERE Sales > AverageSales
```

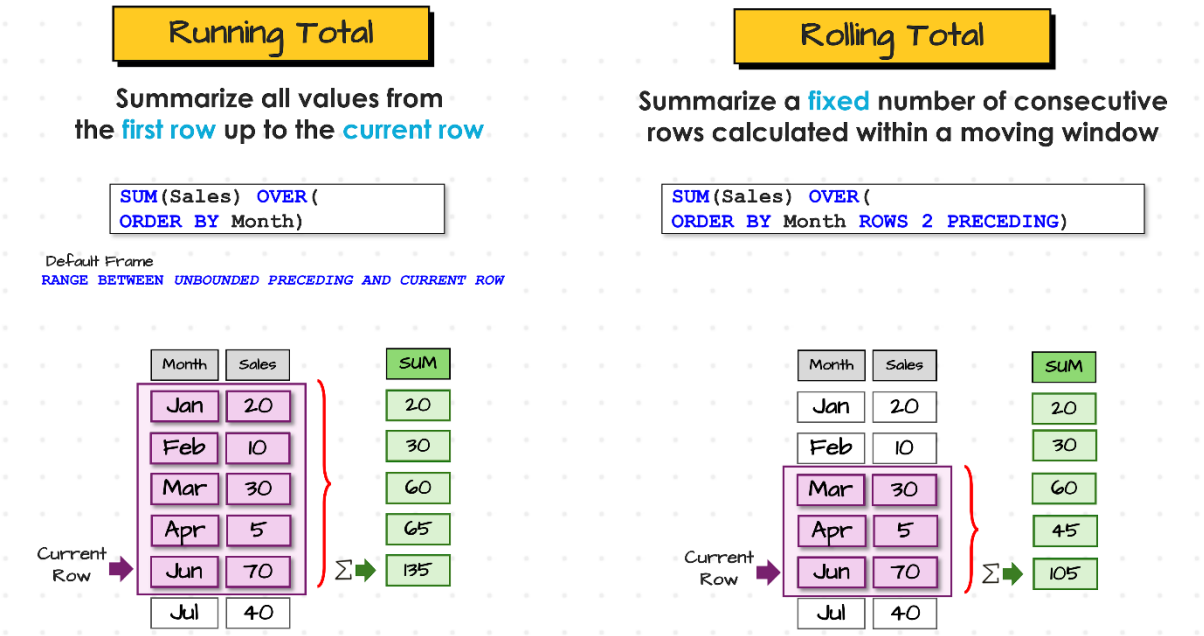
	Sales	AverageSales
1	60	38
2	50	38
3	90	38
4	60	38

```sql
SELECT
	Sales,
	MAX(Sales) OVER() AS HighestSales,
	MIN(Sales) OVER() AS LowestSales,
	Sales - MIN(Sales) OVER() AS DeviationFromMinimum,
	MAX(Sales) OVER() - Sales AS DeviationFromMaximum
FROM Sales.Orders
```

	Sales	HighestSales	LowestSales	DeviationFromMinimum	DeviationFromMaximum
1	10	90	10	0	80
2	15	90	10	5	75
3	20	90	10	10	70
4	60	90	10	50	30
5	25	90	10	15	65
6	50	90	10	40	40
7	30	90	10	20	60
8	90	90	10	80	0
9	20	90	10	10	70
10	60	90	10	50	30

- Running and rolling total:

Mainly for tracking current data with target data. Also used to do trend analysis (analysis based in historical patterns). They aggregate a sequence of members and the aggregation is updated each time we add a new member

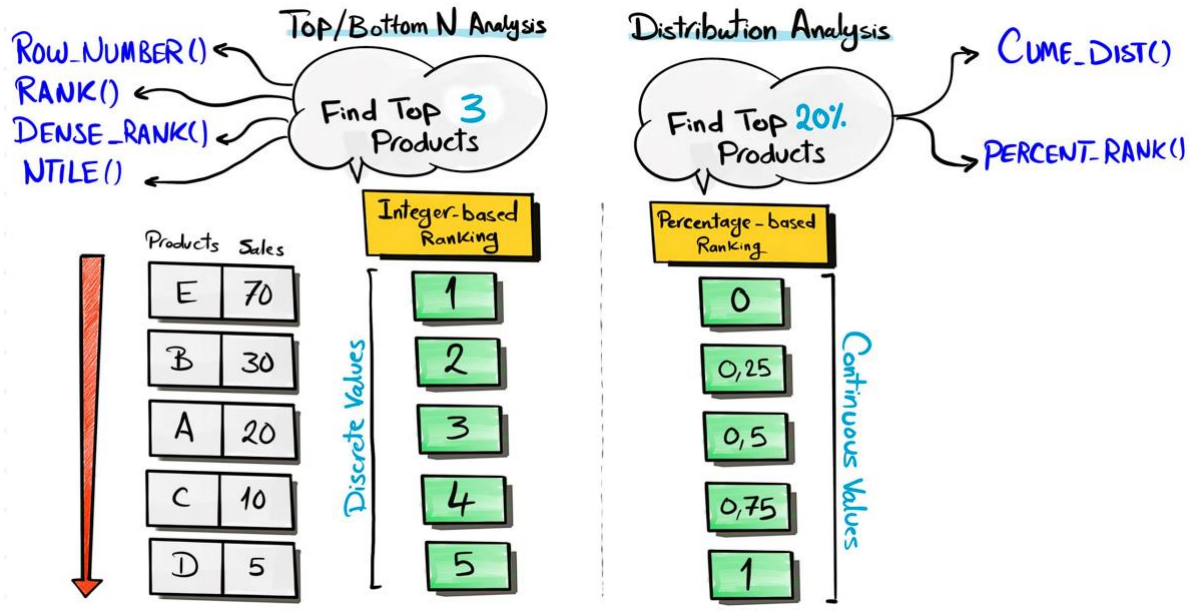


* Sem definir ORDER BY, o default frame é RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDING FOLLOWING

iv. Window ranking functions

Assign a rank to each row in a window

- 2 methods to rank the data:




```
ROW_NUMBER() OVER(ORDER BY coluna_ou_expressão) -- assign an unique sequential
integer to each row in a window without gaps and doesn't handle ties
RANK() OVER(ORDER BY coluna_ou_expressão) -- assign a rank to each row in a
window and handle ties (leaving gaps)
DENSE_RANK() OVER(ORDER BY coluna_ou_expressão) -- assign a rank to each row in a
window and handle ties (without gaps)
NTILE(number) OVER(ORDER BY coluna_ou_expressão) -- divides the rows into a
specified number of approximately equal groups/buckets (larger groups come first)

CUME_DIST() OVER(ORDER BY coluna_ou_expressão) --
= posição da linha / total de linhas;
calculates the cumulative distribution of each row
within the ordered set;
tie rule: the position of the last occurrence of the
same value
PERCENT_RANK() OVER(ORDER BY coluna_ou_expressão) --
= (posição da linha - 1) / (total de linhas - 1);
calculates the percentile rank of each row within the
ordered set;
tie rule: the position of the first occurrence of the
same value
```

Sales	DIST	Per
100	0,2	0
80	0,6	0,25
80	0,6	0,25
50	0,8	0,75
30	1	1

* To rank, SQL must sort the data, por isso precisam do ORDER BY

- ROW_NUMBER() use cases:
  - Top-N analysis (analysis the top performers to do targeted marketing) and bottom-N analysis (analysis the underperformance to manage risks and to do optimizations)

```
SELECT
	ProductID,
	Sales
FROM(
	SELECT
		ProductID,
		Sales,
		ROW_NUMBER() OVER(PARTITION BY ProductID ORDER BY Sales DESC) AS
ProductSalesRank
	FROM Sales.Orders
) T
WHERE ProductSalesRank = 1
```

	ProductID	Sales
1	101	90
2	102	60
3	104	50
4	105	60

  - Generate/assign unique IDs (identifier for each row helps paginating – the process of breaking down a large data into smaller, more manageable chunks)

```
SELECT
	OrderID,
	OrderDate,
	ROW_NUMBER() OVER(ORDER BY OrderID, OrderDate) AS UniqueID
FROM Sales.OrdersArchive
```

	OrderID	OrderDate	UniqueID
1	1	2024-04-01	1
2	2	2024-04-05	2
3	3	2024-04-10	3
4	4	2024-04-20	4
5	4	2024-04-20	5
6	5	2024-05-01	6
7	6	2024-05-05	7
8	6	2024-05-05	8
9	6	2024-05-05	9
10	7	2024-06-15	10

- Identify duplicates

```
SELECT
	OrderID,
	OrderStatus,
	CreationTime
FROM(
	SELECT
		OrderID,
		OrderStatus,
		CreationTime,
		ROW_NUMBER() OVER(PARTITION BY OrderID ORDER BY CreationTime DESC)
AS LastStatus
	FROM Sales.OrdersArchive
) T
WHERE LastStatus = 1
```

	OrderID	OrderStatus	CreationTime
1	1	Shipped	2024-04-01 12:34:56.0000000
2	2	Shipped	2024-04-05 23:22:04.0000000
3	3	Shipped	2024-04-10 18:24:08.0000000
4	4	Delivered	2024-04-20 14:50:33.0000000
5	5	Shipped	2024-05-01 14:02:41.0000000
6	6	Delivered	2024-05-12 20:36:55.0000000
7	7	Shipped	2024-06-16 23:25:15.0000000

- NTILE() use cases:
  - Data segmentation (dividing a dataset into distinct subsets based on a certain criteria)

```
SELECT
	Sales,
	CASE
		WHEN Buckets = 1 THEN 'High'
		WHEN Buckets = 2 THEN 'UpperMedium'
		WHEN Buckets = 3 THEN 'LowerMedium'
		WHEN Buckets = 4 THEN 'Low'
	END AS BucketsRank
FROM(
	SELECT
		Sales,
		NTILE(4) OVER(ORDER BY Sales DESC) AS Buckets
	FROM Sales.Orders
) T
```

	Sales	BucketsRank
1	90	High
2	60	High
3	60	High
4	50	UpperMedium
5	30	UpperMedium
6	25	UpperMedium
7	20	LowerMedium
8	20	LowerMedium
9	15	Low
10	10	Low

- Equalize/balance loading process of ETLs (moving a large table between databases (dataflow) can take days and may encounter network errors due to network stress. So, instead of loading the table all at once, we can split it into smaller buckets and move them sequentially)

```
SELECT
	NTILE(2) OVER(ORDER BY OrderID) AS Buckets,
	*
FROM Sales.Orders
```

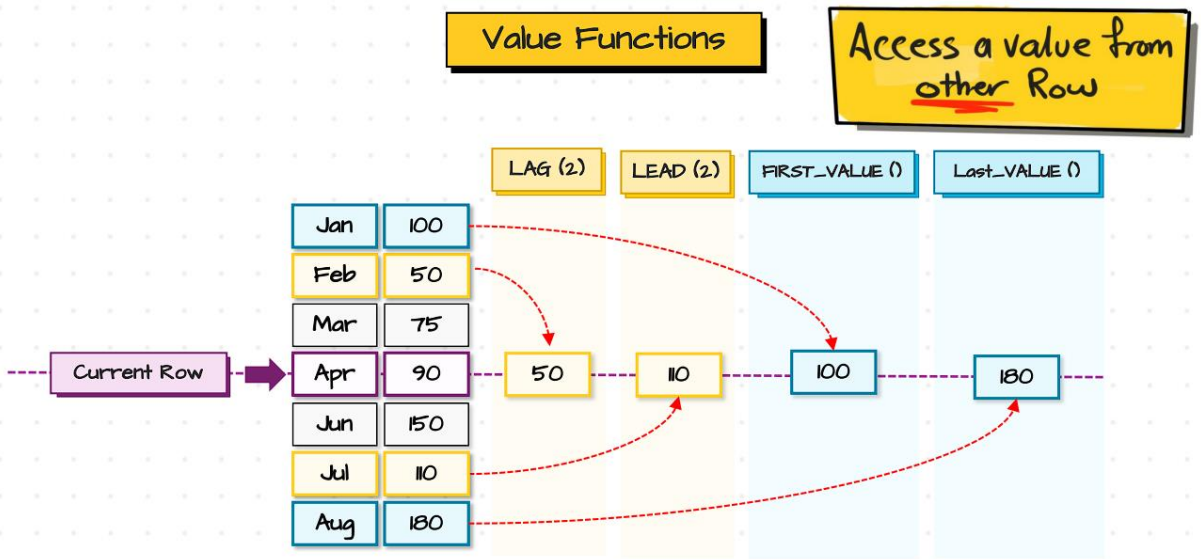
- CUME_DIST() use case:

```
SELECT
	Product,
	Price,
	CONCAT(DistRank * 100, '%') AS FortyPercentHighestPrices
FROM(
	SELECT
		Product,
		Price,
		CUME_DIST() OVER(ORDER BY Price DESC) DistRank
	FROM Sales.Products
) T
WHERE DistRank <= 0.4
```

	Product	Price	FortyPercentHighestPrices
1	Gloves	30	20%
2	Caps	25	40%

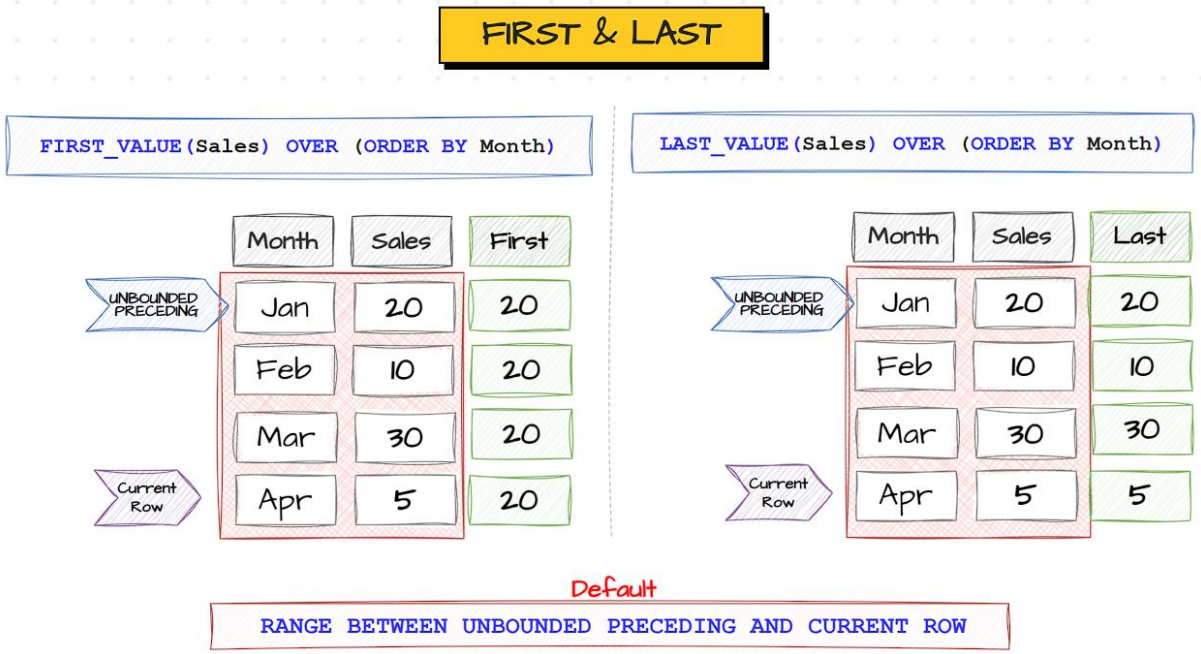
v. Window value functions

Return a specific value in a window to be compared with the value of current row




```
LAG(expression [, integer_offset [, default]]) OVER(ORDER BY coluna_ou_expressão)
-- returns the value from a previous row
LEAD(expression [, integer_offset [, default]]) OVER(ORDER BY
coluna_ou_expressão) -- returns the value from a subsequent row
FIRST_VALUE(expression) OVER(ORDER BY coluna_ou_expressão) -- returns the first
value in a window (frame default = RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW)
LAST_VALUE(expression) OVER(ORDER BY coluna_ou_expressão) -- returns the last
value in a window (frame default = RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW, então é recomendado usar ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
```

* As funções dependem de ordem, por isso precisam do ORDER BY



- LAG() / LEAD() use cases:
  - "Time series analysis" is the main use case – one of the most important questions you're going to get from the decision makers or business is to do Year-over-Year (YoY) or Month-over-Month (MoM) analysis

```
SELECT
	OrderMonth,
	CurrentMonthSales,
	CurrentMonthSales - PreviousMonthSales AS DiffFromPreviousSales,
	NextMonthSales - CurrentMonthSales AS DiffToNextSales,
	CAST(((CurrentMonthSales - PreviousMonthSales) * 100.0) /
PreviousMonthSales AS DECIMAL(10, 2)) AS PercentualDiffFromPreviousSales,
	CAST(((NextMonthSales - CurrentMonthSales) * 100.0) / CurrentMonthSales AS
DECIMAL(10, 2)) AS PercentualDiffToNextSales
FROM(
	SELECT
		MONTH(OrderDate) AS OrderMonth,
		SUM(Sales) AS CurrentMonthSales,
		LAG(SUM(Sales)) OVER(ORDER BY MONTH(OrderDate)) AS
PreviousMonthSales,
		LEAD(SUM(Sales)) OVER(ORDER BY MONTH(OrderDate)) AS NextMonthSales
	FROM Sales.Orders
	GROUP BY MONTH(OrderDate)
) T
```

	OrderMonth	CurrentMonthSales	DifferenceFromPreviousSales	DifferenceToNextSales	PercentualDiffFromPreviousSales	PercentualDiffToNextSales
1	1	105	NULL	90	NULL	85.71
2	2	195	90	-115	85.71	-58.97
3	3	80	-115	NULL	-58.97	NULL

  - Customer retention analysis (measure customer's behavior to help businesses build strong relationships with them)

```
SELECT
	*,
	DATEDIFF(DAY, PreviousOrderDate, OrderDate) AS DaysAfterPreviousOrder,
	AVG(DATEDIFF(DAY, PreviousOrderDate, OrderDate)) OVER(PARTITION BY
CustomerID) AS AverageDaysBetweenOrders
FROM(
	SELECT
		CustomerID,
		OrderDate,
		LAG(OrderDate) OVER(PARTITION BY CustomerID ORDER BY OrderDate) AS
PreviousOrderDate
	FROM Sales.Orders
) T
```

	CustomerID	OrderDate	PreviousOrderDate	DaysAfterPreviousOrder	AverageDaysBetweenOrders
1	1	2025-01-10	NULL	NULL	18
2	1	2025-01-20	2025-01-10	10	18
3	1	2025-02-15	2025-01-20	26	18
4	2	2025-01-01	NULL	NULL	34
5	2	2025-02-01	2025-01-01	31	34
6	2	2025-03-10	2025-02-01	37	34
7	3	2025-01-05	NULL	NULL	34
8	3	2025-02-05	2025-01-05	31	34
9	3	2025-03-15	2025-02-05	38	34
10	4	2025-02-18	NULL	NULL	NULL

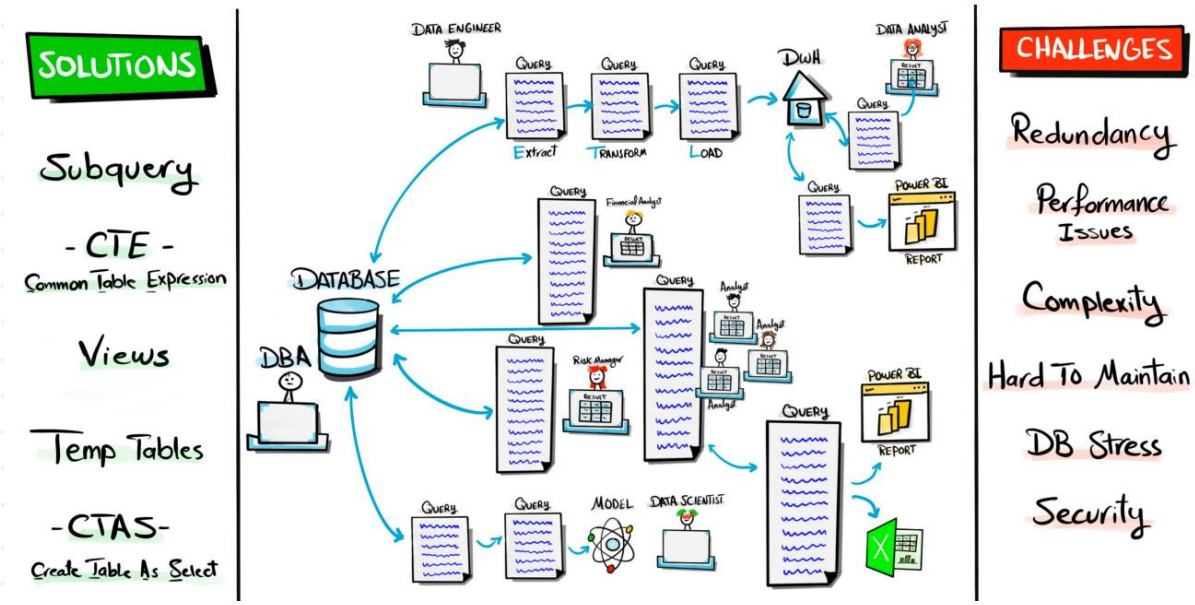
- FIRST_VALUE() / LAST_VALUE() use cases:
    - Comparison analysis with extremes

```
SELECT
	ProductID,
	Sales,
	FIRST_VALUE(Sales) OVER(PARTITION BY ProductID ORDER BY Sales DESC) AS
HighestSalesByProduct,
	LAST_VALUE(Sales) OVER(PARTITION BY ProductID ORDER BY Sales DESC ROWS
BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) AS LowestSalesByProduct,
	FIRST_VALUE(Sales) OVER(PARTITION BY ProductID ORDER BY Sales) AS
BetterLowerSalesByProduct,
	Sales - MIN(Sales) OVER(PARTITION BY ProductID) AS SalesMinDiff
FROM Sales.Orders
```

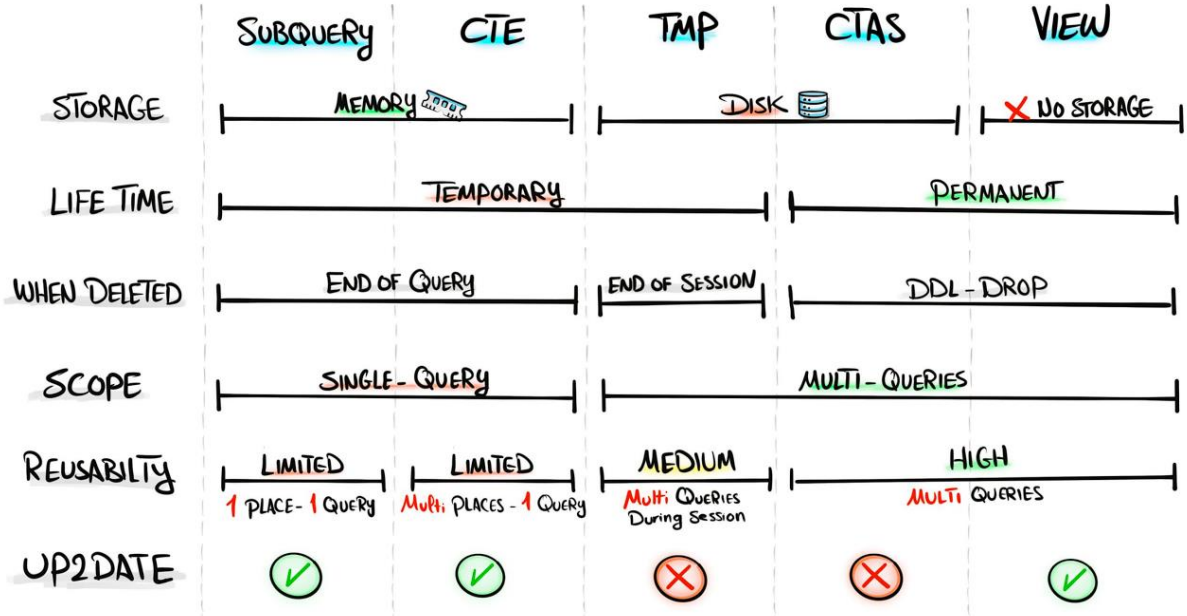
	ProductID	Sales	HighestSalesByProduct	LowestSalesByProduct	BetterLowerSalesByProduct	SalesMinDiff
1	101	10	90	10	10	0
2	101	20	90	10	10	10
3	101	20	90	10	10	10
4	101	90	90	10	10	80
5	102	15	60	15	15	0
6	102	30	60	15	15	15
7	102	60	60	15	15	45
8	104	25	50	25	25	0
9	104	50	50	25	25	25
10	105	60	60	60	60	0

Advanced Level

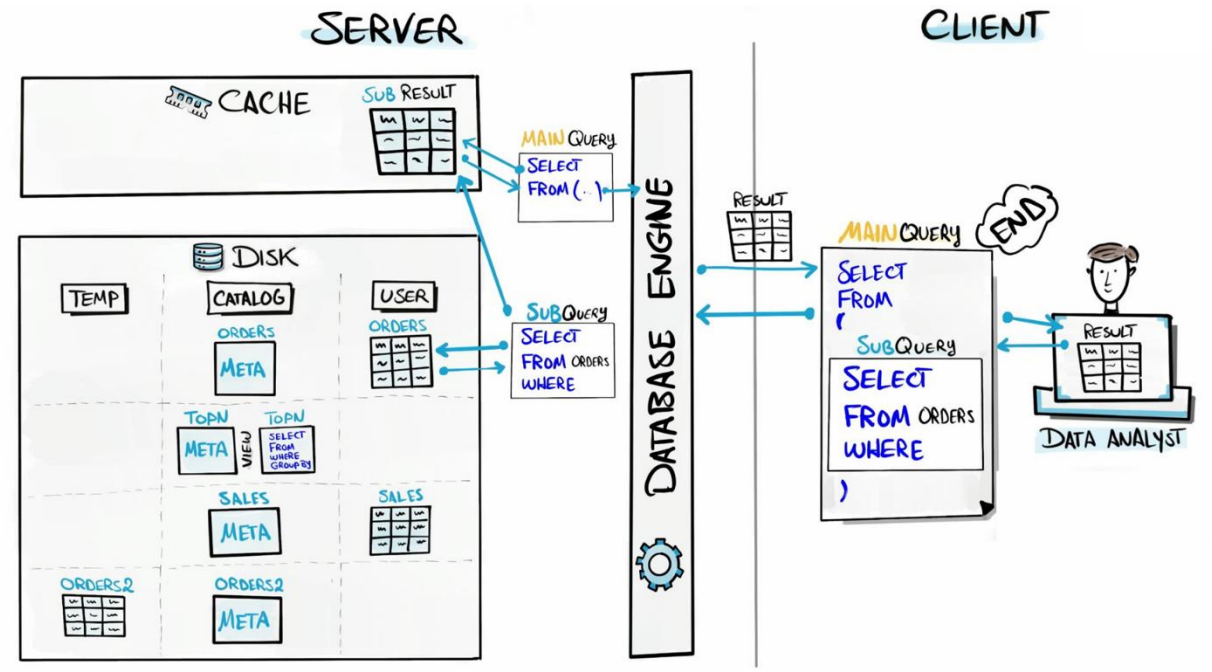
8. Advanced SQL techniques



* Data Warehouse: a special database that collects and integrates data from different sources to enable analytics and support decision-making

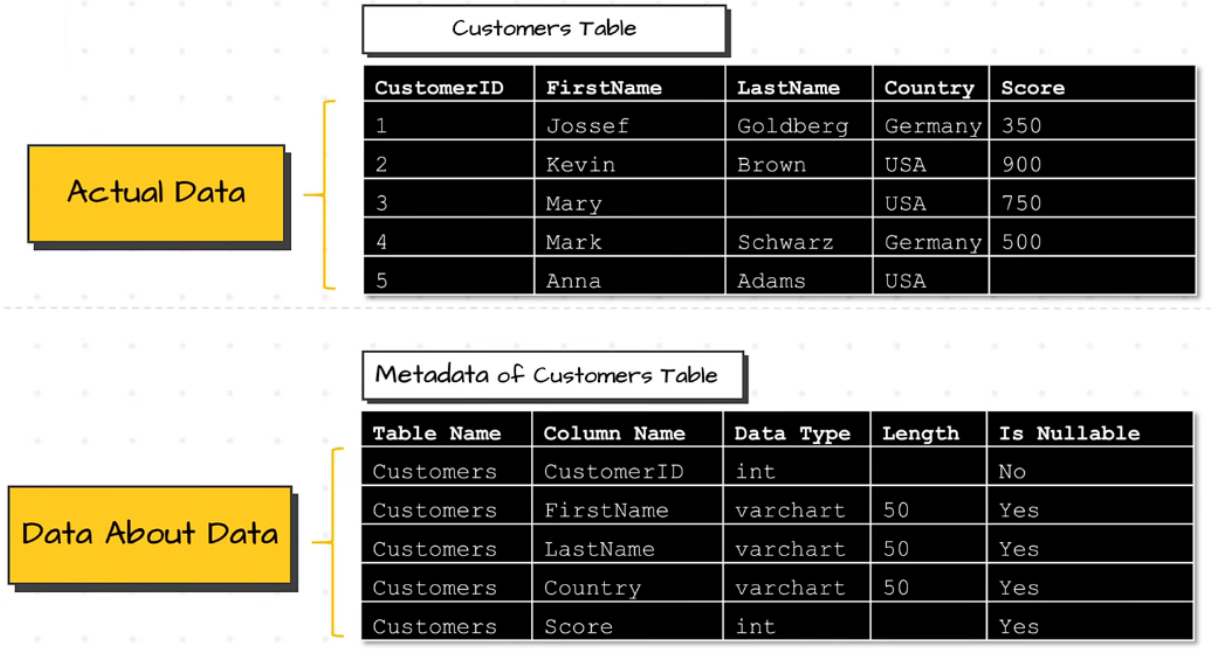


- Simplified Database Architecture:



* Database engine: it is the brain of the database, executing multiple operations such as storing, retrieving and managing data within the database

- 2 main types of storage in a database:
  - Disk storage → Long-term memory, where data is stored permanently, even if you turn of the system. It can hold a large amount of data, but it's slower to read and to write
    - User data storage → The main content of the database, where the actual data that users care about is stored
    - System catalog storage → Internal storage of the database for its own information. It holds the metadata information about the database. You can find those information in a special hidden schema, the "information schema" (a system-defined schema with built-in views that provide info about the database, like tables and columns)



CustomerID	FirstName	LastName	Country	Score
1	Jossef	Goldberg	Germany	350
2	Kevin	Brown	USA	900
3	Mary		USA	750
4	Mark	Schwarz	Germany	500
5	Anna	Adams	USA	

Table Name	Column Name	Data Type	Length	Is Nullable
Customers	CustomerID	int		No
Customers	FirstName	varchart	50	Yes
Customers	LastName	varchart	50	Yes
Customers	Country	varchart	50	Yes
Customers	Score	int		Yes

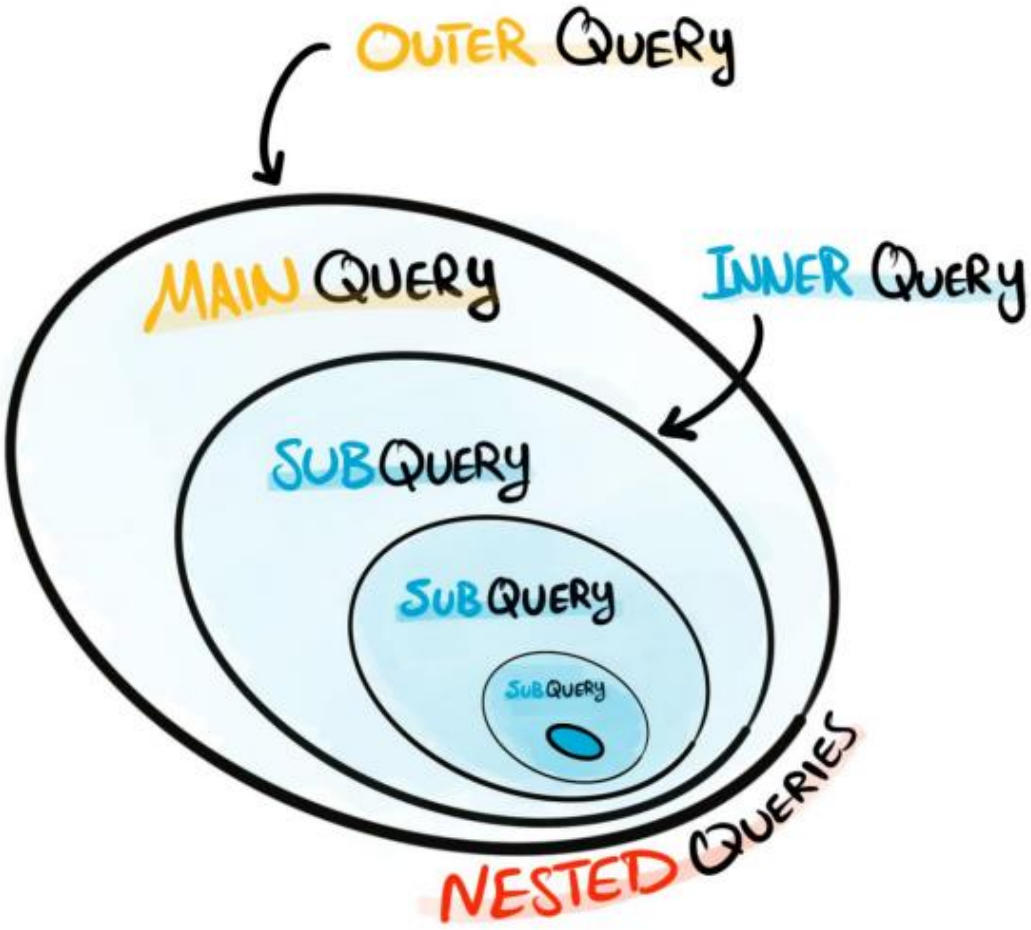
```
SELECT * FROM INFORMATION_SCHEMA.TABLES
```

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
1	SalesDB	Sales	Customers	BASE TABLE
2	SalesDB	Sales	Employees	BASE TABLE
3	SalesDB	Sales	Products	BASE TABLE
4	SalesDB	Sales	Orders	BASE TABLE
5	SalesDB	Sales	OrdersArchive	BASE TABLE

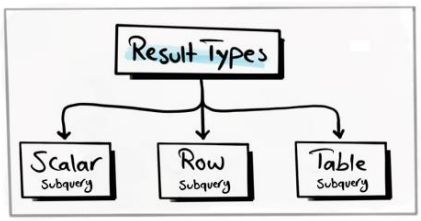
    - Temporary storage → Temporary space used by the database for short-term tasks, like processing queries or sorting data. Once these tasks are done, the storage is cleared
  - Cache storage → Fast short-term memory, where data is stored temporarily (for example, results of subqueries and results of CTEs). It can hold less amount of data

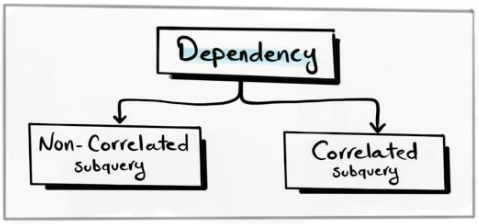
c. Subqueries

The main purpose is to reduce the complexity of the query and make it more readable

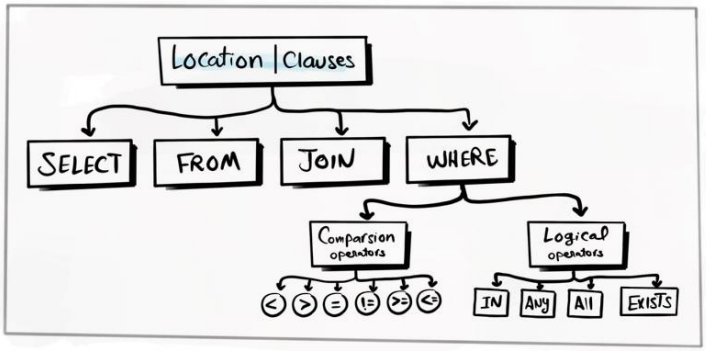


Categories:



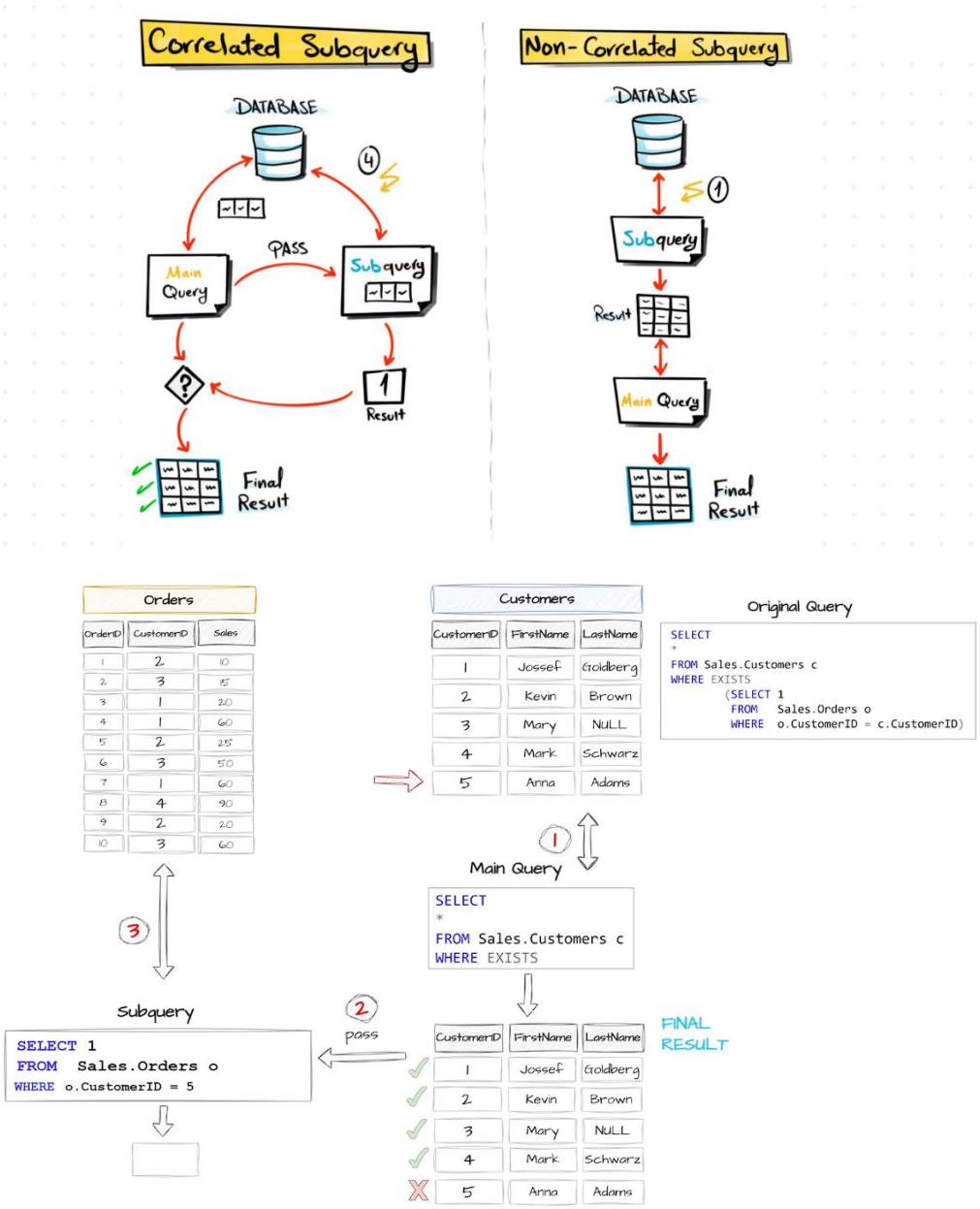


SUBQUERY



- Result types
  - Scalar subquery (returns a single value)
  - Row subquery (returns a single column and multiple rows)
  - Table subquery (returns multiple columns)

- Dependency
  - Non-correlated subquery → Subquery that can run independently from the main query, used for static comparisons and filtering with constants
  - Correlated subquery → Subquery that relays on values from the main query, used for row-by-row comparisons and dynamic filtering



* In the correlated subquery, SQL executes the main query row by row, by iterations – and for each one, it executes the subquery using the row's values, and the result of the subquery determines whether that row is included in the final output. It's dependent of the main query and it has worse performance than the non-correlated subquery

```sql
SELECT
	*,
	(SELECT COUNT(*) FROM Sales.Orders O WHERE O.CustomerID = C.CustomerID) AS
TotalSales
FROM Sales.Customers C
```

	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	0

- Location/Clauses
  - SELECT → Only scalar subqueries are allowed. Used to aggregate data side by side with the main query's data, allowing for direct comparison
  - FROM → Typically used to create temporary results for the main query
  - JOIN → Used to prepare the data (filtering or aggregation) before joining it with other tables

```sql
SELECT
	C.*,
	O.TotalOrders
FROM Sales.Customers C
LEFT JOIN (
	SELECT
		CustomerID,
		COUNT(*) AS TotalOrders
	FROM Sales.Orders
	GROUP BY CustomerID
) O
ON C.CustomerID = O.CustomerID
```

	CustomerID	FirstName	LastName	Country	Score	TotalOrders
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	NULL

  - WHERE → With comparison operators, only scalar subqueries are allowed; with logical operators (IN, NOT IN, EXISTS, NOT EXISTS, ANY, ALL), a row subquery is allowed. Used for complex/dynamic filtering logic

```sql
SELECT
	ProductID,
	Price
FROM Sales.Products
WHERE Price > ( -- comparison operator
	SELECT AVG(Price) FROM Sales.Products
) -- scalar subquery
```

	ProductID	Price
1	104	25
2	105	30

```sql
SELECT
	CustomerID,
	ProductID,
	Sales
FROM Sales.Orders
WHERE CustomerID IN ( -- logical operator IN
	SELECT CustomerID FROM Sales.Customers WHERE Country = 'Germany'
) -- row subquery
```

	CustomerID	ProductID	Sales
1	1	101	20
2	1	105	60
3	1	102	30
4	4	101	90

```sql
SELECT
	EmployeeID,
	FirstName,
	Gender,
	Salary
FROM Sales.Employees
WHERE Gender = 'F' AND Salary > ANY ( -- logical operator ANY (returns TRUE if a
value matches any value in a list)
	SELECT Salary FROM Sales.Employees WHERE Gender = 'M'
) -- row subquery
```

	EmployeeID	FirstName	Gender	Salary
1	3	Mary	F	75000

```sql
SELECT
	EmployeeID,
	FirstName,
	Gender,
	Salary
FROM Sales.Employees
WHERE Gender = 'M' AND Salary > ALL ( -- logical operator ALL (returns TRUE if a
value matches all values in a list)
	SELECT Salary FROM Sales.Employees WHERE Gender = 'F'
) -- row subquery
```

	EmployeeID	FirstName	Gender	Salary
1	4	Michael	M	90000

```sql
SELECT
	O.CustomerID,
	O.Quantity,
	O.Sales
FROM Sales.Orders O
WHERE EXISTS ( -- logical operator EXISTS (checks if subquery returns any rows)
/* Ou: WHERE NOT EXISTS ( -- logical operator NOT EXISTS (checks if subquery
returns no rows) */
	SELECT 1 FROM Sales.Customers C
	WHERE Country = 'Germany' AND O.CustomerID = C.CustomerID
)
```

(EXISTS)

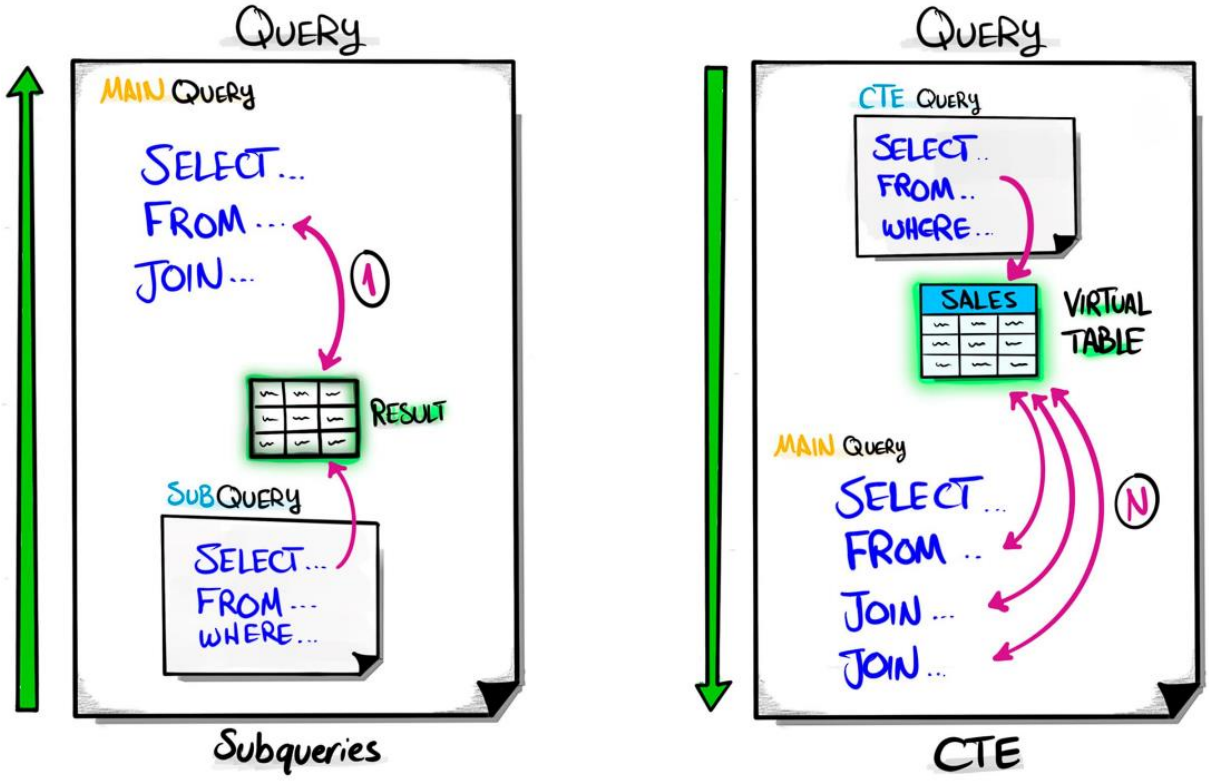
	CustomerID	Quantity	Sales
1	1	2	20
2	1	2	60
3	1	2	30
4	4	3	90

	CustomerID	Quantity	Sales
1	2	1	10
2	3	1	15
3	2	1	25
4	3	2	50
5	2	2	20
6	3	0	60

(NOT EXISTS)

d. CTE

Common table expression: temporary, named result set (virtual table), to simplify and organize complex query (like the subqueries), but that can be used multiple times within your query, avoiding redundancy

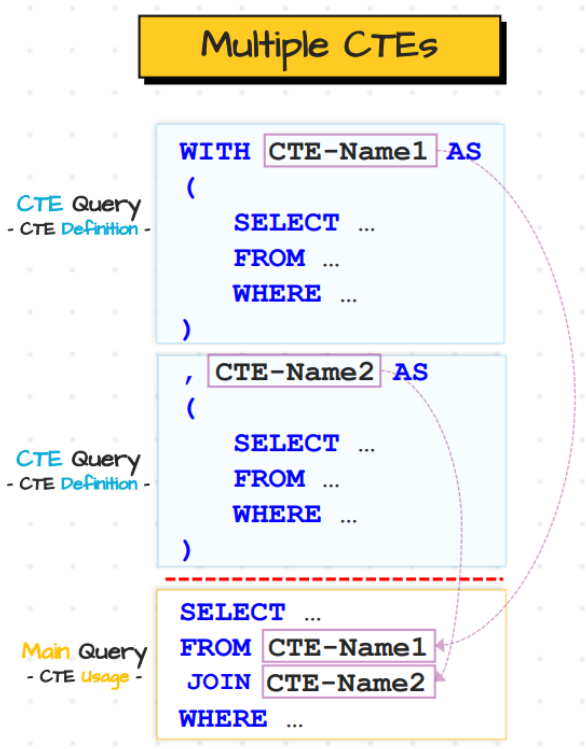


- Restriction: you can't use ORDER BY directly within the CTE

Categories:

- Non-recursive CTE → Executed once, without repetition
    - Standalone CTE → Defined and used independently (it's self-contained and doesn't rely on other CTEs or queries)

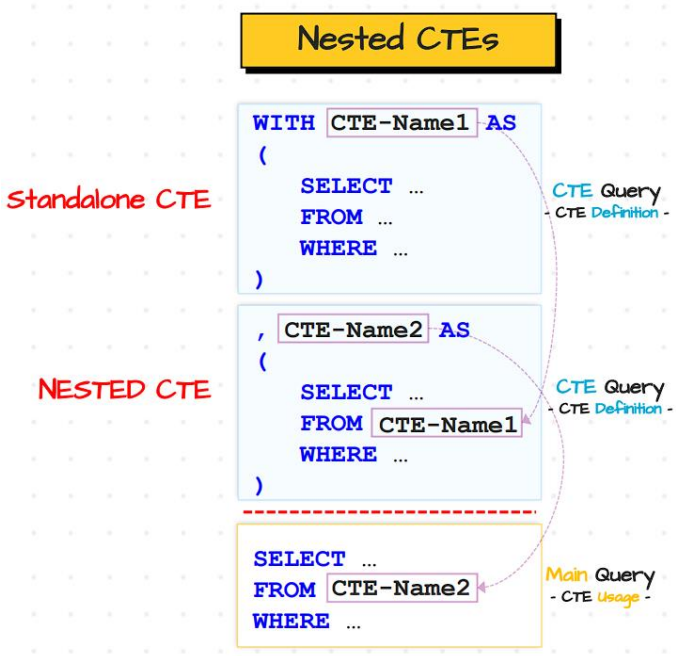
```sql
WITH CTE_Total_Sales AS (
	SELECT
		CustomerID,
		SUM(Sales) AS TotalSales
	FROM Sales.Orders
	GROUP BY CustomerID
),
CTE_Last_Order AS (
	SELECT
		CustomerID,
		MAX(OrderDate) AS
Last_Order
	FROM Sales.Orders
	GROUP BY CustomerID
)
SELECT
	C.CustomerID,
	C.FirstName,
	C.LastName,
	CTS.TotalSales,
	CLO.Last_Order
FROM Sales.Customers C
LEFT JOIN CTE_Total_Sales CTS
ON CTS.CustomerID = C.CustomerID
LEFT JOIN CTE_Last_Order CLO
ON CLO.CustomerID = C.CustomerID
```



	CustomerID	FirstName	LastName	TotalSales	Last_Order
1	1	Jossef	Goldberg	110	2025-02-15
2	2	Kevin	Brown	55	2025-03-10
3	3	Mary	NULL	125	2025-03-15
4	4	Mark	Schwarz	90	2025-02-18
5	5	Anna	Adams	NULL	NULL

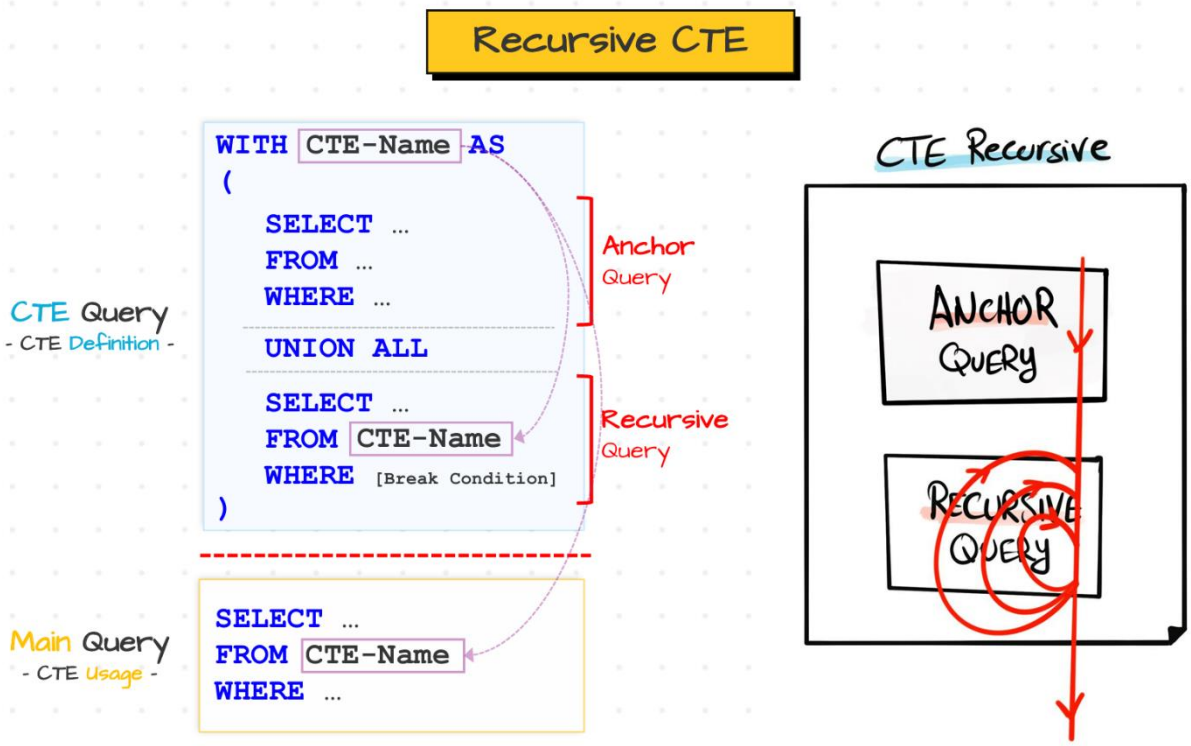
- Nested CTE → A nested CTE uses the result of another CTE, so it can't run independently

```
WITH CTE_Total_Sales AS ({same as the previous exercise}),
CTE_Last_Order AS ({same as the previous exercise}),
CTE_Customer_Rank AS (
	SELECT
		CustomerID,
		TotalSales,
		RANK() OVER(ORDER BY TotalSales DESC) AS CustomerRank
	FROM CTE_Total_Sales
),
CTE_Customer_Segments AS (
	SELECT
		CustomerID,
		CASE
			WHEN TotalSales > 100 THEN 'High'
			WHEN TotalSales > 80 THEN 'Medium'
			ELSE 'Low'
		END AS CustomerSegments
	FROM CTE_Total_Sales
)
SELECT
	C.CustomerID,
	C.FirstName,
	C.LastName,
	CTS.TotalSales,
	CLO.Last_Order,
	CCR.CustomerRank,
	CCS.CustomerSegments
FROM Sales.Customers C
LEFT JOIN CTE_Total_Sales CTS
ON CTS.CustomerID = C.CustomerID
LEFT JOIN CTE_Last_Order CLO
ON CLO.CustomerID = C.CustomerID
LEFT JOIN CTE_Customer_Rank CCR
ON CCR.CustomerID = C.CustomerID
LEFT JOIN CTE_Customer_Segments CCS
ON CCS.CustomerID = C.CustomerID
ORDER BY
	CASE
		WHEN CTS.TotalSales IS
NULL THEN 1
		ELSE 0
	END,
	CCR.CustomerRank
```



	CustomerID	FirstName	LastName	TotalSales	Last_Order	CustomerRank	CustomerSegments
1	3	Mary	NULL	125	2025-03-15	1	High
2	1	Jossef	Goldberg	110	2025-02-15	2	High
3	4	Mark	Schwarz	90	2025-02-18	3	Medium
4	2	Kevin	Brown	55	2025-03-10	4	Low
5	5	Anna	Adams	NULL	NULL	NULL	NULL

- Recursive CTE → Self-referencing query that repeatedly processes data until a specific condition is met. Normally used to navigate through hierarchical structures



  - Break condition: can be used in the WHERE clause or in an INNER JOIN
  - Anchor query: the first to interact with the database, executed only once (the starting point of the iteration), provides the initial intermediate results
  - Recursive query: executed multiple times, will keep repeating and adding data to the intermediate results until the condition is met or there's no more data left to be processed
  - To connect both SELECT statements: UNION ALL (to keep duplicates) or UNION (to remove duplicates)

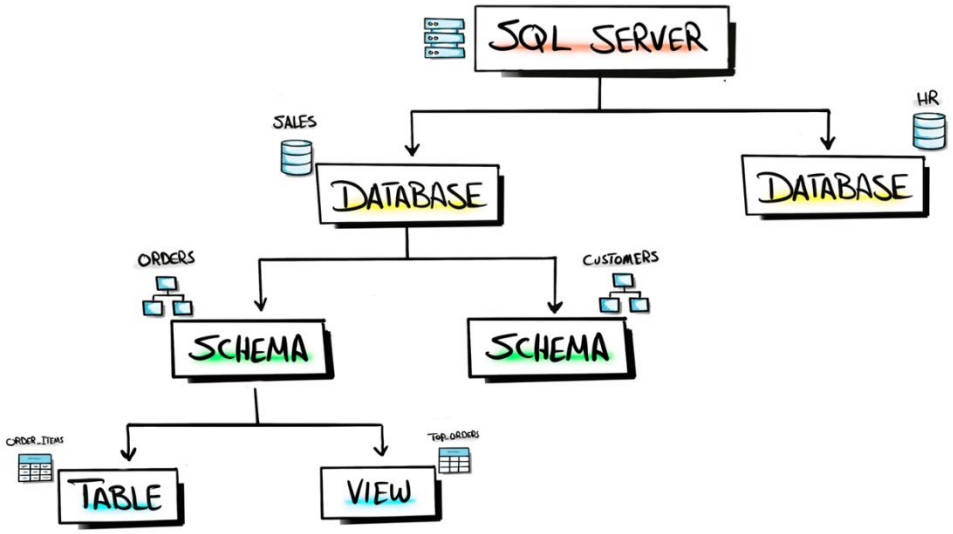
```
WITH Series AS (
      -- Anchor query
      SELECT 1 AS Number
      UNION ALL
      -- Recursive query
      SELECT Number + 1
      FROM Series
      WHERE Number < 10
)
-- Main query
SELECT *
FROM Series
OPTION (MAXRECURSION 9) -- maximum number of recursions (default = 100)
```

```sql
WITH CTE_Employee_Hierarchy AS (
	-- Anchor query
	SELECT
		EmployeeID,
		ManagerID,
		FirstName,
		1 AS Level
	FROM Sales.Employees
	WHERE ManagerID IS NULL
	UNION ALL
	-- Recursive query
	SELECT
		E.EmployeeID,
		E.ManagerID,
		E.FirstName,
		Level + 1
	FROM Sales.Employees E
	INNER JOIN CTE_Employee_Hierarchy CEH
	ON E.ManagerID = CEH.EmployeeID
)
-- Main query
SELECT *
FROM CTE_Employee_Hierarchy
```

	EmployeeID	ManagerID	FirstName	Level
1	1	NULL	Frank	1
2	2	1	Kevin	2
3	3	1	Mary	2
4	5	3	Carol	3
5	4	2	Michael	3

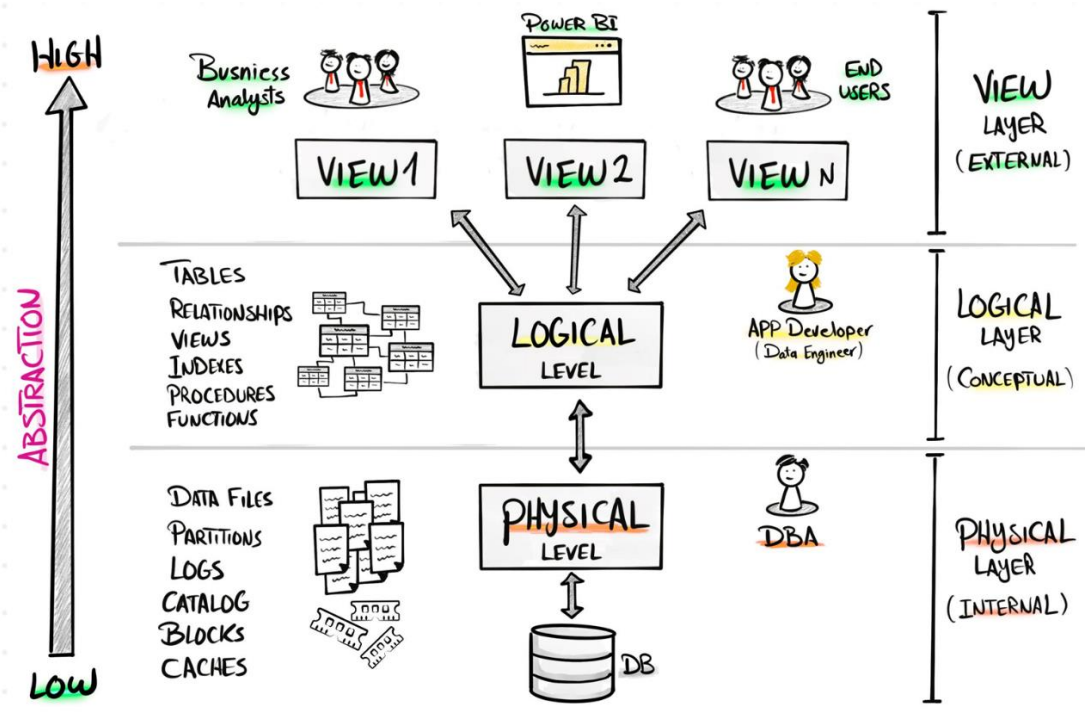
- ✓ Best practice: excess of CTEs can worsen the readability and maintainability of your query (especially if they're nested), so rethink and refactor your CTEs before starting a new one – in general, try not to exceed 5 CTEs in one query

e. Views



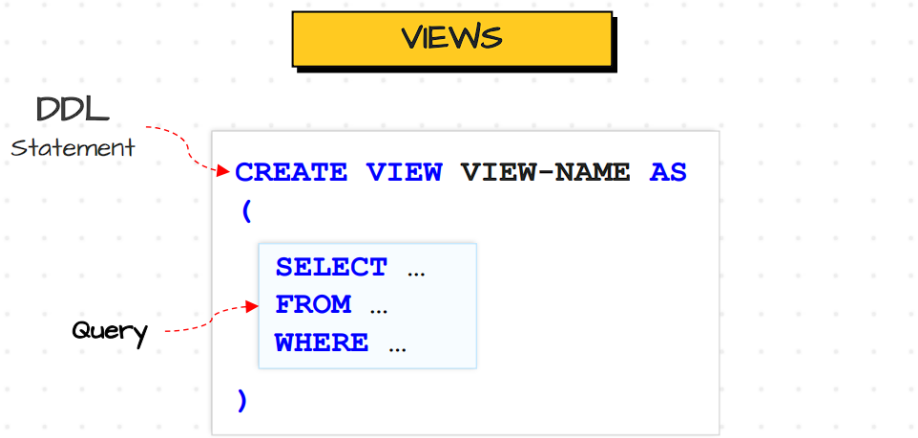
- Database server: stores, manages and provides access to databases for users or applications
    - Database: collection of information stored in a structured way
        - Schema: logical layer that groups related objects together
            - Table: where data is organized into rows and columns (is stored physically in the disk storage as database files)
            - View: virtual table (abstraction layer between user and tables) based on the result set of a query (persisted SQL query in the database) that doesn't store data physically. Used to centralize query logic and to secure sensitive data

- Database 3 levels architecture (levels of data abstraction):



- Views x CTEs

Reduce redundance and improve reusability in multi-queries	Reduce redundance and improve reusability in one query
Persisted logic	Temporary logic
Need to maintain (CREATE / DROP)	No maintenance (auto cleanup)



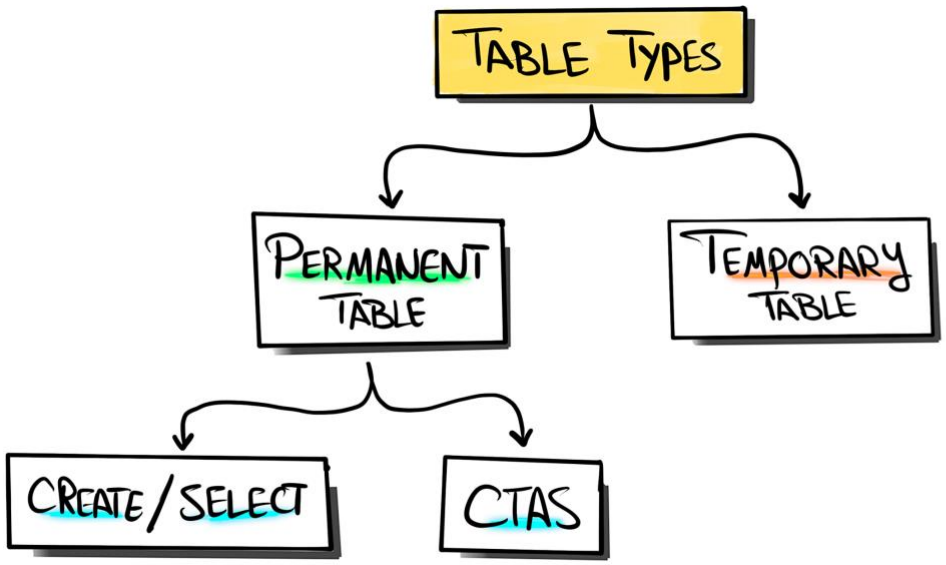
* If a table or view is created without specifying a schema, it defaults to the DBO (Database Owner) – para definir é só fazer "CREATE VIEW Schema.ViewName AS ("

```
CREATE VIEW Sales.V_Order_Details_USA AS (
	SELECT
		O.OrderID,
		O.OrderDate,
		O.Quantity,
		O.Sales,
		P.Product,
		P.Price,
		COALESCE(C.FirstName, '') + ' ' + COALESCE(C.LastName, '') AS
CustomerName,
		C.Country AS CustomerCountry,
		COALESCE(E.FirstName, '') + ' ' + COALESCE(E.LastName, '') AS
EmployeeName,
		E.Department
	FROM Sales.Orders O
	LEFT JOIN Sales.Products P
	ON O.ProductID = P.ProductID
	LEFT JOIN Sales.Customers C
	ON O.CustomerID = C.CustomerID
	LEFT JOIN Sales.Employees E
	ON O.SalesPersonID = E.EmployeeID
	WHERE C.Country = 'USA' -- row security
)
```

```
SELECT * FROM Sales.V_Order_Details_USA
```

	OrderID	OrderDate	Quantity	Sales	Product	Price	CustomerName	CustomerCountry	EmployeeName	Department
1	1	2025-01-01	1	10	Bottle	10	Kevin Brown	USA	Mary	Sales
2	2	2025-01-05	1	15	Tire	15	Mary	USA	Mary	Sales
3	5	2025-02-01	1	25	Caps	25	Kevin Brown	USA	Carol Baker	Sales
4	6	2025-02-05	2	50	Caps	25	Mary	USA	Carol Baker	Sales
5	9	2025-03-10	2	20	Bottle	10	Kevin Brown	USA	Mary	Sales
6	10	2025-03-15	0	60	Tire	15	Mary	USA	Carol Baker	Sales

f. CTAS table & temp tables



- CREATE / INSERT syntax:

```
CREATE TABLE TableName ( -- DDL statement
      ID INT,
      Name VARCHAR(50)
);

INSERT INTO TableName (ID, Name) -- insert statement
VALUES (1, 'Frank');
```

- CTAS (Create Table As Select): Comando para criar uma nova tabela a partir do resultado de uma consulta SELECT. Used in place of slow views, to optimize performance and to create a snapshot of the table to do data quality analysis

```
-- MySQL, Postgres and Oracle syntax
CREATE TABLE TableName AS -- DDL
Statement
SELECT ... -- query
FROM ...
WHERE ...;
```

```
-- SQL Server syntax
SELECT ...
INTO TableName
FROM ...
WHERE ...;
```

- Temporary table: Stores intermediate results in temporary storage within the database during the session (time between connecting to and disconnecting from the database). Once it ends, database drops all temporary tables. Used to intermediate results

```
-- MySQL, Postgres and Oracle syntax
CREATE TEMPORARY TABLE TableName AS --
DDL Statement
SELECT ... -- query
FROM ...
WHERE ...;
```

```
-- SQL Server syntax
SELECT ...
INTO #TableName
FROM ...
WHERE ...;
```

g. Stored procedure

SQL statements can be placed inside a stored procedure or executed from Python code.

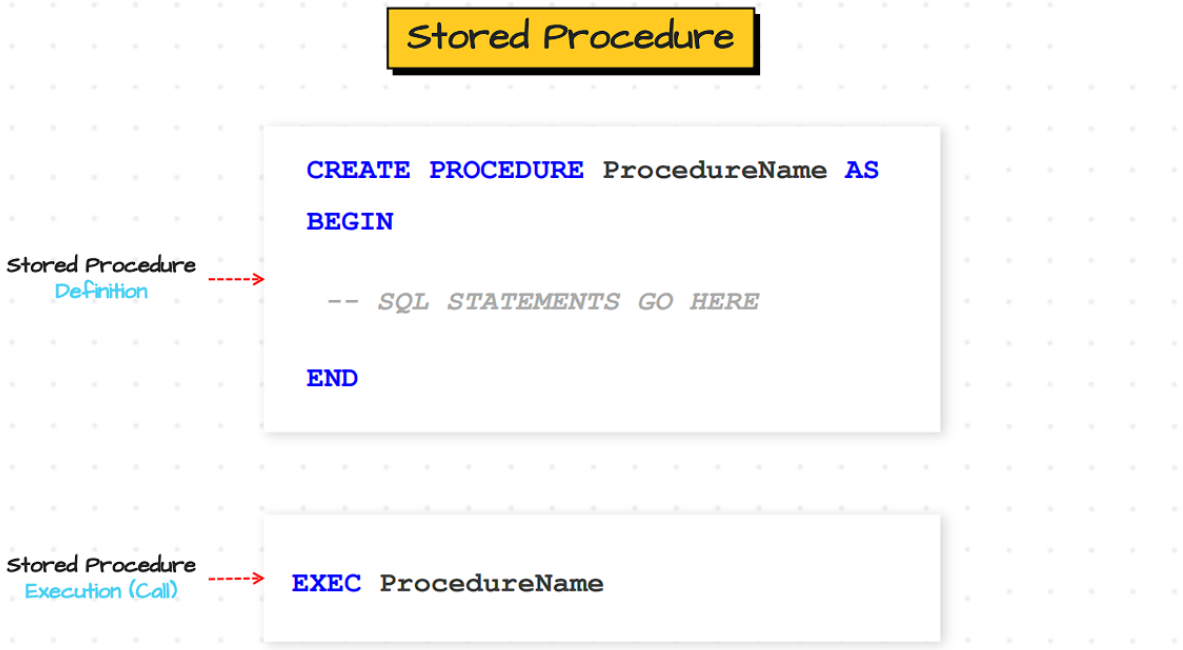
Python's disadvantages:

- If Python runs on a different server, you must build a connection between your application server and the database server (which can slightly reduce performance)
- All the scripts stored inside the stored procedure in the database are pre-compiled

Python's advantages:

- Flexible Python codes
- Great version control
- Easier to implement complex requirements

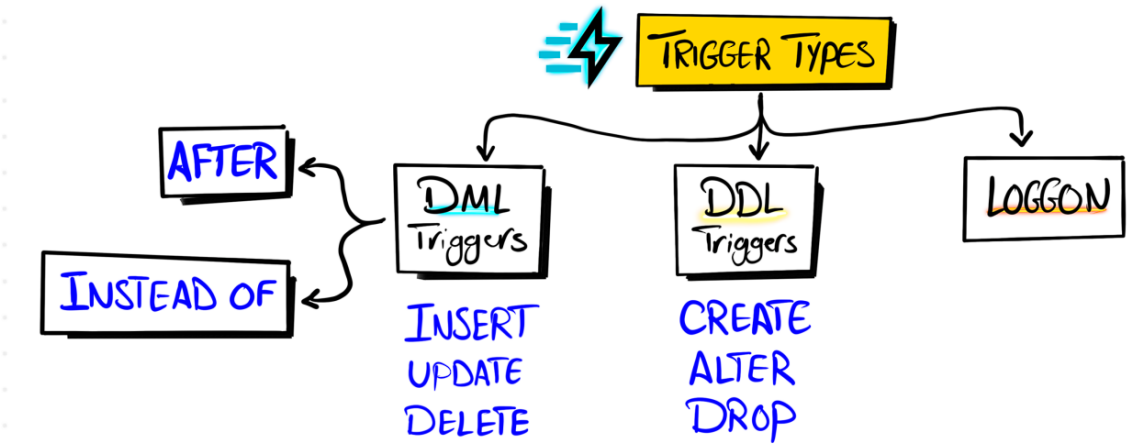
Advice: don't build projects using stored procedures. "I would never recommend you to use stored procedure if you have the possibility to have your code in Python, because I saw a lot of projects using stored procedure and most of them ends in chaos. It is really hard to debug, to test, it's catastrophic. So really don't use in your projects any stored procedures – especially if you have a big project and you have a lot of data and tables and so on, you can manage everything perfectly using Python with platforms like data bricks or snowflakes […]. If you have a small project, it's fine to stay with the stored procedure, but never build a big project using it, because it will never work"



```sql
CREATE PROCEDURE GetCustomerSummary @Country NVARCHAR(50) = 'USA' -- default
parameter
AS
BEGIN
	BEGIN TRY
		-- Declare variables
		DECLARE @TotalCustomers INT, @AverageScore FLOAT;

		-- Prepare data
		IF EXISTS (
			SELECT 1
			FROM Sales.Customers
			WHERE Score IS NULL AND Country = @Country
		)
		BEGIN
			PRINT 'Updating NULL scores to 0';
			UPDATE Sales.Customers
			SET Score = 0
			WHERE Score IS NULL AND Country = @Country;
		END

		ELSE
		BEGIN
			PRINT 'No NULL scores found for ' + @Country;
		END;

		-- Generate summary reports
		SELECT
			@TotalCustomers = COUNT(*),
			@AverageScore = AVG(Score) -- flow control
		FROM Sales.Customers
		WHERE Country = @Country;

		PRINT 'Total customers from ' + @Country + ': ' +
CAST(@TotalCustomers AS NVARCHAR);
		PRINT 'Average score from ' + @Country + ': ' + CAST(@AverageScore
AS NVARCHAR);

		SELECT
			COUNT(OrderID) AS TotalCustomers,
			SUM(Sales) AS TotalSales
		FROM Sales.Orders O
		JOIN Sales.Customers C
		ON O.CustomerID = C.CustomerID
		WHERE C.Country = @Country;
	END TRY

	-- Error handling
	BEGIN CATCH
		PRINT 'An error occured.';
		PRINT 'Error message: ' + ERROR_MESSAGE();
		PRINT 'Error number: ' + CAST(ERROR_NUMBER() AS NVARCHAR);
		PRINT 'Error line: ' + CAST(ERROR_LINE() AS NVARCHAR);
		PRINT 'Error procedure: ' + ERROR_PROCEDURE();
	END CATCH
END
GO

EXEC GetCustomerSummary @Country = 'Germany';
```

h. Triggers

Special stored procedure (set of statements) that automatically runs in response to a specific event on a table or view



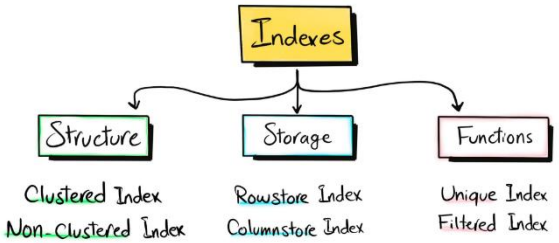
* We're going to focus on the DML triggers. AFTER runs after the event and INSTEAD OF runs during the event

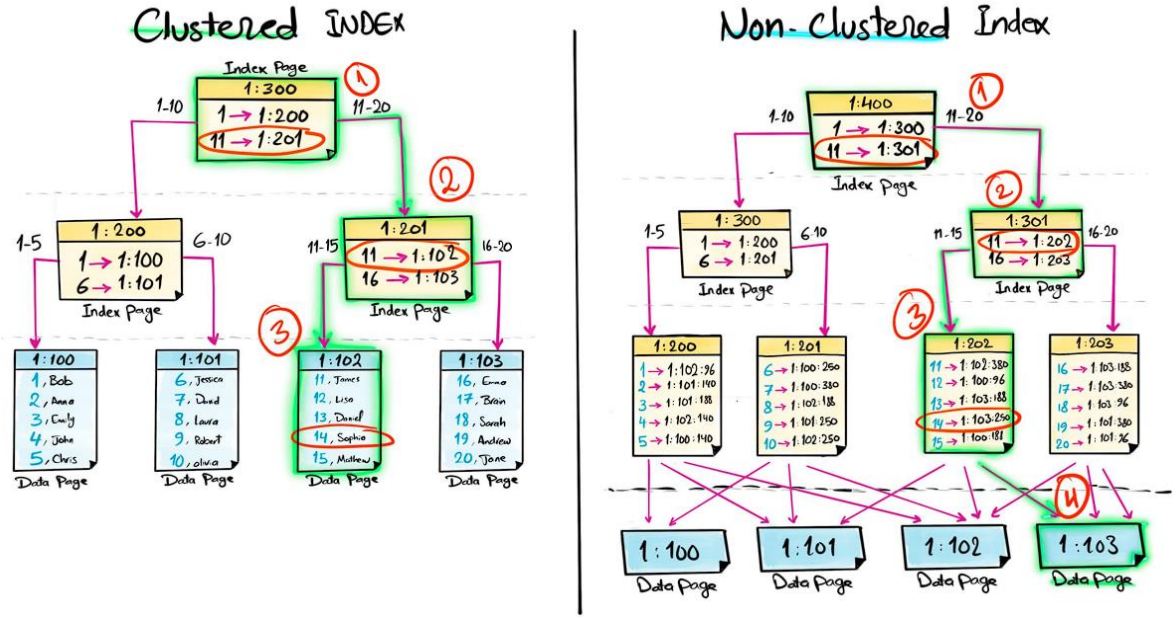
```
CREATE TABLE Sales.EmployeeLogs ( -- table to store log information
	LogID INT IDENTITY(1, 1) PRIMARY KEY,
	EmployeeID INT,
	LogMessage VARCHAR(255),
	LogDate DATE
)
GO

CREATE TRIGGER trg_AfterInsertEmployee ON Sales.Employees -- table on which the
trigger is built in
AFTER INSERT
AS
BEGIN
	INSERT INTO Sales.EmployeeLogs (EmployeeID, LogMessage, LogDate)
	SELECT
		EmployeeID,
		'New employee added: ' + CAST(EmployeeID AS VARCHAR),
		GETDATE()
	FROM INSERTED -- virtual table that holds a copy of the rows that are being
inserted into the target table
END
GO

INSERT INTO Sales.Employees (EmployeeID, FirstName, LastName, Department,
BirthDate, Gender, Salary, ManagerID)
VALUES (6, 'Maria', 'Rosa', 'HR', '1959-12-27', 'F', 80000, 3);

SELECT * FROM Sales.EmployeeLogs;
```

	LogID	EmployeeID	LogMessage	LogDate
1	1	6	New employee added: 6	2025-09-08

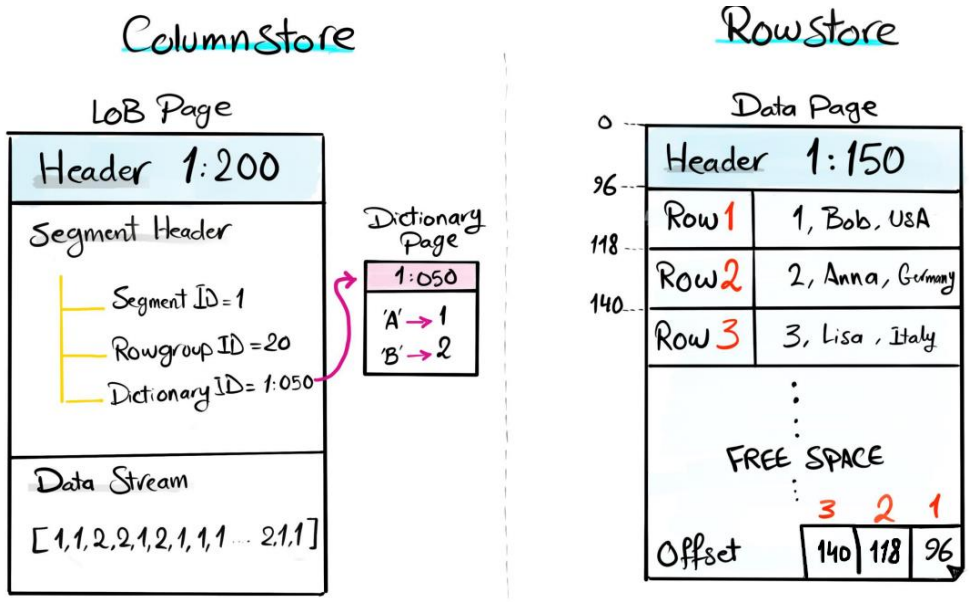
9. Performance optimization

a. Indexes





	Clustered Index	Non-Clustered Index
Definition	Physically sorts and stores rows	Separate structure with pointers to the data
Numebr of Indexes	One Index per Table	Multiple indexes are allowed
Read Performance	Faster	Slower
Write Performance	Slower, due to potential data row reordering	Faster, since physical data order is unaffected
Storage Efficiency	More storage-efficient	Requires additional storage space
Use Case	• Unique Column • Not frequently modified Column • Improve range query performance	• Columns frequently used in search conditions and joins • Exact match queries

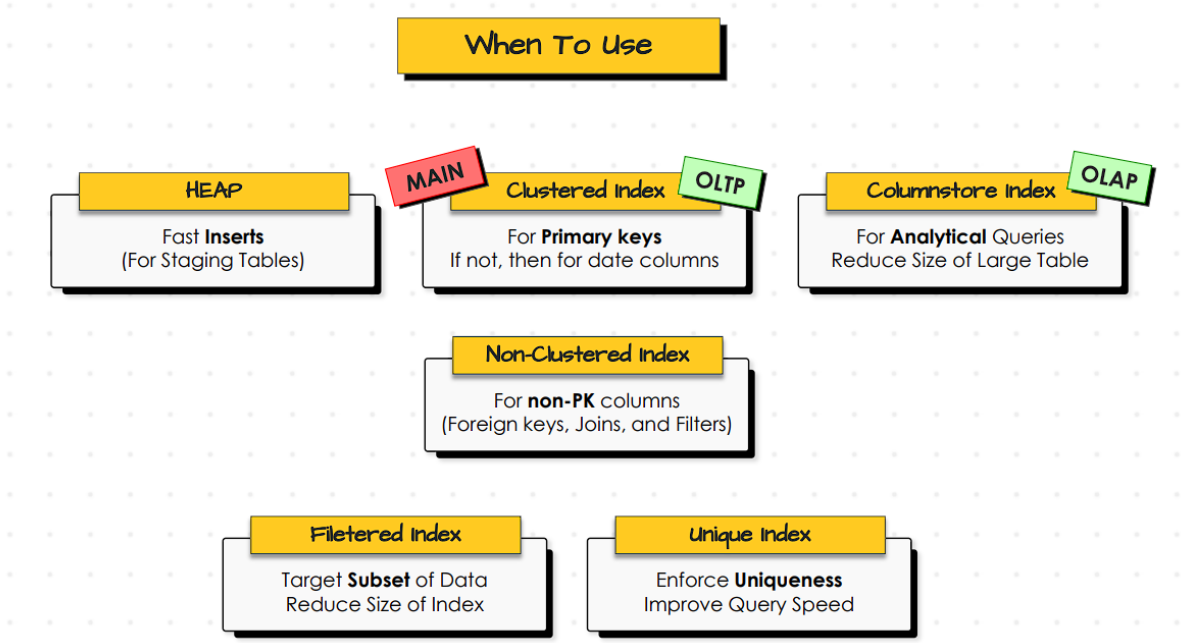


	Rowstore Index	Columnstore Index
Definition	Organizes and stores data **row by row**	Organizes and stores data **column by column**
Storage Efficiency	**Less efficient in storage**	**Highly efficient with Compression**
Read/Write Optimization	**Fair speed for read & write operations**	**Fast read performance** **Slow write performance**
I/O Efficiency	**Lower (retrieves all columns)**	**Higher (retrieves specific columns)**
Best for ..	**OLTP (Transactional)** commerce, banking, Financial systems, order processing	**OLAP (Analytical)** Data Warehouse, Business intelligence, Reporting, Analytics
Use Case	**- High-frequency transaction applications** **- Quick access to complete records**	**- Big Data Analytics** **- Scanning of large datasets** **- Fast aggregation**

```
CREATE INDEX IndexName -- default: (NON-UNIQUE) NONCLUSTERED (ROWSTORE)
ON TableName (Columns) -- columns (em ROWSTORE) = chaves de ordenação do índice

CREATE UNIQUE NONCLUSTERED COLUMNSTORE INDEX IndexName
ON TableName (Columns) -- columns (em NONCLUSTERED COLUMNSTORE) = colunas
incluídas no novo armazenamento

CREATE CLUSTERED COLUMNSTORE INDEX IndexName
ON TableName
-- In the CLUSTERED INDEX, the base table is restructured
-- CLUSTERED COLUMNSTORE INDEX don't allow columns and can't be UNIQUE, because
all columns are always included automatically in the new storage
```

- Filtered INDEX: includes only rows meeting the specified conditions

```
CREATE UNIQUE NONCLUSTERED INDEX IndexName -- can't create a filtered INDEX on a
CLUSTERED INDEX, nor on a COLUMNSTORE INDEX
ON TableName (Columns)
WHERE [Condition]
```



✓ INDEX management & monitoring:

1. Monitor index usage and duplicates

To optimize performance and storage usage, it's important to drop unused and duplicated indexes

```
-- To verify index usage
SELECT
	TBL.name AS TableName,
	IDX.name AS IndexName,
	IDX.type_desc AS IndexType,
	IDX.is_primary_key AS IsPrimaryKey,
	IDX.is_unique AS IsUnique,
	IDX.is_disabled AS IsDisabled,
	S.user_seeks AS UserSeeks,
	S.user_scans AS UserScans,
	S.user_lookups AS UserLookups,
	S.user_updates AS UserUpdates,
	COALESCE(S.last_user_seek, S.last_user_scan) AS LastUpdate
FROM sys.indexes IDX -- dados dos indexes
JOIN sys.tables TBL -- para manter apenas os indexes das tabelas
ON IDX.object_id = TBL.object_id
LEFT JOIN sys.dm_db_index_usage_stats S -- para verificar os usos
ON IDX.object_id = S.object_id AND IDX.index_id = S.index_id
ORDER BY TBL.name, IDX.name
```

```sql
-- To verify duplicated indexes
SELECT TBL.name AS TableName,
       COL.name AS IndexColumn,
       IDX.name AS IndexName,
       IDX.type_desc AS IndexType,
       COUNT(*) OVER (PARTITION BY TBL.name, COL.name) RepetitionCount
FROM sys.indexes IDX
JOIN sys.tables TBL
ON IDX.object_id = TBL.object_id
JOIN sys.index_columns IC -- to find the columns involved in the index
ON IDX.object_id = IC.object_id AND IDX.index_id = IC.index_id
JOIN sys.columns COL -- to get the full name of the columns
ON IC.object_id = COL.object_id AND IC.column_id = COL.column_id
ORDER BY RepetitionCount DESC
```

2. Monitor missing indexes

```sql
SELECT * FROM sys.dm_db_missing_index_details -- recommendations from the database
```

Advice: evaluate the recommendations before creating any index

3. Update statistics

Database engines usually use statistics to understand which index should be used for our query – for correct decisions, the statistics must be updated

```sql
-- To verify modification informations
SELECT
       SCHEMA_NAME(T.schema_id) AS SchemaName,
       T.name AS TableName,
       S.name AS StatisticName,
       SP.last_updated AS LastUpdate,
       DATEDIFF(DAY, SP.last_updated, GETDATE()) AS DaysFromLastUpdate,
       SP.rows AS Rows,
       SP.modification_counter AS ModificationsSinceLastUpdate
FROM sys.stats S -- list of statistics inside the database
JOIN sys.tables T
ON S.object_id = T.object_id
CROSS APPLY sys.dm_db_stats_properties(S.object_id, S.stats_id) AS SP -- to
retrieve detailed properties of each statistic
ORDER BY SP.modification_counter DESC

-- Copy and paste the StatisticName for updating it individually
UPDATE STATISTICS Sales.Employees PK_employees
-- Or update all statistics of a selected table
UPDATE STATISTICS Sales.Customers
-- Or update all statistics of the whole database (this might take a long time!)
EXEC sp_updatestats
```

Advice: update statistics after migrating data and regularly on weekends

4. Monitor fragmentations (unused spaces in data pages or out of order pages)

```
SELECT * -- the most important information is the avg_fragmentation_in_percent,
which shows how out of order the index pages are (0 = no fragmentation)
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')

-- To verify which object and index are fragmented
SELECT
      TBL.name AS TableName,
      IDX.name AS IndexName,
      S.avg_fragmentation_in_percent AS AvgFragmentation,
      S.page_count AS PageCount
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') S
INNER JOIN sys.tables TBL
ON S.object_id = TBL.object_id
INNER JOIN sys.indexes IDX
ON S.object_id = IDX.object_id
AND S.index_id = IDX.index_id
ORDER BY S.avg_fragmentation_in_percent DESC
```

Advice: if the avg_fragmentation_in_percent is below 10%, no action is needed; if it's between 10-30%, reorganize; if it's above 30%, rebuild the whole index

- Reorganize → Defragments leaf nodes to keep them sorted (light operation)

```
-- Copy and paste the IndexName and the TableName where the index exists
ALTER INDEX PK_customers ON Sales.Customers REORGANIZE
```

- Rebuild → Recreates the index from scratch (heavy operation)

```
-- Copy and paste the IndexName and the TableName where the index exists
ALTER INDEX PK_customers ON Sales.Customers REBUILD
```

- ✓ Execution plan: roadmap generated by a database on how it will execute your query – helps implementing better indexes and improving indexing strategy

- Ferramentas de Análise de Execução:
  - Exibir Plano de Execução Estimado (Ctrl+L): predicts the execution plan without actually running the query
  - Incluir Plano de Execução Real (Ctrl+M): shows the execution plan as it occurred after running the query
  - Estatísticas de Consultas Dinâmicas: shows the real-time execution flow as the query runs

Not matching predictions and the actual execution plan indicates issues like inaccurate statistics or outdated indexes, leading to poor performance

Advice: after creating a new index, check the execution plan to see if the query uses it

- Operadores de Acesso a Dados:
  - Table Scan: reads the entire table (can slow query performance) – HEAP structure
  - Index Scan: scans all data in an index to find matching rows
  - Index Seek: a targeted search within an index that retrieves only the rows that match the search criteria
  - …

- Join algorithms
  - Nested Loops: compares tables row by row (best for small tables)
  - Hash Match: matches rows using a hash table (best for large tables)
  - Merge Join: merge two sorted tables (efficient when both tables are sorted)
  - …

- ✓ SQL Hints: commands you add to a query to force the database to run it in a specific way for better performance

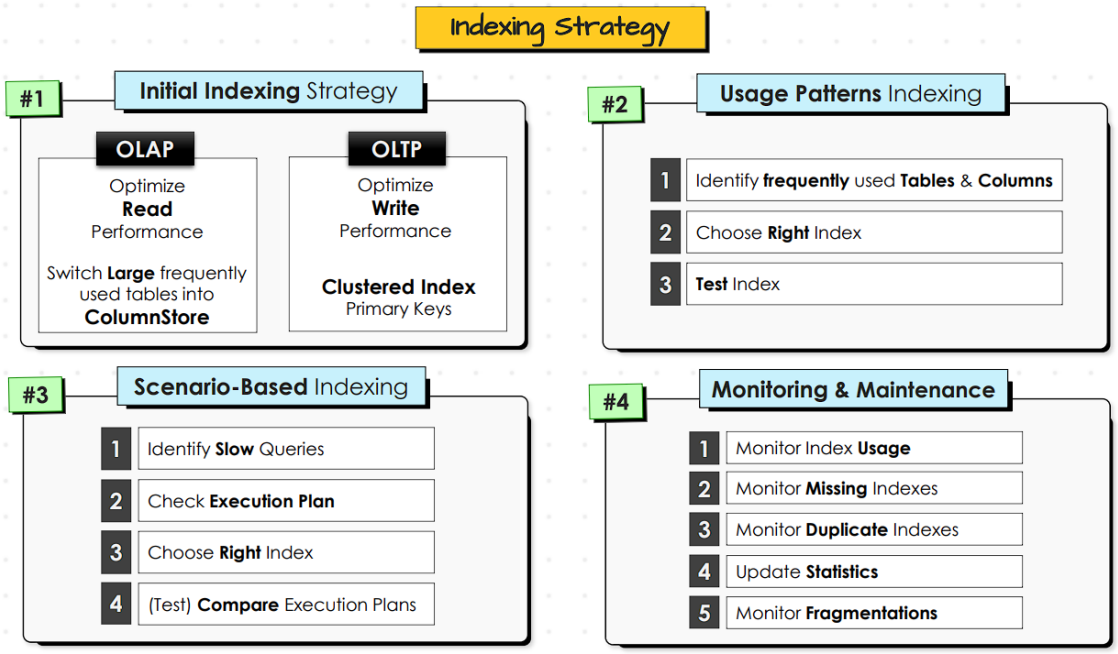
- Examples:

```
SELECT
      O.Sales,
      C.Country
FROM Sales.Orders O
LEFT JOIN Sales.Customers C
ON O.CustomerID = C.CustomerID
OPTION (HASH JOIN); -- query hint

SELECT
      O.Sales,
      C.Country
FROM Sales.Orders O
LEFT JOIN Sales.Customers C WITH (FORCESEEK) -- table hint
ON O.CustomerID = C.CustomerID;

SELECT
      O.Sales,
      C.Country
FROM Sales.Orders O
LEFT JOIN Sales.Customers C WITH (INDEX([PK_customers])) -- specific index
ON O.CustomerID = C.CustomerID;
```

Powerful, but be careful (advices): test hints in each project environment (Development, Production, etc.) as performance may vary; don't use hints as a permanent fix for your queries – they are quick workarounds, not solutions –, you still have to find the cause and fix it

✓ Indexing strategy: for each SQL data project we have to make sure that we create a clear guidance about the index strategy, and everyone in the team have to commit and follow it to make sure that each index created in the project fulfill a purpose. Otherwise, there will be a lot of unused indexes, redundancy, waste of storage, and the whole system of your project is going to be slower and worse

- Golden rule: avoid over indexing ("less is more") – indexes slow down write performance (each time you insert, update or delete data, the database updates their indexes); excess of indexes increases planning time of the execution plan and cause it to choose a suboptimal plan



- Initial strategy: when starting a new SQL project, define objectives clearly. To define the goal of your indexing strategy, you have to understand your system. There are mainly two types of databases:
  - OLAP (Online Analytical Processing): used for data analytics, e.g. Data Warehouse – in the backend, we Extract data from multiple sources, Transform it and Load it into one big storage (ETL process); and in the frontend we have reports and dashboards, where the data is summarized, aggregated and presented for the end user, for analysis. Write operations are big, but used less frequently; and to generate the reports, there will be heavy reading on the DWH database, with huge queries accessing the database. So, usually the main objective is to read
  - OLTP (Online Transaction Processing): used for transactions, e.g. e-commerce, finance, banking, etc. – at the backend the data is stored in a database and on the frontend we have applications for the end users to interact with the database, causing a mix of read and write operations. Usually the main pain point is the write operation

- Usage patterns: Identify frequently used tables and columns. GPT prompt:

“Analyze the following SQL queries and generate a report on table and column usage statistics. For each table, provide:

1. The total number of times the table is used across all queries;
2. A breakdown of each column in the table, showing the number of times each column appears and the primary purpose of the column’s usage (e.g., filtering, joining, grouping, aggregating).

Sort the tables in descending order based on their total usage”

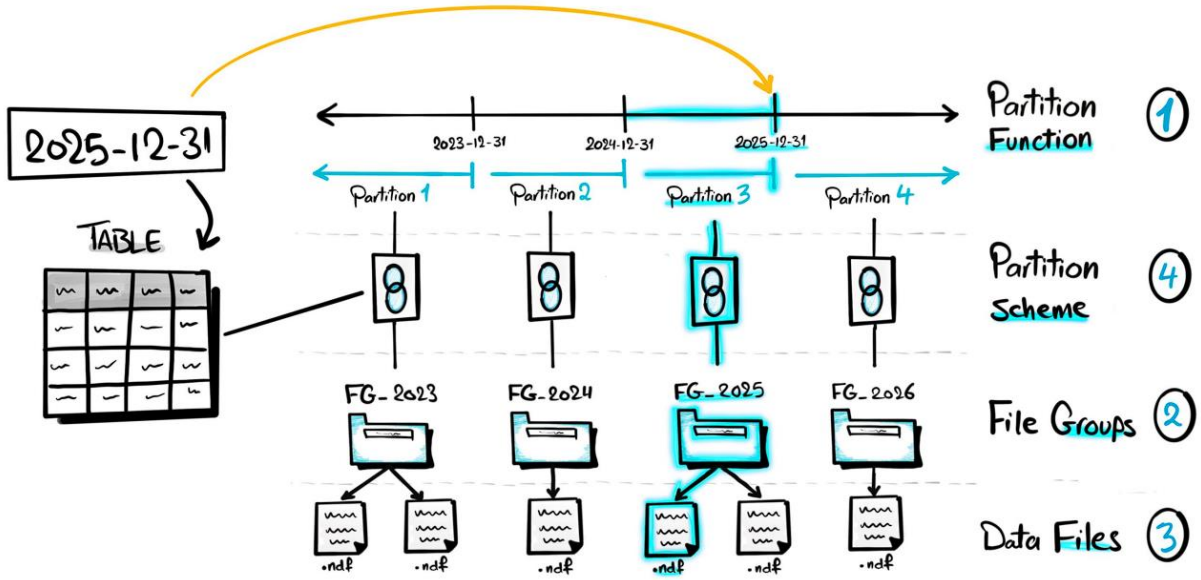
Then choose the right index type and test it.

- Scenario-based indexing: identify slow queries (by analysis on the logs or reported by users), check their execution plan to choose the right index and test comparing execution plans

- Monitoring & maintenance: “usually for monitoring I build an automated dashboard in PowerBI or Tableau [...] or you can buy advanced tools to do it”

b. Partitions

Technique to optimize performance that physically divides a table into smaller partitions while still being treated as a single logical table. By changing a partition (INSERT, UPDATE, DELETE), only its indexes are updated. So, it improves significantly the performance of your table whether if whether you are reading or writing data. We usually partition by a date (e.g., year). Modern databases usually support parallel processing (e.g., Azure Synapse), so if there's the infrastructure the database engine can process each partition independently and parallelly.



1. Create partition function: logical definition of how to divide the data into partitions (based on partition key, like dates, regions, etc.)

```
CREATE PARTITION FUNCTION PartitionName (DataType)
AS PartitionFunctionType Side (LEFT inclui o limite na esquerda, RIGHT na
direita) Boundaries

CREATE PARTITION FUNCTION PartitionByYear (DATE)
AS RANGE LEFT FOR VALUES ('2023-12-31', '2024-12-31') -- partições = limites + 1
```

Advice: check all partition functions created in the database before creating a new one

```
SELECT
	name,
	function_id,
	type,
	type_desc,
	boundary_value_on_right
FROM sys.partition_functions
```

	name	function_id	type	type_desc	boundary_value_on_right
1	PartitionByYear	65536	R	RANGE	0

2. Create file groups: logical container of one or more data files to help organize partitions. Like folders to insert files. For each partition, we create a file group

```
ALTER DATABASE SalesDB ADD FILEGROUP FG_2023;
ALTER DATABASE SalesDB ADD FILEGROUP FG_2024;
ALTER DATABASE SalesDB ADD FILEGROUP FG_2025;
```

Check (as usual, after creating, check if everything is correct):

```
SELECT
	name, -- PRIMARY: default file group where all database objects are stored
	data_space_id,
	type_desc,
	is_default
FROM sys.filegroups
WHERE type = 'FG'
```

	name	data_space_id	type_desc	is_default
1	PRIMARY	1	ROWS_FILEGROUP	1
2	FG_2023	2	ROWS_FILEGROUP	0
3	FG_2024	3	ROWS_FILEGROUP	0
4	FG_2025	4	ROWS_FILEGROUP	0

3. Create data files: physical storage in the database (contain the actual data)

```
ALTER DATABASE SalesDB
ADD FILE (
	NAME = P_2023, -- logical name
	FILENAME = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2023.ndf' -- physical name (place
partition files in the correct folder - in real projects, you have to ask the
database administrators about the exact location)
)
TO FILEGROUP FG_2023;

ALTER DATABASE SalesDB
ADD FILE (
	NAME = P_2024, -- logical name
	FILENAME = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2024.ndf' -- physical name (place
partition files in the correct folder - in real projects, you have to ask the
database administrators about the exact location)
)
TO FILEGROUP FG_2024;

ALTER DATABASE SalesDB
ADD FILE (
	NAME = P_2025, -- logical name
	FILENAME = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2025.ndf' -- physical name (place
partition files in the correct folder - in real projects, you have to ask the
database administrators about the exact location)
)
TO FILEGROUP FG_2025;
```

Check:

```
SELECT
	FG.name AS FileGroupName,
	MF.name AS LogicalFileName,
	MF.physical_name AS PhysicalFilePath,
	MF.size / 128 AS SizeMB
FROM sys.filegroups FG
JOIN sys.master_files MF
ON FG.data_space_id = MF.data_space_id
WHERE MF.database_id = DB_ID('SalesDB')
```

	FileGroupName	LogicalFileName	PhysicalFilePath	SizeMB
1	PRIMARY	SalesDB	C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\SalesDB.mdf	8
2	FG_2023	P_2023	C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2023.ndf	8
3	FG_2024	P_2024	C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2024.ndf	8
4	FG_2025	P_2025	C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\P_2025.ndf	8

4. Create partition scheme: connection between partitions and prepared file groups (define which partition corresponds to each file group)

```
CREATE PARTITION SCHEME SchemePartitionByYear
AS PARTITION PartitionByYear
TO (FG_2023, FG_2024, FG_2025) -- order the file groups according to the result of the function's partitions
```

Check:

```
SELECT
	PS.name AS PartitionSchemaName,
	PF.name AS PartitionFunctionName,
	DS.destination_id AS PartitionNumber,
	FG.name AS FileGroupName
FROM sys.partition_schemes PS
JOIN sys.partition_functions PF ON PS.function_id = PF.function_id
JOIN sys.destination_data_spaces DS ON PS.data_space_id = DS.partition_scheme_id
JOIN sys.filegroups FG ON DS.data_space_id = FG.data_space_id
```

	PartitionSchemaName	PartitionFunctionName	PartitionNumber	FileGroupName
1	SchemePartitionByYear	PartitionByYear	1	FG_2023
2	SchemePartitionByYear	PartitionByYear	2	FG_2024
3	SchemePartitionByYear	PartitionByYear	3	FG_2025

5. With all the logic layers prepared, we create a partitioned table

```
CREATE TABLE Sales.Orders_Partitioned (
	OrderID INT,
	OrderDate DATE,
	Sales INT
) ON SchemePartitionByYear (OrderDate)
```

6. Insert data into the partitioned table

```
INSERT INTO Sales.Orders_Partitioned
VALUES (1, '2023-05-15', 100);

SELECT * FROM Sales.Orders_Partitioned
```

	OrderID	OrderDate	Sales
1	1	2023-05-15	100

Check:

```
SELECT
      P.partition_number AS PartitionNumber,
      F.name AS PartitionFileGroup,
      P.rows AS NumberOfRows
FROM sys.partitions P
JOIN sys.destination_data_spaces DDS ON P.partition_number = DDS.destination_id
JOIN sys.filegroups F ON DDS.data_space_id = F.data_space_id
WHERE OBJECT_NAME(P.object_id) = 'Orders_Partitioned'
```

	PartitionNumber	PartitionFileGroup	NumberOfRows
1	1	FG_2023	1
2	2	FG_2024	0
3	3	FG_2025	0

✓ To analyze the benefit of the partition, create a mirror table without the partition, then write a query for both and compare the execution plans

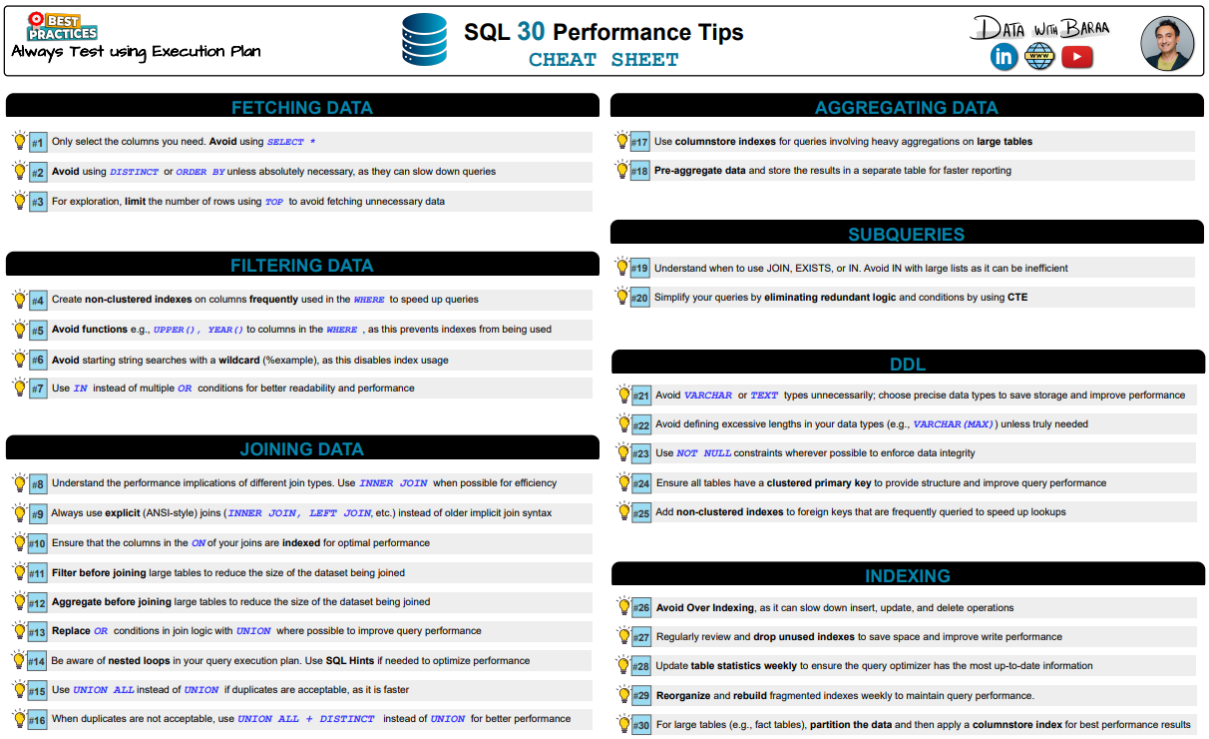
```
SELECT *
INTO Sales.Orders_NoPartition
FROM Sales.Orders_Partitioned
```

```
SELECT *
FROM Sales.Orders_Partitioned
WHERE OrderDate = '2023-05-15'
```

```
SELECT *
FROM Sales.Orders_NoPartition
WHERE OrderDate = '2023-05-15'
```

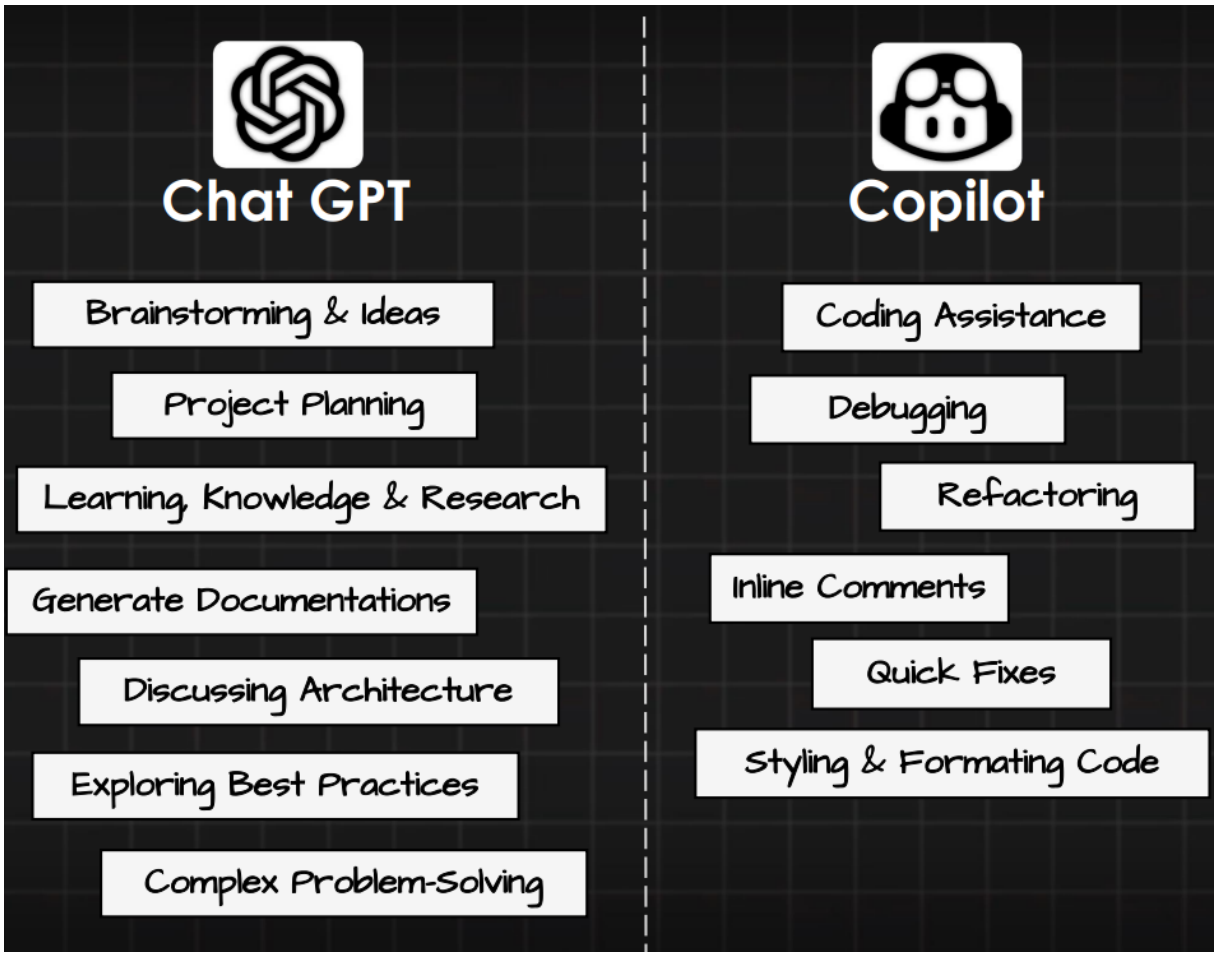
c. Performance tips

- Golden rule: for small to medium tables (up to hundreds of thousands rows), the query optimizer may react similarly to different query styles if you're following the best practices, so always check the execution plan to confirm performance improvements when optimizing your query – if there's no improvement, then just focus on readability



* #7, #13: avoid OR operator in WHERE and JOIN, because it avoid indexes and creates loops; #12: Always avoid using correlated subqueries, because SQL execute aggregations for every row; #14: use HASH JOIN instead

10. AI & SQL


Chat GPT
Brainstorming & Ideas
Project Planning
Learning, Knowledge & Research
Generate Documentations
Discussing Architecture
Exploring Best Practices
Complex Problem-Solving
Copilot
Coding Assistance
Debugging
Refactoring
Inline Comments
Quick Fixes
Styling & Formating Code